LLMs Get Lost In Multi-Turn Conversation

Philippe Laban[*◇] **Hiroaki Hayashi**[*♣] **Yingbo Zhou**[♣] **Jennifer Neville**[◇]
[◇]Microsoft Research [♣]Salesforce Research
{plaban,jenneville}@microsoft.com
{hiroakihayashi,yingbo.zhou}@salesforce.com

Abstract

Large Language Models (LLMs) are conversational interfaces. As such, LLMs have the potential to assist their users not only when they can fully specify the task at hand, but also to help them define, explore, and refine what they need through multi-turn conversational exchange. Although analysis of LLM conversation logs has confirmed that underspecification occurs frequently in user instructions, LLM evaluation has predominantly focused on the single-turn, fully-specified instruction setting. In this work, we perform large-scale simulation experiments to compare LLM performance in single- and multi-turn settings. Our experiments confirm that all the top open- and closed-weight LLMs we test exhibit significantly lower performance in multi-turn conversations than single-turn, with an average drop of 39% across six generation tasks. Analysis of 200,000+ simulated conversations decomposes the performance degradation into two components: a minor loss in aptitude and a significant increase in unreliability. We find that LLMs often make assumptions in early turns and prematurely attempt to generate final solutions, on which they overly rely. In simpler terms, we discover that **when LLMs take a wrong turn in a conversation, they get lost and do not recover**.

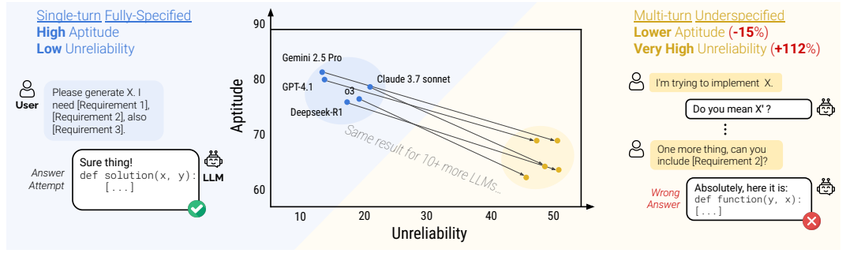


Figure 1: Our simulated conversations for 6 generation tasks on the 15 LLMs observe a major performance drop in multi-turn settings (-39%), explained by some loss in Aptitude, and large loss in Reliability.

1 Introduction

Today's large language models (LLMs) function as conversational interfaces (*e.g.*, ChatGPT, Gemini, Claude), enabling users to interact with the LLM through multiple conversation turns. Such interaction promises to help users not only when they know what they need (i.e., they can fully specify their requirements in an instruction), but also when they don't. In such cases, users might start with an underspecified instruction and further clarify their needs through turn interactions. Though studies of LLM conversation logs have confirmed that underspecification in user instructions is prevalent (Herlihy et al., 2024), LLM systems are typically evaluated in single-turn, fully-specified settings.

Even though a growing body of work proposes to evaluate LLMs in a **multi-turn** fashion, we identify in our review (Section 2) that most prior work treats the conversation as *episodic*: conversation turns

[*]Equal Contributions

大语言模型在多轮对话中迷失方向

PhilippeLaban[*◇] **Hiroaki Hayashi**[*♣] **Yingbo Zhou**[♣] **Jennifer Neville**[◇◇]微软研究院[♣]Salesforce研究院{plaban,jenneville}@microsoft.com
{hiroakihayashi,yingbo.zhou}@salesforce.com

摘要

大语言模型是对话界面。因此，大语言模型不仅能在用户完全指定当前任务时提供协助，还能通过多轮对话交流，帮助用户定义、探索和细化其需求。尽管对大语言模型对话日志的分析已证实，用户指令中频繁出现欠明确性，但大语言模型的评估主要集中于单轮、完全指定指令设置。在本工作中，我们进行了大规模模拟实验，以比较大语言模型在单轮与多轮设置下的性能。我们的实验证实，我们测试的所有顶尖开源与闭源权重大语言模型，在多轮对话中的性能均显著低于单轮对话，在六项生成任务中平均下降39%。对200,000+ 个模拟对话的分析将性能下降分解为两个部分：轻微的能力损失和显著增加的不可靠性。我们发现，大语言模型常在早期轮次做出假设，并过早尝试生成最终解决方案，且对此过度依赖。简而言之，我们发现**当大语言模型在对话中走错一步，它们就会迷失方向且无法恢复**。

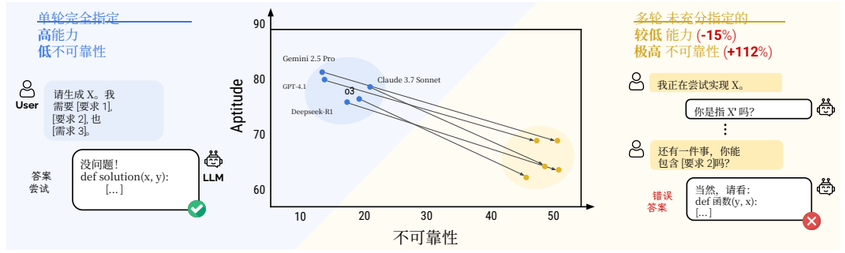


图1：我们在15个大语言模型上对6项生成任务进行的模拟对话观察到一个显著的多轮对话场景性能下降（-39%），这可由部分能力损失以及巨大的可靠性损失来解释。

1 引言

当今的大语言模型（LLMs）作为对话界面运行（例如，ChatGPT、Gemini、Claude），使用户能够通过多个对话轮次与人工智能交互。这种交互不仅承诺在用户清楚自身需求时提供帮助（即他们可以在一条指令中完全指定其要求），而且在用户不清楚时也能发挥作用。在这种情况下，用户可能从一个未充分指定的指令开始，并通过轮次交互进一步澄清其需求。尽管对大语言模型对话日志的研究已证实用户指令中的欠明确性普遍存在（Herlihy等人，2024），但大语言模型系统通常在单轮、完全指定的设置中进行评估。

尽管越来越多的研究提出以**多轮**方式评估大语言模型，但我们在综述（第2节）中发现，大多数先前工作将对话视为片段式：对话轮次

[*]同等贡献

might relate to each other, but the conversation can effectively be decomposed as an array of subtasks that can be evaluated in isolation. We argue that episodic tasks move away from what is prevalent in human conversation: underspecification (Zipf, 1949; Herlihy et al., 2024).

In this work, we close this gap by creating a simulation environment for multi-turn underspecified conversations – sharded simulation – that leverages existing instructions from high-quality single-turn benchmarks. At a high level, the sharding process we propose transforms existing single-turn instructions into *sharded instructions*, a set of smaller instructions that jointly deliver the same information as the original instruction. Sharded simulation then ensures that each turn of conversation reveals at most one shard of information per conversation turn, enforcing that the instruction is gradually revealed through the conversation.

On the set of tasks that we experimented on, we observed that models engaged in multi-turn underspecified conversations achieved an average performance of 65%–a 25-point drop from single-turn performances of 90% when they receive the entire instruction at the beginning of the conversation. Notably, we observe this drop in performance even in two-turn conversations, and across all LLMs we test, from small open-weights (LLama3.1-8B-Instruct) to state-of-the-art (Gemini 2.5 Pro).

Furthermore, we decompose the performance degradation into two components: (1) loss in aptitude, and (2) increase in unreliability. We find that in single-turn, LLMs with higher aptitude tend to be more reliable (*e.g.*, GPT-4.1). On the other hand, all LLMs exhibit very high unreliability in multi-turn settings, regardless of aptitude. We refer to this as the *lost in conversation phenomenon*: when LLMs take a wrong turn in multi-turn conversation, they get lost and do not recover.

We investigate several explanations for this effect and show that the LLMs tend to (1) generate overly verbose responses, leading them to (2) propose final solutions prematurely in conversation, (3) make incorrect assumptions about underspecified details, and (4) rely too heavily on previous (incorrect) answer attempts.

Our findings highlight a gap between how LLMs are used in practice and how the models are being evaluated. Ubiquitous performance degradation over multi-turn interactions is likely a reason for low uptake of AI systems (Southworth et al., 2023; Brauner et al., 2023; Horowitz et al., 2024), particularly with novice users who are less skilled at providing complete, detailed instructions from the onset of conversation (Zamfirescu-Pereira et al., 2023; Knoth et al., 2024). We provide actionable recommendations based on small-scale experiments and make a concrete call-to-action to LLM builders, urging them to prioritize multi-turn reliability in conjunction with aptitude.

2 Background and Related Work

Previous-generation language models (e.g., BART (Lewis et al., 2019), GPT-2 (Radford et al., 2019), or T5 (Raffel et al., 2020)) were designed for single-turn tasks, and so do the evaluation. Conversational agents were instead built as modular systems using language models as a component (Konrád et al., 2021), and were evaluated through human protocols (Deriu et al. 2021; Murakhovs' ka et al. 2023, *inter alia*).

While a series of work on multi-turn evaluation emerged since the rise of ChatGPT (Zheng et al., 2023b; Kwan et al., 2024), we argue that such works frame conversation as *episodic*: every turn is a self-contained subtask that can be graded in isolation. We show that such a design overestimates LLM capability. In fact, when the model must fuse dispersed clues across turns, performance drops sharply and consistently (Appendix G.3). Indeed, real users often issue **underspecified** instructions both in human-AI communication (Herlihy et al., 2024) and in natural human communication–named as "principle of least effort" (Zipf, 1949). We place underspecification at the center of our study.[1]

A second limitation of prior work on multi-turn episodic evaluation is the mismatch in task granularity: multi-turn subtasks differ from the single-turn counterpart, which prevents direct comparison. We run both settings on *a common set of tasks*, allowing a precise observation of performance degradation from single to multi-turn. Crucially, we focus on open-ended generation–the predominant real-world use case in code and natural-language tasks (Zheng et al., 2023a; Handa et al., 2025)–rather than short-form generation or classification.

[1]Appendix A reviews related work specifically focused on underspecified communication.

可能相互关联，但对话实际上可以分解为一系列可以独立评估的子任务。我们认为，片段式任务偏离了人类对话中普遍存在的现象：欠明确性（齐普夫，1949；Herlihy等人，2024）。

在这项工作中，我们通过创建一个用于多轮次欠明确对话的模拟环境——分片模拟——来填补这一空白，该环境利用了来自高质量单轮基准测试的现有指令。从高层次来看，我们提出的分片过程将现有的单轮指令转化为分片指令，这是一组更小的指令，它们共同传递与原始指令相同的信息。分片模拟随后确保对话的每一轮次最多揭示一个信息分片，从而强制指令在对话过程中逐步被揭示。

在我们进行实验的任务集上，我们观察到参与多轮次欠明确对话的模型平均性能为65%——相较于在对话开始时接收完整指令时90%的单轮性能，下降了25个百分点。值得注意的是，我们甚至在两轮对话中也观察到了这种性能下降，并且在我们测试的所有大语言模型中皆是如此，从小型开源权重模型（LLama3.1-8B-Instruct）到最先进技术模型（Gemini 2.5 Pro）。

此外，我们将性能下降分解为两个组成部分：(1) 能力损失，以及 (2) 不可靠性增加。我们发现，在单轮对话中，能力较高的大语言模型往往更可靠（例如，GPT-4.1）。另一方面，所有大语言模型在多轮对话场景中都表现出极高的不可靠性，无论其能力如何。我们将此称为对话迷失现象：当大语言模型在多轮对话中走错一步时，它们就会迷失方向且无法恢复。

我们研究了导致此效应的几种解释，并表明大语言模型倾向于：(1) 生成过于冗长的响应，导致它们 (2) 在对话中过早提出最终解决方案，(3) 对未充分指定的细节做出错误假设，以及 (4) 过度依赖先前（错误的）答案尝试。

我们的研究结果突显了大语言模型在实际使用方式与模型评估方式之间的差距。在多轮交互中普遍存在的性能下降，很可能是人工智能系统采用率低的原因之一（Southworth等人, 2023; Brauner等人, 2023; Horowitz 等人 , 2024 ），特别是对于新手用户而言，他们不太擅长从对话开始就提供完整、详细的指令（Zamfirescu-Pereira等人, 2023; Knoth等人, 2024）。我们基于小规模实验提供了可操作的建议，并向大语言模型构建者发出了具体的行动呼吁，敦促他们将多轮对话的可靠性与能力提升一同作为优先事项。

2 背景与相关工作

上一代语言模型（例如 BART（Lewis 等人，2019）、GPT-2（Radford 等人，2019）或 T5（Raffel 等人，2020））是为单轮任务设计的，其评估亦是如此。而对话智能体则是作为模块化系统构建的，将语言模型作为其中一个组件（Konrád 等人，2021），并通过人工协议进行评估（Deriu 等人，2021；Murakhovs' ka 等人，2023，以及其他）。

尽管自 ChatGPT 兴起以来，出现了一系列关于多轮评估的工作（Zheng 等人，2023b；Kwan 等人，2024），但我们认为这类工作将对话框定为片段式的：每一轮都是一个自包含的子任务，可以独立评分。我们表明，这样的设计高估了大语言模型的能力。事实上，当模型必须融合跨越多个轮次的分散线索时，其性能会急剧且持续地下降（附录G.3）。确实，真实用户无论是在人机沟通（**未充分指定的**指令（Herlihy 等人，2024）还是在自然的人类交流中——被称为"最小努力原则"（Zipf，1949）。我们将欠明确性置于我们研究的核心。[1]

先前关于多轮情景评估研究的第二个局限在于任务粒度的不匹配：多轮子任务与单轮对应任务不同，这阻碍了直接比较。我们在一组共同的任务上运行两种设置，从而能够精确观察从单轮到多轮的性能下降。关键的是，我们专注于开放式生成——这是代码和自然语言任务中主要的现实世界用例（Zheng等人，2023a；Handa等人，2025）——而非短格式生成或分类。

[1]附录 A 回顾了专门针对未充分指定的沟通的相关工作。

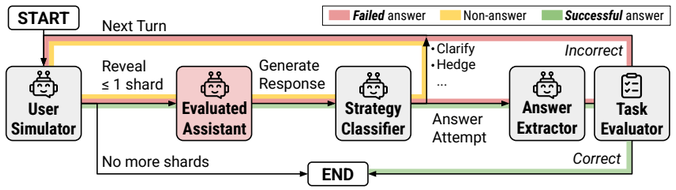


Figure 2: Sharded Conversation Simulation Diagram. The simulation subject is highlighted in red.

Scaling multi-turn experimentation requires simulating a user. Possible options range from templates (Choi et al., 2018; Reddy et al., 2019; Laban et al., 2023; Deng et al., 2024), LLM (Poelitz & McKenna, 2025; Li et al., 2024; Chang et al., 2025; Liang et al., 2024), fixed annotations (Finch et al., 2023; Chang et al., 2025), or real users (Ram et al., 2018; Laban et al., 2021; Chiang et al., 2024), each trading realism for cost and scalability. We adopt an LLM-based simulator that balances diversity and control, treating it strictly as a probe of *LLM behavior*, not user behavior (due to the approximation on the user end). In addition, our simulation is simplistic and idealized, and the degradations we report likely underestimate those in genuine human-AI conversations (Section 8).

3 Simulating Underspecified, Multi-Turn Conversation

We develop a simulation environment that repurposes existing tasks from single-turn benchmarks. First, we apply a *sharding process* to transform original fully-specified instructions into *sharded instructions*. Second, we implement a sharding simulation environment that carries out a multi-turn conversation based on a sharded instruction.

3.1 Sharding Process

To construct high-quality sharded instructions from the original fully-specified instructions, we define a set of properties that a sharded instruction must satisfy, such as information preservation, clear initial intent, and order insensitivity. We refer to Appendix B for a precise and mathematical definition of shards and their properties. Based on these properties, we develop a semi-automatic sharding process to scale the creation of sharded instructions, which involves (1) segmentation (2) rephrasing (3) verification (4) manual inspection.[2] At a high level, a sharded instruction is composed of *a set of shards*, each introducing a single element from the original instruction. Taken jointly, the set of shards reflects the same information provided in the fully-specified instruction, with the information explicitly divided across shards. During manual inspection, we reviewed every sharded instruction, merging, splitting, or reordering shards to enhance the naturalness of the sharded sample, and editing phrasing to ensure that each shard represents a natural unit of information a user might reasonably provide in a single turn and not an adversarial representation of the original instruction. This process ensures that the experiments we carried out used sharded instructions that adhered to the properties we defined. Example pairs of fully-specified and sharded instructions are shown in Figure 4.

3.2 Simulating Sharded Conversations

Figure 2 outlines our multi-turn simulator for sharded tasks. We implement a loop with three LLM-backed roles. The **assistant** is the model under test, the **user** owns the full sharded instruction and chooses which shard to disclose each turn, and the **system** tags and grades assistant replies.

At turn 1 the user reveals Shard 1, the assistant responds freely, and the system maps that reply to one of seven strategies—*clarification*, *refusal*, *hedging*, *interrogation*, *discussion*, *missing*, or *answer attempt*—following Herlihy et al. (2024). If a reply contains an answer attempt, we extract the answer span (e.g., a number or code block) to shield scoring from surrounding text, then pass it to a task-specific evaluator. Subsequent turns repeat: the user may expose one additional shard, the assistant replies, and any answer attempt is scored. A conversation's final score is the maximum over all per-turn scores: in a conversation with N shards, the assistant gets up to N answer attempts, and

[2]The sharding process is described in depth in Appendix C.

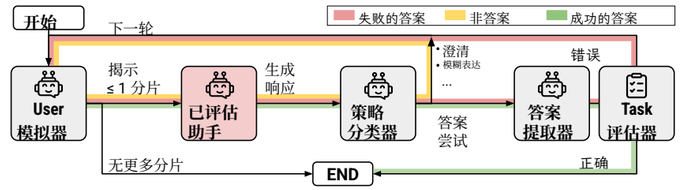


图2：分片对话模拟图。模拟主体以红色高亮显示。

扩展多轮实验需要模拟用户。可能的选项范围包括模板（Choi等人，2018；Reddy等人，2019；Laban等人，2023；Deng等人，2024）、大语言模型（Poelitz & McKenna，2025；Li等人，2024；Chang等人，2025；Liang等人，2024）、固定标注（Finch等人，2023；Chang等人，2025）或真实用户（Ram等人，2018；Laban等人，2021；Chiang等人，2024），每种方法都在真实性与成本及可扩展性之间进行权衡。我们采用一种基于大语言模型的模拟器，它在多样性和控制性之间取得平衡，并严格将其视为对大语言模型行为的探测，而非用户行为（由于在用户端的近似性）。此外，我们的模拟是简化且理想化的，我们所报告的性能下降很可能低估了真实人机对话中的下降程度（第8节）。

3 模拟未充分指定的多轮对话

我们开发了一个模拟环境，该环境重新利用了来自单轮基准测试的现有任务。首先，我们应用一个分片过程，将原始的完整指定指令转化为分片指令。其次，我们实现了一个分片模拟环境，该环境基于分片指令执行多轮对话。

3.1 分片过程

为了从原始的完整指定指令中构建高质量的分片指令，我们定义了一组分片指令必须满足的属性，例如信息保留、清晰的初始意图和顺序不敏感性。关于分片及其属性的精确定义和数学定义，请参阅附录B。基于这些属性，我们开发了一个半自动分片流程来规模化创建分片指令，该流程包括（1）分割（2）改写（3）验证（4）人工检查。[2] 概括来说，一个分片指令由一组分片组成，每个分片引入原始指令中的一个单一元素。这些分片共同反映了完整指定指令中提供的相同信息，信息被明确地划分到各个分片中。在人工检查过程中，我们审查了每一个分片指令，通过合并、拆分或重新排序分片来增强分片样本的自然度，并编辑措辞以确保每个分片代表用户在单轮对话中可能合理提供的一个自然信息单元，而不是原始指令的对抗性表示。这个过程确保了我们进行的实验所使用的分片指令符合我们定义的属性。完整指定指令与分片指令的示例如图4所示。

3.2 模拟SHARDED对话

图 2 概述了我们用于分片任务的多轮模拟器。我们实现了一个包含三个基于大语言模型角色的循环。被测试的模型是**助手**，拥有完整分片指令并选择每轮披露哪个分片的是**用户**，而**系统**则负责标记和评估助手回复。

在第1轮，用户揭示分片1，助手自由回应，系统将该回复映射到七种策略之一——澄清、拒绝、模糊表达、询问、讨论、缺失或答案尝试——遵循Herlihy等人（2024）的方法。如果回复包含答案尝试，我们会提取答案片段（例如，一个数字或代码块），以使评分免受周围文本的影响，然后将其传递给任务特定评估器。后续轮次重复进行：用户可能揭示一个额外的分片，助手回应，任何答案尝试都会被评分。一次对话的最终得分是所有每轮得分中的最高分：在一次包含 N 个分片的对话中，助手最多有 N 次答案尝试，并且

[2]分片过程的详细描述见附录C。

is credited for the best one. From that perspective, the assistant has some advantage in the sharded setting over single-turn simulations as they only permit at most one answer attempt. The conversation ends when an answer is deemed correct or no shards remain.

Choosing the next shard is non-trivial because assistants often ask shard-specific follow-ups. We therefore instantiate the user simulator with an LLM (GPT-4o-mini), giving it the entire instruction and conversation history so it can select and lightly rephrase the shard that best fits the exchange, for instance, responding to a clarification question with the relevant shard rather than revealing shards in a fixed or random order. Before the first turn, the assistant receives only minimal context (e.g., a tool list) as a system prompt; it is never told that the conversation will be underspecified or multi-turn, enabling us to measure default behavior.

The strategy classifier and answer extractor are also prompt-based GPT-4o-mini modules. The classifier's primary role in the simulation loop is to detect *answer attempt* turns, which trigger answer extraction and scoring; it does not directly influence the user simulator's shard selection. To ensure the validity of the framework, we manually reviewed several hundred conversations and found that errors in any component occurred in fewer than 5% of cases, and less than 2% of cases where the assistant was disfavored.[3] We thus regard the simulator as sufficiently accurate for our experiments.

Appendix K provides an example simulated sharded conversation.

3.3 Simulation Types

We simulate five types of conversation, each with a different pace of information:

Fully-specified (short-form: Full) Single-turn task: the original instruction appears in turn 1. This serves as the baseline.

Sharded Multi-turn task: the sharded instruction is revealed across turns as described above. Core setting for underspecification.

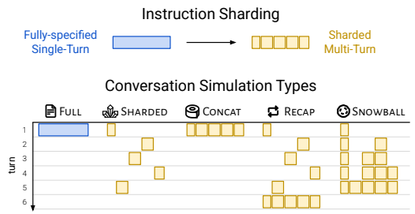


Figure 3: Conversation simulation types based on sharded instructions. Once a fully-specified instruction (blue block) is sharded (yellow blocks), the "shards" can be used to simulate single-turn (Full, Concat) or multi-turn (Sharded, Recap, Snowball) conversations, affecting the pace of information disclosure.

Concat All shards are concatenated into one bullet-point prompt in a single turn. This setting removes underspecification (like Full) while retaining sharding rephrasing; a control to ensure the performance drop in Sharded is due to multi-turn underspecification, not due to rephrasing.

Recap runs a Sharded conversation then adds one recap turn that restates every shard, giving the model a last, "fully-specified" chance. This setting tests whether a simple agent-style recap mitigates Sharded degradation.

Snowball At each turn, the user reveals the next shard and repeats all prior shards, "snowballing" the context. This setting evaluates whether continual reminders reduce the memory burden over long, multi-turn interactions.

4 Experiment

4.1 Task Selection

We construct sharded instructions for six tasks that we use in a large-scale simulation experiment. For each task, we select instructions from one or two high-quality single-turn benchmarks and apply a semi-automatic sharding process (outlined in Appendix C). For each task, we prepare 90-120 sharded instructions. We select popular and diverse generation tasks across programming and non-programming use cases. Figure 4 provides an example of an original and sharded instruction for each task, which we introduce below:

[3]See Appendix D for details on the annotation process.

最佳的一次尝试将获得最终得分。从这个角度看，助手在分片设置中相比单轮模拟具有一定优势，因为单轮模拟最多只允许一次答案尝试。当答案被判定为正确或没有剩余分片时，对话结束。

选择下一个分片并非易事，因为助手经常会提出针对特定分片的后续问题。因此，我们使用一个大语言模型（GPT-4o-mini）来实例化用户模拟器，为其提供完整的指令和对话历史，使其能够选择并略微改写最符合当前交流的分片。例如，它会用相关的分片来回应澄清问题，而不是以固定或随机的顺序揭示分片。在第一轮次之前，助手仅通过系统提示接收到最少的上下文（例如，一个工具列表）；它永远不会被告知对话将是未充分指定的或多轮的，这使我们能够测量其默认行为。

策略分类器和答案提取器同样是基于提示词的GPT-4o-mini模块。分类器在模拟循环中的主要作用是检测答案尝试轮次，这会触发答案提取和评分；它并不直接影响用户模拟器的分片选择。为确保框架的有效性，我们手动审查了数百个对话，发现任何组件出现错误的情况少于5%，而助手处于不利地位的情况则少于2%。[3] 因此，我们认为该模拟器对我们的实验而言足够准确。

附录K提供了一个模拟分片对话的示例。

3.3 模拟类型

我们模拟了五种类型的对话，每种对话的信息披露节奏各不相同：

完全明确-指定（简称：完整）单轮任务：原始指令出现在第1轮。这作为基线。

分片多轮任务：如上所述，分片指令在多轮对话中逐步揭示。这是欠明确性研究的核心设置。

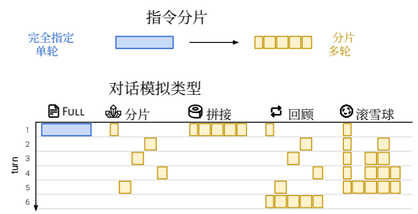


图3：基于分片指令的对话模拟类型。一旦一个完全指定的指令（蓝色块）被分片（黄色块），这些"分片"可用于模拟单轮（完整，拼接）或多轮（分片，重述，滚雪球）对话，从而影响信息披露的节奏。

拼接所有分片被拼接成一个要点提示词，置于单轮对话中。此设置消除了欠明确性（与完整设置类似），同时保留了分片改写；这是一个对照设置，用以确保分片设置中的性能下降是由于多轮欠明确性，而非改写本身。

Recap 运行一个分片对话，然后添加一个复述轮次，重述每个分片，为模型提供最后一次"完全指定"的机会。此设置测试简单的智能体式复述是否能缓解分片性能下降。

Snowball 在每一轮次，用户揭示下一个分片并重复所有先前的分片，使上下文"滚雪球"式增长。此设置评估持续的提醒是否能减轻长多轮交互中的记忆负担。

4 实验

4.1 任务选择

我们为大规模模拟实验中使用的六项任务构建了分片指令。针对每项任务，我们从一个或两个高质量单轮基准测试中选取指令，并应用半自动分片流程（概述见附录C）。针对每项任务，我们准备了90-120条分片指令。我们选取了编程和非编程用例中流行且多样化的生成任务。图4 提供了每项任务的原始指令和分片指令示例，我们将在下文介绍：

[3]详见附录D 以了解标注流程的详细信息。

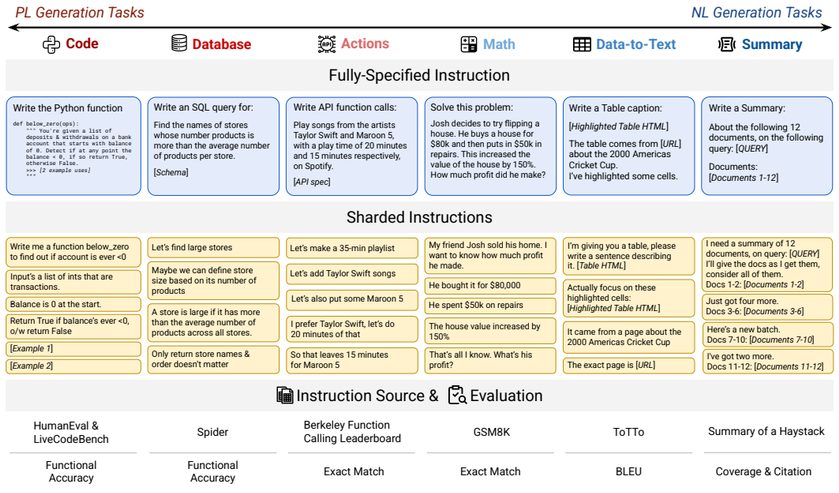


Figure 4: Six sharded tasks included in our experiments. We include tasks that involve generating programming and natural language. For each task, an illustrative fully-specified instruction and its sharded counterpart. We sharded 90 to 120 instructions based on existing datasets (Instruction Origin), and re-purposing evaluation.

Code The assistant writes a Python function that implements the instruction. The original instructions are sourced from HumanEval (Chen et al., 2021) and LiveCodeBench (Jain et al., 2024).

Database The assistant generates an SQL query given a database's schema and a user query in natural language (Text-to-SQL). The original instructions and databases are sourced from the popular Spider dataset (Yu et al., 2018).

Actions The assistant generates API calls that satisfy a user request given a set of API schemas. We source API schemas and user instructions from Berkeley Function Calling Leaderboard (BFCL) (Yan et al., 2024), a standard benchmark for measuring function calling capabilities.

Math The assistant numerically solves an elementary math word problem. Problems are sourced from the GSM8K dataset (Cobbe et al., 2021).

Data-to-text The assistant produces a caption describing a table and several elements of related metadata. The ToTTo dataset (Parikh et al., 2020) is used to prepare sharded instructions.

Summary The assistant generates a summary with citations given a user query and a set of (around twenty) documents. We re-purpose the instructions from Summary of a Haystack (Laban et al., 2024). The instruction in this task requires long-context understanding, a skill models are known to struggle with (Huang et al., 2023; Karpinska et al., 2024; Kim et al., 2024a).

For each task, we reuse the metrics used in the original benchmarks to assess instance-level correctness. We measure Code and Database with the functional accuracy, Actions and Math with semantic equivalence to the reference answer, all of which render a binary correctness. Data-to-Text and Summary are *refinement tasks*, which get scored on a range (0-100): BLEU (Papineni et al., 2002) for Data-to-Text and a custom LLM-as-a-judge metric ("Joint Score") for Summary. We map binary accuracy to the range of 0-100 (0 = failure, 100 = success) so that all tasks produce scores on a common scale, facilitating aggregation.[4]

[4]Appendix J lists task-specific sharding details

图4：我们实验中包含的六个分片任务。我们纳入了涉及生成编程和自然语言的任务。对于每个任务，展示了一个说明性的完全指定指令及其对应的分片版本。我们基于现有数据集（指令来源）对90到120条指令进行了分片，并重新利用了评估方法。

代码助手编写一个实现该指令的Python函数。原始指令来源于HumanEval(Chen et al., 2021)和LiveCodeBench (Jainet al., 2024)。

数据库助手根据数据库模式和自然语言用户查询生成SQL查询（文本转SQL）。原始指令和数据库来源于流行的Spider数据集（Yu等人, 2018）。

行动助手根据一组API模式生成满足用户请求的API调用。我们从伯克利函数调用排行榜（BFCL）（Yan等人, 2024）获取API模式和用户指令，这是一个衡量函数调用能力的标准基准。

数学助手以数值方式解决一个基础数学文字问题。问题来源于GSM8K数据集（Cobbe等人, 2021）。

数据到文本助手生成一个描述表格及其若干相关元数据元素的标题。使用ToTTo数据集（Parikh等人,2020）来准备分片指令。

摘要助手根据用户查询和一组（约二十个）文档生成带引用的摘要。我们重新利用了"大海捞针摘要"（Laban等人, 2024）中的指令。此任务中的指令需要长上下文理解能力，这是已知模型难以掌握的技能（Huang等人,2023; Karpinska等人, 2024; Kim等人, 2024a）。

对于每个任务，我们复用原始基准测试中使用的指标来评估实例级别的正确性。我们使用功能准确率衡量代码和数据库任务，使用与参考答案的语义等价性衡量行动和数学任务，所有这些都产生二元正确性判断。数据到文本和摘要是精炼任务，其分数在一个范围（0-100）内评定：数据到文本使用BLEU（Papineni等人, 2002），摘要则使用自定义的LLM作为评判者的指标（"联合得分"）。我们将二元准确率映射到0-100的范围（0 = 失败对应0， 100 = 成功对应100），以便所有任务都在一个统一的尺度上产生分数，便于聚合。[4]

[4]附录 J列出了任务特定的分片详细信息

4.2 SIMULATION METRICS

LLMs employ a stochastic process to generate text. When setting the generation parameters to their default (*e.g.*, temperature = 1.0), they generate many distinct responses for a fixed conversation state. We leverage this property to conduct repeated simulations for a given instruction and quantify the variations that occur. Suppose the i-th simulation yields a score $S_i \in [0, 100]$ from a task-specific evaluator. By running N simulations for an instruction and obtaining the set of scores $S = \{S_i\}_{i=1}^{N}$, we define three metrics: **averaged performance** $\overline{P}$, **aptitude** A^{90}, and **unreliability** U_{10}^{90}:

$$\overline{P} = \sum_{i=1}^{N} S_i/N, \quad A^{90} = \text{percentile}_{90}(S), \quad U_{10}^{90} = \text{percentile}_{90}(S) - \text{percentile}_{10}(S).$$

Averaged performance $\overline{P}$ is an unbiased estimate of a model's mean score on an instruction in a given simulation type. Aptitude A^{90} is an estimate of a model's 90th percentile score on a given instruction, *i.e.* a best-case metric that estimates scores obtained in the top 10% of simulations conducted. Unreliability U_{10}^{90} is an interpercentile range estimate measuring the gap between the 90th and 10th percentile estimates, giving a sense of *level of degradation* that occurs in response quality due to stochasticity in the LLM.[5]

Each metric is computed on a per-instruction basis and can be averaged across a corpus of instructions to obtain corpus-level metrics.

For the rest of the paper, we refer to reliability and unreliability interchangeably, with reliability defined as $R_{10}^{90} = 100 - U_{10}^{90}$. We also simplify the notations to A for aptitude and U for unreliability, though the metrics can be generalized to other percentile thresholds (*e.g.*, A^{80} or U_5^{95}). Figure 5a visually connects the aptitude and unreliability metrics to box-plot visualizations. The height of the upper whisker of the box plot represents aptitude (A), and the distance between the upper and lower whiskers represents Unreliability (U).

4.3 SIMULATION SCALE AND PARAMETERS

In the main simulation experiment, we simulate conversations across three types: FULL, CONCAT, and SHARDED on around 100 instances for each of the six tasks. We experiment with 15 LLMs, running $N = 10$ simulations for each pair of model and simulation type, totaling more than 200,000 simulated conversations. All simulations are conducted with a default temperature of $T = 1$, however, we performed a supplementary experiment (Section G.2) that explores the effect of temperature on aptitude and reliability. Although simulating ten conversations for each `(LLM, instance, simulation type)` increases experimental costs ten-fold, it allows us to not only measure averaged performance ($\overline{P}$) more accurately, but also study aptitude and reliability of LLM systems in depth in Section 5.2.

We select 15 LLMs from eight model families: OpenAI (GPT-4o-mini, GPT-4o (Hurst et al., 2024), o3 (OpenAI, 2025), and GPT-4.1), Anthropic (Claude 3 Haiku, Claude 3.7 Sonnet), Google's Gemini (2.5 Flash, 2.5 Pro; Team et al. 2023), Meta's Llama (Llama3.1-8B-Instruct, Llama3.3-70B-Instruct, Llama 4 Scout; Grattafiori et al. 2024), AI2 OLMo-2-13B (OLMo et al., 2024), Microsoft Phi-4 (Abdin et al., 2024), Deepseek-R1 (Guo et al., 2025), and Cohere Command-A (Cohere et al., 2025). The selection prioritizes the evaluation of state-of-the-art models: small (8B) to large (300B+), open- to closed-weights, and two reasoning models (o3, R1) to probe test-time compute in multi-turn conversations. Details on model versioning and access are listed in Appendix I. We estimate the total cost of conducting simulations to be around $5,000.

All tasks in our experiments require fewer than 20k tokens of total context, with the summarization task requiring the most and all others typically fitting within 8k tokens. Two models (Phi-4, OLMo-2-13B) could not accommodate the summarization task's context length and were excluded from it (indicated by dashes in Table 1). We emphasize that the intent of these experiments is to study multi-turn behavior at *regular* context lengths, not to stress-test long-context capabilities.

4.2 模拟指标

大语言模型采用随机过程来生成文本。当将生成参数设置为默认值时（例如，温度 = 1.0），它们会针对固定的对话状态生成许多不同的响应。我们利用这一特性，对给定的指令进行重复模拟，并量化出现的变异。假设第 i次模拟从任务特定评估器获得分数 $S_i \in [0, 100]$。通过对一条指令运行 N 次模拟并获得分数集合 $S = \{S_i\}_{i=1}^{N}$，我们定义三个指标：**平均性能** P、**能力** A^{90}和**不可靠性**U_{10}^{90}：

$$\overline{P} = \sum_{i=1}^{N} S_i/N, \quad A^{90} = \text{percentile}_{90}(S), \quad U_{10}^{90} = \text{percentile}_{90}(S) - \text{percentile}_{10}(S).$$

平均性能 P是模型在给定模拟类型下对一条指令的平均分数的无偏估计。能力 A^{90} 是模型在给定指令上的第90百分位数分数的估计，即一个最佳情况指标，用于估计在已进行的模拟中排名前10%所获得的分数。不可靠性 U_{10}^{90}是一个百分位间距估计，衡量第90百分位数与第10百分位数估计值之间的差距，反映了由于大语言模型的随机性导致响应质量发生的退化程度。[5]

每个指标均基于单条指令计算，并可在一个指令语料库上进行平均，以获得语料库级别的指标。

在本文的其余部分，我们将可靠性和不可靠性互换使用，其中可靠性定义为 $R_{10}^{90} = 100 - U_{10}^{90}$。我们也将符号简化为 A 表示能力，U表示不可靠性，尽管这些指标可以推广到其他百分位阈值（例如，A^{80} 或 U_5^{95}）。图5a直观地将能力和不可靠性指标与箱线图可视化联系起来。箱线图上须线的高度代表能力（A），上下须线之间的距离代表不可靠性（U）。

4.3 模拟规模与参数

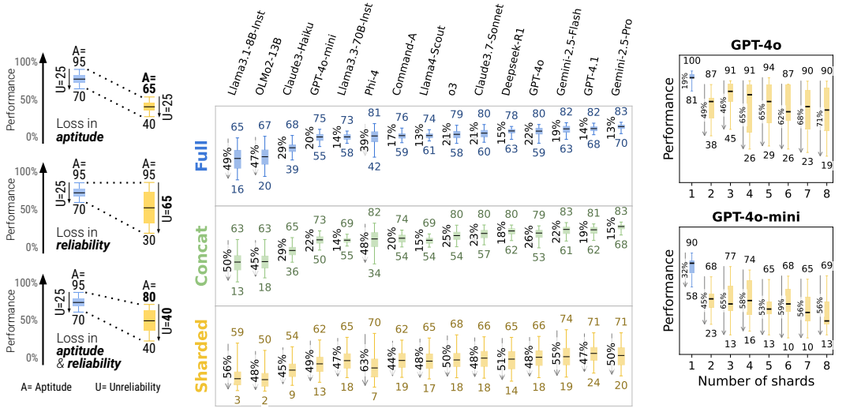
在主要的模拟实验中，我们模拟了三种类型的对话：FULL、CONCAT和SHARDED针对六项任务中的每项，各使用约100个实例。我们对15个大语言模型进行了实验，为每个模型与模拟类型的组合运行 $N = 10$ 次模拟，总计超过200,000次模拟对话。所有模拟均采用默认温度 $T = 1$进行，不过，我们进行了一项补充实验（章节G.2），探讨了温度对能力和可靠性的影响。尽管为每个`(LLM, instance, simulation type)`模拟十次对话会使实验成本增加十倍，但这不仅能让我们更准确地测量平均性能（P），还能在章节5.2中深入研究生大语言模型系统的能力和可靠性。

我们从八个模型系列中选择了15个大语言模型：OpenAI（GPT-4o-mini、GPT-4o（Hurst等人，2024）、o3（OpenAI，2025）和GPT-4.1）、Anthropic（Claude 3 Haiku、Claude 3.7 Sonnet）、谷歌的Gemini（2.5 Flash、2.5 Pro；Team等人 2023）、Meta的Llama（Llama3.1-8B-Instruct、Llama3.3-70B-Instruct、Llama4 Scout；Grattafiori等人 2024）、AI2 OLMo-2-13B（OLMo等人，2024）、微软Phi-4（Abdin等人，2024）、Deepseek-R1（Guo等人，2025）以及Cohere Command-A（Cohere等人，2025）。选择优先考虑评估最先进模型：从小型（8B）到大型（300B+），从开源权重到闭源权重，以及两个推理模型（o3、R1），以探究多轮对话中的测试时计算。关于模型版本和访问的详细信息列于附录I。我们估计进行模拟的总成本约为5,000美元。

我们实验中的所有任务所需的总上下文均少于2万个词元，其中摘要任务需求最多，其他任务通常可容纳在8千词元内。有两个模型（Phi-4、OLMo-2-13B）无法满足摘要任务的上下文长度要求，因此被排除在该任务之外（在表1中用短横线标示）。我们强调，这些实验的目的是研究常规上下文长度下的多轮对话行为，而非对长上下文能力进行压力测试。

Model	FULL						CONCAT						SHARDED						Overall	
3.1-8B	27.4	64.1	82.9	13.7	63.9	7.6	21.2	47.7	83.0	15.7	62.6	6.5	21.7	25.9	45.5	13.3	37.4	3.4	91.6	62.5
OLMo2	18.8	54.8	56.1	17.2	80.0	-	16.3	40.5	49.8	14.3	80.1	-	14.4	22.4	13.8	9.0	46.3	-	86.5	50.5
3-Haiku	44.8	85.0	83.5	29.8	73.9	11.6	36.3	76.5	80.2	30.1	76.1	9.2	31.5	31.8	55.9	18.6	47.1	1.6	91.6	52.4
4o-mini	75.9	89.3	94.1	35.9	88.1	14.9	66.7	90.7	92.2	31.2	88.0	12.5	50.3	40.2	52.4	19.8	58.7	7.2	93.0	56.2
3.3-70B	72.0	91.1	95.0	34.1	91.7	15.8	52.7	87.9	97.0	32.0	91.8	14.7	51.6	35.4	71.0	22.4	61.5	10.5	93.2	64.2
Phi-4	53.2	87.6	82.7	23.9	89.2	-	48.4	79.6	76.0	28.6	90.4	-	39.1	33.1	34.1	23.2	52.5	-	99.0	61.7
CMD-A	72.0	91.9	98.5	27.7	94.5	24.3	61.6	86.1	98.4	33.2	91.9	21.3	44.9	33.6	72.0	27.9	66.0	4.9	97.3	60.4
4-Scout	73.9	92.7	98.0	35.2	96.3	13.7	60.3	81.5	98.3	28.2	92.9	13.7	46.4	27.1	69.9	26.1	67.0	12.3	91.0	66.1
o3	86.4	92.0	89.8	40.2	81.6	30.7	87.2	83.3	91.5	39.4	80.0	30.4	53.0	35.4	60.2	21.7	63.1	26.5	98.1	64.1
3.7-Sonnet	78.0	93.9	95.4	45.6	85.4	29.3	76.2	81.5	96.0	53.3	87.2	28.9	65.6	34.9	33.3	35.1	70.0	23.6	100.4	65.9
R1	99.4	92.1	97.0	27.0	95.5	26.1	97.1	89.9	97.0	36.7	92.9	24.4	70.9	31.5	47.5	20.0	67.3	17.2	103.6	60.8
4o	88.4	93.6	96.1	42.1	93.8	23.9	82.9	91.7	97.1	32.2	91.9	23.9	61.3	42.3	65.0	20.5	67.9	10.6	94.5	57.9
2.5-Flash	97.0	96.3	88.4	51.2	90.6	29.1	92.5	95.5	89.2	51.9	88.4	29.4	68.3	51.3	42.6	31.0	66.1	26.1	99.3	65.8
4.1	96.6	93.0	94.7	54.6	91.7	26.5	88.7	86.5	98.5	54.4	89.7	26.8	72.6	46.0	62.9	28.6	70.7	13.3	97.9	61.8
2.5-Pro	97.4	97.3	97.8	54.8	90.2	31.2	95.7	94.9	98.1	56.9	89.3	31.8	68.1	43.8	36.3	46.2	64.3	24.9	100.1	64.5

Table 1: Averaged Performance ($\overline{P}$) of LLMs on six tasks (Code, Database, Actions, Data-to-text, Math, and Summary). For each task, conversations are simulated in three settings: FULL, CONCAT, and SHARDED. Models are sorted in ascending order of average FULL scores across tasks. Background color indicates the level of degradation from the FULL setting. The last two columns average the performance drops from the CONCAT and SHARDED compared to the FULL in percentages across the six tasks.



(a) Visualizing Aptitude and Unreliability.

(b) Observed Model Degradations

(c) Gradual Sharding Results

Figure 5: (a) Visual introduction to the concepts of Aptitude and Unreliability when overlaid on a box-plot visualization, (b) reliability results based on experimental simulations with 15 LLMs, (c) summary of results from gradual sharding experiment, with instructions sharded in gradually larger shard sets (from 1 to 8 shards).

5 RESULTS

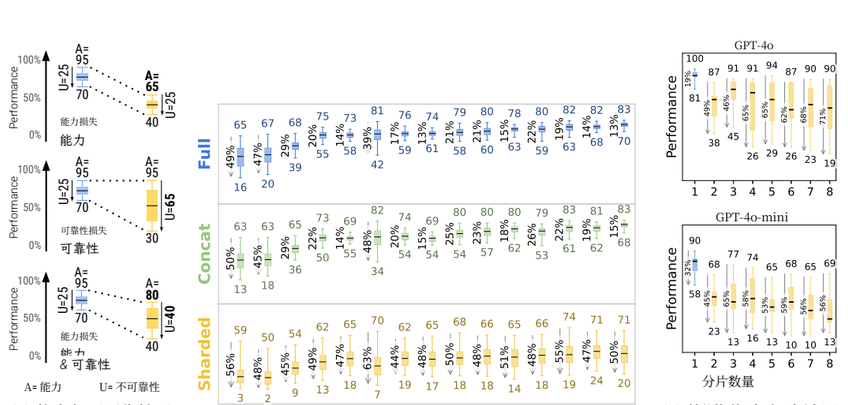
5.1 AVERAGE PERFORMANCE FINDINGS

Table 1 summarizes results from the simulation. At a high level, **every model sees its performance degrade on every task when comparing FULL and SHARDED performance, with an average degradation of -39%**. We name this phenomenon `Lost in Conversation`: models that achieve stellar (90%+) performance in the lab-like setting of fully-specified, single-turn conversation struggle on *the exact same tasks* in more realistic, underspecified, and multi-turn conversations.

[5]Appendix E shows how a drop in performance from 90% to 60% can stem from aptitude loss, reliability issues, or both, depending on the error breakdown.

模型	FULL						拼接						分片						总体	
3.1-8B	27.4	64.1	82.9	13.7	63.9	7.6	21.2	47.7	83.0	15.7	62.6	6.5	21.7	25.9	45.5	13.3	37.4	3.4	91.6	62.5
OLMo2	18.8	54.8	56.1	17.2	80.0	-	16.3	40.5	49.8	14.3	80.1	-	14.4	22.4	13.8	9.0	46.3	-	86.5	50.5
3-Haiku	44.8	85.0	83.5	29.8	73.9	11.6	36.3	76.5	80.2	30.1	76.1	9.2	31.5	31.8	55.9	18.6	47.1	1.6	91.6	52.4
4o-mini	75.9	89.3	94.1	35.9	88.1	14.9	66.7	90.7	92.2	31.2	88.0	12.5	50.3	40.2	52.4	19.8	58.7	7.2	93.0	56.2
3.3-70B	72.0	91.1	95.0	34.1	91.7	15.8	52.7	87.9	97.0	32.0	91.8	14.7	51.6	35.4	71.0	22.4	61.5	10.5	93.2	64.2
Phi-4	53.2	87.6	82.7	23.9	89.2	-	48.4	79.6	76.0	28.6	90.4	-	39.1	33.1	34.1	23.2	52.5	-	99.0	61.7
CMD-A	72.0	91.9	98.5	27.7	94.5	24.3	61.6	86.1	98.4	33.2	91.9	21.3	44.9	33.6	72.0	27.9	66.0	4.9	97.3	60.4
4-Scout	73.9	92.7	98.0	35.2	96.3	13.7	60.3	81.5	98.3	28.2	92.9	13.7	46.4	27.1	69.9	26.1	67.0	12.3	91.0	66.1
o3	86.4	92.0	89.8	40.2	81.6	30.7	87.2	83.3	91.5	39.4	80.0	30.4	53.0	35.4	60.2	21.7	63.1	26.5	98.1	64.1
3.7-Sonnet	78.0	93.9	95.4	45.6	85.4	29.3	76.2	81.5	96.0	53.3	87.2	28.9	65.6	34.9	33.3	35.1	70.0	23.6	100.4	65.9
R1	99.4	92.1	97.0	27.0	95.5	26.1	97.1	89.9	97.0	36.7	92.9	24.4	70.9	31.5	47.5	20.0	67.3	17.2	103.6	60.8
4o	88.4	93.6	96.1	42.1	93.8	23.9	82.9	91.7	97.1	32.2	91.9	23.9	61.3	42.3	65.0	20.5	67.9	10.6	94.5	57.9
2.5-Flash	97.0	96.3	88.4	51.2	90.6	29.1	92.5	95.5	89.2	51.9	88.4	29.4	68.3	51.3	42.6	31.0	66.1	26.1	99.3	65.8
4.1	96.6	93.0	94.7	54.6	91.7	26.5	88.7	86.5	98.5	54.4	89.7	26.8	72.6	46.0	62.9	28.6	70.7	13.3	97.9	61.8
2.5-Pro	97.4	97.3	97.8	54.8	90.2	31.2	95.7	94.9	98.1	56.9	89.3	31.8	68.1	43.8	36.3	46.2	64.3	24.9	100.1	64.5

表1：大语言模型在六项任务（代码、数据库、行动、数据到文本、数学和摘要）上的平均表现（P）。对于每项任务，对话在三种设置下进行模拟：完整设置、拼接和分片。模型按任务间平均完整设置分数升序排列。背景颜色表示相对于完整设置的性能下降程度。最后两列计算了拼接和分片设置相比完整设置，在六项任务上平均性能下降的百分比。



(a) 能力与不可靠性可视化。

(b) 观察到的模型性能退化

(c) 渐进分片实验结果

图 5: (a) 将能力和不可靠性概念叠加在箱线图可视化上的直观介绍，(b) 基于对 15 个大语言模型的实验模拟得出的可靠性结果，(c) 渐进分片实验的结果摘要，指令被逐步分入更大的分片集合（从 1 到 8 个分片）。

5 结果

5.1 平均性能发现

表1 总结了模拟结果。总体而言，**在比较完整和分片性能时，每个模型在每项任务上的性能均出现下降，平均降幅为-39%**。我们将此现象命名为 `Lost in Conversation`：那些在实验室般的、完全指定的单轮对话设置中取得卓越（90%+）性能的模型，在更为现实、未充分指定的多轮对话中处理完全相同的任务时却表现挣扎。

[5]附录 E 展示了性能从 90% 下降到 60% 可能源于能力损失、可靠性问题或两者兼有，具体取决于错误细分情况。

In comparison, models perform roughly equivalently in the CONCAT setting, with CONCAT performance averaging 95.1% of the FULL counterpart. This implies that the loss in performance for SHARDED is not explained by potential loss of information in sharded instructions, as such a loss would be reflected in lower CONCAT performance. We observe that smaller models (Llama3.1-8B-Instruct, OLMo-2-13B, Claude 3 Haiku) have more pronounced CONCAT degradations (86-92), and interpret this as indicating that smaller models struggle to generalize as well as larger models: benign rephrasing affects performance more than for larger, more robust models. This lack of robustness to paraphrasing can be observed visually in Table 1: CONCAT degradation (red background) is more pronounced in the top rows (weaker models) than the bottom rows (stronger models).

The last column of the Table (/) aggregates performance degradation across the six tasks, summarizing the magnitude of the Lost in Conversation effect for each model. Surprisingly, **more performant models (Claude 3.7 Sonnet, Gemini 2.5, GPT-4.1) get equally lost in conversation compared to smaller models (Llama3.1-8B-Instruct, Phi-4)**, with average degradations of 30-40%. This is in part due to metric definitions. Since smaller models achieve lower absolute scores in FULL, they have less scope for degradation than the better models. In short, no matter how strong an LLM's single-turn performance is, we observe large performance degradations in the multi-turn setting.

When looking at the task-specific breakdown, some models see more muted degradations in certain tasks. For instance, Command-A sees the least degradation on Actions, while Claude 3.7 Sonnet and GPT-4.1 conserve performance well on Code, and Gemini 2.5 Pro in Data-to-Text. This finding indicates that the multi-turn capabilities of models are not uniform across domains and validates the importance of benchmarking models across a wide variety of tasks to investigate model capabilities.

Additional test-time compute (reasoning tokens) does not help models navigate multi-turn underspecification, as the two reasoning models (o3, Deepseek-R1) deteriorate in similar ways to non-reasoning models. This result confirms that **additional test-time compute does not, on its own, allow models to strategize over multi-turn conversation**. The analysis we conduct identifies a potential root cause: reasoning models tend to generate lengthier responses (on avg. 33% longer than non-reasoning LLMs). As we find in Appendix F, longer assistant responses tend to contain more assumptions, which can confuse the model about user requirements vs. its own prior responses.

5.2 APTITUDE VS. RELIABILITY ANALYSIS

Results presented in Table 1 present averaged performance degradation ($\overline{P}$). We now report on the aptitude and reliability analysis based on metrics A and U. Figure 5b visually summarizes the results of the reliability analysis we conducted on the 15 LLMs included in our simulation experiment. First, looking at the two single-turn settings, we see that models that are more able (higher A) tend to be more reliable (lower U). For instance, the two most able models (GPT-4.1 and Gemini 2.5 Pro) achieve the lowest unreliability. At the lower end, the two models with the lowest aptitude (Llama3.1-8B-Instruct and OLMo-2-13B) are also the most unreliable. In summary, **in single-turn settings, models with higher aptitude tend to be more reliable.** This fact is known in the community, with arguments made that better models require less prompt engineering, as they are more robust to variations in inputs and outputs (Li et al., 2023).

The sharded setting paints a different picture. Aptitude degrades in a non-significant way between the full and sharded settings, with an average drop of 16%. On the other hand, unreliability skyrockets with an average increase of 112% (more than doubling). More interestingly, though better models tend to have slightly higher multi-turn aptitude, all models tend to have similar levels of unreliability. In other words, **in multi-turn, underspecified settings, all models we test exhibit very high unreliability, with performance degrading 50 percent points on average between the best and worst simulated run for a fixed instruction.** This refines our definition of the *lost in conversation* phenomenon: when comparing single- and multi-turn settings, we find that large performance degradations ($\overline{P}$) are due in large part to increased model unreliability (U), rather than a loss in aptitude (A).

Appendix F explores potential root causes for models getting lost in conversations. We identify four specific causes: (1) LLMs prematurely propose full answer attempts, making assumptions about problem specifications that lead to confusion (§F.1), (2) they overly rely on previous (incorrect) answer attempts leading to lengthier "bloated" answers (§F.2), (3) LLMs overly adjust their answers based on the first and last turn of conversation, evidenced by a loss-of-middle-turns phenomenon

相比之下，模型在拼接设置中的表现大致相当，其拼接性能平均为完整对应版本的95.1%。这意味着分片指令的性能损失不能归因于分片指令中潜在的信息丢失，因为此类损失会反映在更低的拼接性能上。我们观察到，较小的模型（Llama3.1-8B-Instruct、OLMo-2-13B、Claude 3 Haiku）表现出更明显的拼接性能下降（86-92），并将其解释为表明较小模型在泛化能力上不如较大模型：良性的改写对性能的影响比对更大、更鲁棒的模型更为显著。这种对改述缺乏鲁棒性的现象可以在表1中直观地看到：拼接性能下降（红色背景）在顶部行（较弱模型）比底部行（较强模型）更为明显。

表格的最后一列（ / ）汇总了六项任务中的性能下降情况，概括了每个模型所经历的对话中迷失效应的严重程度。令人惊讶的是，**性能更强的模型（Claude 3.7 Sonnet、Gemini 2.5、GPT-4.1）在对话中迷失的程度与较小模型（Llama3.1-8B-Instruct、Phi-4）相当**，平均性能下降幅度达30-40%。这部分是由于指标定义所致。由于较小模型在完整任务中取得的绝对分数较低，其性能下降的空间自然优于更好的模型。简而言之，无论一个大语言模型的单轮对话性能有多强，我们在多轮对话设置中都观察到了显著的性能下降。

当查看具体任务细分时，某些模型在特定任务中的性能下降较为缓和。例如，Command-A在行动任务上性能下降最少，而Claude 3.7 Sonnet和GPT-4.1在代码任务上性能保持良好，Gemini 2.5 Pro则在数据到文本任务上表现稳定。这一发现表明，模型的多轮对话能力在不同领域并不均衡，并验证了在广泛多样的任务上对模型进行基准测试以探究其能力的重要性。

额外的测试时计算（推理令牌）并不能帮助大语言模型应对多轮对话中的未明确指定问题，因为两种推理模型（o3、Deepseek-R1）的性能下降方式与非推理模型相似。这一结果证实了**额外的测试时计算本身并不能让模型在多轮对话中进行策略规划**。我们进行的分析确定了一个潜在的根本原因：推理模型倾向于生成更长的响应（平均比非推理大语言模型长33%）。正如我们在附录F中所发现的，更长的助手响应往往包含更多假设，这可能会让模型混淆用户要求与其自身先前的响应。

5.2 能力与可靠性分析

表1 中呈现的结果展示了平均性能下降（P）。我们现在基于指标 A 和 $\overline{U}$报告能力与可靠性分析。图5b 直观地总结了我们对我们模拟实验中包含的15个大语言模型进行的可靠性分析结果。首先，观察两个单轮设置，我们发现能力更强的模型（A值更高）往往更可靠（U值更低）。例如，两个能力最强的模型（GPT-4.1和Gemini 2.5 Pro）实现了最低的不可靠性。在能力较低的一端，两个能力最低的模型（Llama3.1-8B-Instruct和OLMo-2-13B）也是最不可靠的。总之，**在单轮设置中，能力更高的模型往往更可靠**。这一事实在社区中是已知的，有观点认为更好的模型需要更少的提示工程，因为它们对输入和输出的变化更具鲁棒性（Li等人，2023）。

分片设置则描绘了一幅不同的图景。能力在完整设置与分片设置之间的下降并不显著，平均降幅为16%。另一方面，不可靠性则急剧飙升，平均增幅达112%（超过一倍）。更有趣的是，虽然更好的模型往往具有稍高的多轮对话能力，但所有模型都倾向于表现出相似水平的不可靠性。换句话说，**在多轮、欠明确设定中，我们测试的所有模型都表现出极高的不可靠性，对于一条固定的指令，其最佳与最差模拟运行之间的性能平均下降50个百分点**。这完善了我们对对话迷失现象的定义：在比较单轮与多轮设置时，我们发现巨大的性能下降（P）在很大程度上是由于模型不可靠性（U）的增加，而非能力（A）的损失。

附录 F 探讨了模型在对话中迷失的潜在根本原因。我们识别出四个具体原因：(1) 大语言模型过早提出完整答案尝试，对问题规范做出导致混淆的假设（§F.1），(2) 它们过度依赖先前（错误）的答案尝试，导致产生更冗长的“臃肿”答案（§F.2），(3) 大语言模型过度根据对话的首轮和末轮调整其答案，这由中间轮次缺失现象所证明

(§F.3), and (4) they produce overly verbose answers, which likely introduces assumptions that detract attention from user utterances (§F.4).

5.3 Gradual Sharding Experiment

Revealing minimal amounts of information in each turn (Section 3.2) can seem unrealistic and adversarial. To explore the relationship between the granularity of sharding and the severity of the effect, we propose the gradual sharding experiment.

In the gradual sharding experiment, we selected 31 instructions from our original experiment across multiple tasks and expanded each sharded instruction into seven variants, with the shard-set size growing from 2 to 8 shards. The instruction selection and sharding process are detailed in Appendix L. The process ensured that at each shard set size (from 1 to 8), task complexity is fixed, and the only modified factor is sharding granularity.

We ran simulations for the gradual sharding experiments with two models (GPT-4o and GPT-4o-mini), with results summarized in Figure 5c. We find that both models get lost in conversation (a minor degradation in aptitude and a large increase in unreliability) with two-shard instructions and beyond. In other words, the gradual sharding experiment indicates that **any conversation that involves underspecification and occurs in two or more turns leads to models getting lost in conversation**. For users, the granularity at which information is specified does not majorly impact reliability: providing all the information at once (1-shard) is the only effective method to improve reliability.

We note a design decision that affects interpretation: because we selected only instructions with exactly 8 shards, the resulting instructions are inherently more complex than the corpus average (3 to 5 shards). Consequently, even the 2-shard variants in the gradual sharding experiment represent more complex instructions, which may explain why the large drop at $N{=}2$ is not followed by a proportionally steeper decline as N grows to 8. A complementary experiment using instructions with varying instruction complexity counts would likely yield a more gradual degradation curve, but would confound granularity with task complexity.

6 Implications Summary

System and Agent Builders (Appendix G.1) Modern LLM applications often rely on agent frameworks like LangChain and Autogen to orchestrate problem decomposition, retrieval, and tool use, raising the question of whether LLMs need native multi-turn capabilities at all. We simulate two agent-style interactions (Recap and Snowball) that repeatedly inject past user instructions to mitigate the performance drop seen in underspecified multi-turn scenarios. In Recap, once all shards have been revealed, a final corrective turn informs the assistant that its prior solutions were incorrect, restates all shards as a consolidated bullet list, and asks it to try once more. Both strategies improve over Sharded but fall short of Full or Concat performance. Snowball offers the more achievable improvement in practice (15–20%) through cumulative repetition. **The findings show that offloading memory to agent frameworks is not sufficient; LLMs should natively support multi-turn interaction.** We also investigate the effect of altering the system prompt, providing an explicit hint to the assistant that the conversation is likely to be multi-turn and underspecified, and found that such a system prompt hint leads to modest gains in performance (+1% across tasks), but does not effectively help the model avoid getting lost in conversation.

LLM Builders (Appendix G.2). While the community has focused on improving LLM aptitude, our findings emphasize the importance of model reliability, especially in multi-turn settings. We conducted a temperature ablation experiment (setting $T = 1.0, 0.5, 0.0$) and found that reliability does improve at lower temperatures in single-turn conversations, but not in Sharded multi-turn ones. Even at $T = 0.0$, multi-turn unreliability remains high (30%) due to cascading effects of early stochastic variation. **In short, lowering temperature does not mitigate unreliability in multi-turn contexts**. We urge LLM builders to jointly optimize for aptitude and reliability, developing models that: (1) maintain similar aptitude in single- and multi-turn settings, (2) demonstrate low unreliability ($U_{10}^{90} < 15$) in multi-turn, and (3) achieve this performance at standard temperature ($T = 1$).

（§F.3），以及 (4) 它们产生过于冗长的回答，这可能引入分散对用户话语注意力的假设（§F.4）。

5.3 渐进分片实验

在每一轮次中揭示最少量的信息（第3.2节）可能显得不切实际且具有对抗性。为了探索分片粒度与效应严重程度之间的关系，我们提出了渐进分片实验。

在渐进分片实验中，我们从原始实验中选取了31条跨越多重任务的指令，并将每条分片指令扩展为七个变体，分片集大小从2个分片增长到8个分片。指令选择与分片过程的细节详见附录L。该过程确保了在每个分片集大小（从1到8）下，任务复杂度是固定的，唯一被修改的因素是分片粒度。

我们使用两个模型（GPT-4o和GPT-4o-mini）运行了渐进分片实验的模拟，结果总结于图5c中。我们发现，两个模型在处理两分片及以上的指令时都会出现对话迷失（能力轻微下降，不可靠性大幅增加）。换言之，渐进分片实验表明，**任何涉及欠明确性且发生在两轮或更多轮次中的对话都会导致模型陷入对话迷失**。对于用户而言，信息被指定的粒度并不会对可靠性产生重大影响：一次性提供所有信息（1分片）是提高可靠性的唯一有效方法。

我们注意到一个影响解读的设计决策：由于我们只选择了恰好有8个分片的指令，因此生成的指令本质上比语料库的平均水平（3到5个分片）更复杂。因此，即使在渐进分片实验中，2分片的变体也代表了更复杂的指令，这或许可以解释为何在 $N{=}2$ 处的大幅下降之后，随着 N 增长到8，性能并未出现成比例的更陡峭下滑。一项使用不同指令复杂度计数的补充实验可能会产生更平缓的性能退化曲线，但这会将粒度与任务复杂性混为一谈。

6 影响摘要

系统与智能体构建者（附录G.1）现代LLM应用通常依赖LangChain和Autogen等智能体框架来编排问题分解、检索和工具使用，这引发了一个问题：大语言模型是否真的需要原生的多轮对话能力。我们模拟了两种智能体式交互（回顾和滚雪球），它们通过反复注入过去的用户指令来缓解在欠规范的多轮场景中观察到的性能下降。在回顾策略中，一旦所有分片都已揭示，最后一个纠正轮次会告知助手其先前的解决方案是错误的，将所有分片重新陈述为一个整合的要点列表，并要求它再试一次。这两种策略都比分片策略有所改进，但未能达到完整或拼接策略的性能水平。滚雪球策略通过累积重复，在实践中提供了更易实现的改进（15–20%）。**研究结果表明，将记忆卸载给智能体框架是不够的；大语言模型应原生支持多轮交互**。我们还研究了改变系统提示的效果，即向助手提供一个明确的提示，表明对话很可能是多轮且欠规范的，结果发现这种系统提示提示能带来适度的性能提升（跨任务平均+1%），但并不能有效帮助模型避免在对话中迷失。

大语言模型构建者（附录G.2）。尽管社区一直致力于提升大语言模型能力，但我们的研究结果强调了模型可靠性的重要性，尤其是在多轮对话场景中。我们进行了一项温度消融实验（设置 $T = 1.0$, 0.5, 0.0），发现在单轮对话中，较低温度下可靠性确实有所提升，但在分片的多轮对话中则不然。即使在 $T = 0.0$，由于早期随机性变化的级联效应，多轮对话的不可靠性仍然很高（30%）。**简而言之，降低温度并不能缓解多轮次上下文中的不可靠性**。我们敦促大语言模型构建者同时优化能力和可靠性，开发出能够满足以下条件的模型：(1) 在单轮与多轮设置中保持相近的能力，(2) 在多轮对话中表现出较低的不可靠性（$U_{10}^{90} < 15$），以及 (3) 在标准温度（$T = 1$）下实现此性能。

NLP Practitioners (Appendix G.3). Our proposed sharding procedure is semi-automated but still requires manual effort (3 hours per 100 samples) to ensure quality. We hypothesize that tasks most vulnerable to multi-turn degradation share three properties: (1) they are generative (not extractive), (2) sufficiently complex such that sharding would yield 3+ shards per instruction, and (3) are non-episodic tasks, with each new shard requiring the modification of the entire solution. For applicable tasks, we encourage researchers to release sharded variants alongside fully specified datasets.

Conversational System Users (Appendix G.4). Users should be aware of LLMs' reliability limitations, particularly in multi-turn settings. We offer two practical recommendations. First, "if time allows, try again"—starting a new conversation with the same information often yields better outcomes than persisting with a model that has become lost in conversation. Second, "consolidate before retrying"—since LLMs struggle with information dispersed across multiple turns, consolidating requirements into a single instruction improves both aptitude and reliability. Users can achieve this by asking the LLM to consolidate all user turns into an instruction used in a new conversation. These strategies remain cumbersome workarounds rather than principled solutions, highlighting the need for LLMs that can reliably handle multi-turn conversations without requiring such interventions.

Appendix G shares additional perspective for each of the audiences listed above.

7 Conclusion

In this work, we conduct a large-scale simulation of single- and multi-turn conversations with LLMs, and find that on a fixed set of tasks, LLM performance degrades significantly in multi-turn, underspecified settings. LLMs get lost in conversation, which materializes as a significant decrease in reliability as models struggle to maintain context across turns, make premature assumptions, and over-rely on their previous responses. Additional experiments reveal that known remediations that work for simpler settings (agent-like concatenation or decreasing temperature) are ineffective in multi-turn settings, and we call on LLM builders to prioritize the reliability of models in multi-turn settings.

8 Ethics Statement

The following section reflects on the limitations of the work we presented. We emphasize that the findings in this paper should be interpreted as the outcome of a controlled underspecification stress test, rather than a direct measurement of LLM behavior in natural dialogue with real users.

A first limitation of our work is the reliance on fully automated simulation. By relying on an LLM to simulate user utterances, we can scale our experiments, including running the same simulation multiple times, which would be cost-prohibitive with real users. However, the simulations we obtain are not representative of natural human-AI conversation. The properties of the sharding process (defined in Appendix C) and of the simulation environment (see Section 3.2) ensure that the simulated conversations follow a rather narrow structure, likely not modeling the full range of conversation dynamics that occur with a large, diverse user population. For example, the simulation process ensures a new shard of information is revealed at each turn, and that the last turn of the conversation has specified all the information needed to complete the task which might not happen with real users. Properties P1, P2, and P5 of the sharding process also restrict the scope of the conversation, as sharded instructions closely match an existing fully-specified instruction, with the high-level intent always identified in the conversation's first turn. The minimal nature of shards is also unrealistic and potentially adversarial, though the gradual sharding experiment finds that different levels of shard granularity lead to similar performance degradations, as soon as conversations occur over two turns or more. Apart from sharding granularity, automatic simulation also lacks the nuance that can occur when a human is involved in conversation, from misunderstandings over terminology, giving up due to frustration with system failures (Wester et al., 2024), or the lack of a feasible end goal for certain conversations (e.g., the user wanting a solution to an unsolved problem). Because of these factors, we believe conducted simulations represent a benign testing ground for LLM multi-turn capabilities. To mitigate the artificiality of the setup, we put several guardrails in place: shards were manually reviewed, merged, and edited to represent natural information splits (Section 3.1), and the user simulator selects shards contextually rather than in a fixed or random order (Section 3.2).

自然语言处理从业者（附录G.3）。我们提出的分片程序是半自动化的，但仍需要人工投入（每100个样本3小时）以确保质量。我们假设，最容易受到多轮对话性能下降影响的任务具有三个共同属性：(1) 它们是生成式（而非抽取式）任务，(2) 足够复杂，使得分片会产生 3+ 个分片/指令，以及 (3) 属于非片段式任务，每个新分片都需要修改整个解决方案。对于适用的任务，我们鼓励研究人员在发布完整数据集的同时，也发布其分片变体。

对话系统用户（附录G.4）。用户应意识到大语言模型在可靠性方面的局限，尤其是在多轮对话场景中。我们提供两条实用建议。首先，“如果时间允许，请重试”——使用相同信息开启一个新对话，通常比坚持使用一个已在对话中迷失的模型能获得更好的结果。其次，“在重试前进行整合”——由于大语言模型难以处理分散在多个轮次中的信息，将需求整合为一条指令可以同时提升其能力和可靠性。用户可以通过要求大语言模型将所有用户轮次整合成一条用于新对话的指令来实现这一点。这些策略仍是繁琐的权宜之计，而非原则性的解决方案，这突显了我们需要能够可靠处理多轮对话而无需此类干预的大语言模型。附录G 为上述列出的每个受众群体提供了额外的视角。

7 结论

在这项工作中，我们对大语言模型进行了大规模的单轮及多轮对话模拟，发现在一组固定任务上，大语言模型的性能在多轮、欠明确设定中显著下降。大语言模型会在对话中迷失，具体表现为可靠性显著降低，因为模型难以跨轮次维持上下文、过早做出假设并过度依赖其先前的响应。额外的实验揭示，在较简单场景中有效的已知补救措施（如类智能体拼接或降低温度）在多轮对话场景中无效，我们呼吁大语言模型构建者优先考虑模型在多轮对话场景中的可靠性。

8 伦理声明

以下部分反思了我们所展示工作的局限性。我们强调，本文的研究结果应被解读为一次受控的欠明确性压力测试的产物，而非对真实用户自然对话中大语言模型行为的直接测量。

我们工作的第一个局限在于依赖完全自动化的模拟。通过依赖大语言模型来模拟用户话语，我们可以扩展实验规模，包括多次运行同一模拟，而这对于真实用户而言成本过高。然而，我们获得的模拟并不能代表自然的人机对话。分片过程的特性（定义见附录C）和模拟环境的特性（见第3.2节）确保了模拟对话遵循一种相当狭窄的结构，很可能无法模拟庞大、多样化的用户群体中发生的全部对话动态。例如，模拟过程确保在每一轮次都揭示一个新的信息分片，并且对话的最后一轮已指定完成任务所需的全部信息，而这在真实用户中可能不会发生。分片过程的特性P1、P2和P5也限制了对话的范围，因为分片指令与现有的完全指定指令高度匹配，且高层意图总是在对话的第一轮次就被识别。分片的极简性质也是不现实的，并且可能具有对抗性，尽管渐进分片实验发现，只要对话发生在两轮或更多轮次，不同的分片粒度会导致相似的性能下降。除了分片粒度，自动模拟还缺乏人类参与对话时可能出现的细微差别，例如术语误解、因系统故障而感到沮丧而放弃（Wester等人,2024），或者某些对话缺乏可行的最终目标（例如，用户想要一个未解决问题的解决方案）。由于这些因素，我们认为所进行的模拟为大语言模型多轮对话能力提供了一个良性的测试场。为了减轻设置的人为性，我们设置了若干防护措施：分片经过人工审查、合并和编辑，以代表自然的信息分割（第3.1节），并且用户模拟器根据上下文选择分片，而非固定或随机顺序（第3.2节）。

These guardrails ensure a minimum level of conversational realism, though more work is needed to approach fully natural user simulation (Naous et al., 2025; Dou et al., 2025; Mehri et al., 2025; Ross & Andreas, 2025). **Because of the overly simplified conditions of simulation, we believe the degradation observed in experiments is most likely an underestimate of LLM unreliability, and how frequently LLMs get lost in conversation in real-world settings.** The experiments serve as a scalable, low-cost experimental environment for studying LLMs in multi-turn settings. We also note that the scoring mechanism itself favors the sharded setting: because the assistant may produce answer attempts on multiple turns, it can be credited for an early correct guess—whether due to benchmark memorization, lucky inference about missing shards, or lenient evaluation. Despite this structural advantage, multi-turn performance remains substantially below single-turn baselines, confirming that additional answer attempts do not compensate for the reliability loss observed in multi-turn conversations.

A second limitation of our work is the focus on analytical tasks. Although we selected a diverse set of both programming and natural language tasks, we restricted experiments to tasks that involve an analytical solution. This restriction limits the scope of our findings, as we do not establish whether models get lost in conversation on more open-ended tasks, such as creative writing (Chakrabarty et al., 2024). This was a conscious choice: though there has been some progress on creative writing evaluation, it is still an active area of research (Chakrabarty et al., 2025), and we relied on more established tasks and metrics for the initial set of experiments. Determining whether degradation occurs – and if so, identifying the magnitude – on creative tasks is an important direction for future work.

A third limitation of the work is the focus on text-only tasks in the English language. Establishing whether models get lost in conversation in other languages, or in tasks that involve multiple modalities in either user or assistant utterances, could help establish the scope of the degradation observed in LLM multi-turn capabilities.

9 Reproducibility Statement

We plan to release the code used for simulation, and relevant data such as the sharded instructions and corpus of simulated conversations publicly upon acceptance of the work. We hope the transparency will allow the community to verify and build upon our work, for instance by building future systems that are more reliable in multi-turn settings.

There are two considerations that limit the absolute reproducibility of our experiments. First, a majority of our experiments were conducted with API-based LLMs (closed-source), as they are widely considered the state-of-the-art for LLM at the time of our experiments. API providers are known to deprecate old models over time in favor of newer versions, which likely means running our exact experiments will not be possible once the models are retired. Second, all our experiments involve language models that are probabilistic in nature, leading to statistical variance in the results we produce. Though we took steps to improve the statistical robustness of our results (e.g., running each simulated conversations 10 independent times), scaling up the experiments further might help identify findings that we could not with our achieved experimental budget.

References

Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J Hewett, Mojan Javaheripi, Piero Kauffmann, et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024.

Catarina G Belem, Pouya Pezeskhpour, Hayate Iso, Seiji Maekawa, Nikita Bhutani, and Estevam Hruschka. From single to multi: How llms hallucinate in multi-document summarization. *arXiv preprint arXiv:2410.13961*, 2024.

Philipp Brauner, Alexander Hick, Ralf Philipsen, and Martina Ziefle. What does the public think about artificial intelligence?—a criticality map to understand bias in the public perception of ai. In *Frontiers of Computer Science*, 2023. URL https://api.semanticscholar.org/CorpusID:257598212.

这些防护措施确保了最低限度的对话真实性，但要接近完全自然的用户模拟还需要更多工作（Naous等人, 2025; Dou等人, 2025; Mehri等人, 2025;Ross&Andreas, 2025）。**由于模拟条件过于简化，我们认为实验中观察到的性能下降很可能低估了大语言模型的不可靠性，以及大语言模型在现实场景中陷入对话迷失的频率**。这些实验为研究大语言模型在多轮对话场景中提供了一个可扩展、低成本的实验环境。我们还注意到，评分机制本身有利于分片设置：因为助手可能在多个轮次产生答案尝试，它可以因早期正确猜测而获得认可——无论是由于基准记忆、对缺失分片的幸运推断，还是宽松的评估。尽管存在这种结构性优势，多轮对话性能仍然显著低于单轮基线，这证实了额外的答案尝试并不能弥补在多轮对话中观察到的可靠性损失。

我们工作的第二个局限是聚焦于分析性任务。尽管我们选取了编程和自然语言任务中多样化的集合，但我们将实验限制在涉及分析性解决方案的任务上。这一限制缩小了我们研究结果的适用范围，因为我们未能确定模型是否会在更具开放性的任务（例如创意写作）中陷入对话迷失（Chakrabarty等人, 2024）。这是一个有意识的选择：尽管创意写作评估已取得一些进展，但这仍是一个活跃的研究领域（Chakrabarty等人, 2025），我们在初始实验集中依赖了更成熟的任务和指标。确定在创意任务中是否会出现性能下降——如果会，识别其程度——是未来工作的重要方向。

这项工作的第三个局限是仅聚焦于英语的纯文本任务。确定模型在其他语言中，或在涉及用户或助手话语多模态的任务中是否会出现对话迷失，将有助于界定所观察到的大语言模型多轮对话能力下降的范围。

9 可复现性声明

我们计划在论文被接受后，公开发布用于模拟的代码，以及相关数据，如分片指令和模拟对话语料库。我们希望这种透明度能让社区验证并基于我们的工作继续发展，例如构建在未来多轮对话场景中更可靠的系统。

有两个因素限制了我们实验的绝对可复现性。首先，我们的大部分实验是使用基于API的大语言模型（闭源）进行的，因为在我们的实验进行时，它们被广泛认为是LLM领域的最先进技术。众所周知，API提供商会随时间推移淘汰旧模型，转而支持新版本，这很可能意味着一旦这些模型退役，我们将无法完全复现我们的实验。其次，我们所有的实验都涉及本质上具有概率性的语言模型，这导致了我们所得结果的统计方差。尽管我们采取了措施来提高结果的统计稳健性（例如，将每个模拟对话独立运行10次），但进一步扩大实验规模可能有助于发现我们在现有实验预算下未能识别的新发现。

参考文献

Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J Hewett, Mojan Javaheripi, Piero Kauffmann, 等人。Phi-4 技术报告。*arXiv* 预印本 *arXiv:2412.08905*，2024。

Catarina G Belem, Pouya Pezeskhpour, Hayate Iso, Seiji Maekawa, Nikita Bhutani, 以及 Estevam Hruschka。从单文档到多文档：大语言模型在多文档摘要中的幻觉问题。*arXiv* 预印本 *arXiv:2410.13961*，2024。

Philipp Brauner、Alexander Hick、Ralf Philipsen和Martina Ziefle。公众如何看待人工智能？——理解公众对AI认知偏见的关键性地图。载于计算机科学前沿，2023。网址 https://api.semanticscholar.org/CorpusID:257598212。

Tuhin Chakrabarty, Philippe Laban, Divyansh Agarwal, Smaranda Muresan, and Chien-Sheng Wu. Art or artifice? large language models and the false promise of creativity. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pp. 1–34, 2024.

Tuhin Chakrabarty, Philippe Laban, and Chien-Sheng Wu. Ai-slop to ai-polish? aligning language models through edit-based writing rewards and test-time computation. *arXiv preprint arXiv:2504.07532*, 2025.

Serina Chang, Ashton Anderson, and Jake M Hofman. Chatbench: From static benchmarks to human-ai evaluation. *arXiv preprint arXiv:2504.07114*, 2025.

Harrison Chase. Langchain, October 2022. URL https://github.com/langchain-ai/langchain.

Akshay Chaturvedi, Kate Thompson, and Nicholas Asher. Nebula: A discourse aware minecraft builder. *ArXiv*, abs/2406.18164, 2024. URL https://api.semanticscholar.org/CorpusID:270738020.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael Jordan, Joseph E Gonzalez, et al. Chatbot arena: An open platform for evaluating llms by human preference. In *Forty-first International Conference on Machine Learning*, 2024.

Eunsol Choi, He He, Mohit Iyyer, Mark Yatskar, Wen-tau Yih, Yejin Choi, Percy Liang, and Luke Zettlemoyer. Quac: Question answering in context. *arXiv preprint arXiv:1808.07036*, 2018.

Eunsol Choi, Jennimaria Palomaki, Matthew Lamm, Tom Kwiatkowski, Dipanjan Das, and Michael Collins. Decontextualization: Making sentences stand-alone. *Transactions of the Association for Computational Linguistics*, 9:447–461, 2021.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Team Cohere, Arash Ahmadian, Marwan Ahmed, Jay Alammar, Yazeed Alnumay, Sophia Althammer, Arkady Arkhangorodsky, Viraat Aryabumi, Dennis Aumiller, Raphaël Avalos, et al. Command a: An enterprise-ready large language model. *arXiv preprint arXiv:2504.00698*, 2025.

Yang Deng, Xuan Zhang, Wenxuan Zhang, Yifei Yuan, See-Kiong Ng, and Tat-Seng Chua. On the multi-turn instruction following for conversational web agents. *arXiv preprint arXiv:2402.15057*, 2024.

Jan Deriu, Alvaro Rodrigo, Arantxa Otegi, Guillermo Echegoyen, Sophie Rosset, Eneko Agirre, and Mark Cieliebak. Survey on evaluation methods for dialogue systems. *Artificial Intelligence Review*, 54:755–810, 2021.

Yao Dou, Michel Galley, Baolin Peng, Chris Kedzie, Weixin Cai, Alan Ritter, Chris Quirk, Wei Xu, and Jianfeng Gao. Simulatorarena: Are user simulators reliable proxies for multi-turn evaluation of ai assistants? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 35200–35278, 2025.

Victor S Ferreira. Ambiguity, accessibility, and a division of labor for communicative success. *Psychology of Learning and motivation*, 49:209–246, 2008.

Sarah E Finch, James D Finch, and Jinho D Choi. Don't forget your abc's: Evaluating the state-of-the-art in chat-oriented dialogue systems. In *The 61st Annual Meeting Of The Association For Computational Linguistics*, 2023.

Steven Frisson. Semantic underspecification in language processing. *Lang. Linguistics Compass*, 3: 111–127, 2009. URL https://api.semanticscholar.org/CorpusID:13384476.

Tuhin Chakrabarty、Philippe Laban、Divyansh Agarwal、Smaranda Muresan和吴建升。艺术还是人工制品？大语言模型与创造力的虚假承诺。载于*2024*年计算机系统中的人为因素*CHI*会议论文集，第1–34页，2024。Tuhin Chakrabarty、Philippe Laban和吴建升。从AI-粗制滥造到AI-润色？通过基于编辑的写作奖励和测试时计算对齐语言模型。*arXiv*预印本 *arXiv:2504.07532*，2025。Serina Chang、Ashton Anderson和Jake M Hofman。Chatbench：从静态基准测试到人机评估。*arXiv*预印本 *arXiv:2504.07114*，2025。Harrison Chase。Langchain，2022年10月。网址 https://github.com/langchain-ai/langchain。Akshay Chaturvedi、Kate Thompson和Nicholas Asher。Nebula：一个语篇感知的Minecraft建造者。*ArXiv*，abs/2406.18164，2024。网址 https://api.semanticscholar.org/CorpusID:270738020。Mark Chen、Jerry Tworek、Heewoo Jun、Qiming Yuan、Henrique Ponde De Oliveira Pinto、Jared Kaplan、Harri Edwards、Yuri Burda、Nicholas Joseph、Greg Brockman等。评估在代码上训练的大语言模型。*arXiv*预印本 *arXiv:2107.03374*，2021。Wei-Lin Chiang、Lianmin Zheng、Ying Sheng、Anastasios Nikolas Angelopoulos、Tianle Li、Dacheng Li、Banghua Zhu、Hao Zhang、Michael Jordan、Joseph E Gonzalez等。聊天机器人竞技场：一个基于人类偏好评估大语言模型的开放平台。载于第四十一届国际机器学习大会，2024。Eunsol Choi、He He、Mohit Iyyer、Mark Yatskar、Wen-tau Yih、Yejin Choi、Percy Liang和Luke Zettlemoyer。QuAC：上下文问答。*arXiv*预印本 *arXiv:1808.07036*，2018。Eunsol Choi、Jennimaria Palomaki、Matthew Lamm、Tom Kwiatkowski、Dipanjan Das和Michael Collins。去语境化：使句子独立成句。计算语言学协会汇刊，9:447–461，2021。Karl Cobbe、Vineet Kosaraju、Mohammad Bavarian、Mark Chen、Heewoo Jun、Lukasz Kaiser、Matthias Plappert、Jerry Tworek、Jacob Hilton、Reiichiro Nakano等。训练验证器解决数学应用题。*arXiv*预印本 *arXiv:2110.14168*，2021。Cohere团队、Arash Ahmadian、Marwan Ahmed、Jay Alammar、Yazeed Alnumay、Sophia Althammer、Arkady Arkhangorodsky、Viraat Aryabumi、Dennis Aumiller、Raphaël Avalos等。Command A：一个企业级大语言模型。*arXiv*预印本 *arXiv:2504.00698*，2025。Yang Deng、Xuan Zhang、Wenxuan Zhang、Yifei Yuan、See-Kiong Ng和Tat-Seng Chua。论对话式网络智能体的多轮指令遵循。*arXiv*预印本 *arXiv:2402.15057*，2024。Jan Deriu、Alvaro Rodrigo、Arantxa Otegi、Guillermo Echegoyen、Sophie Rosset、Eneko Agirre和Mark Cieliebak。对话系统评估方法综述。人工智能评论，54:755–810，2021。Yao Dou、Michel Galley、Baolin Peng、Chris Kedzie、Weixin Cai、Alan Ritter、Chris Quirk、Wei Xu和Jianfeng Gao。模拟器竞技场：用户模拟器是评估AI助手多轮性能的可靠代理吗？载于*2025*年自然语言处理实证方法会议论文集，第35200–35278页，2025。Victor S Ferreira。歧义性、可及性与沟通成功的分工。学习与动机心理学，49:209–246，2008。Sarah E Finch、James D Finch和Jinho D Choi。别忘了你的ABC：评估面向聊天的对话系统的最新技术。载于计算语言学协会第*61*届年会，2023。Steven Frisson。语言处理中的语义欠明确性。语言与语言学指南，3: 111–127，2009。网址 https://api.semanticscholar.org/CorpusID:13384476。

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Kunal Handa, Alex Tamkin, Miles McCain, Saffron Huang, Esin Durmus, Sarah Heck, Jared Mueller, Jerry Hong, Stuart Ritchie, Tim Belonax, et al. Which economic tasks are performed with ai? evidence from millions of claude conversations. *arXiv preprint arXiv:2503.04761*, 2025.

Christine Herlihy, Jennifer Neville, Tobias Schnabel, and Adith Swaminathan. On overcoming miscalibrated conversational priors in llm-based chatbots. *arXiv preprint arXiv:2406.01633*, 2024.

Michael C Horowitz, Lauren Kahn, Julia Macdonald, and Jacquelyn Schneider. Adopting ai: how familiarity breeds both trust and contempt. *AI & society*, 39(4):1721–1735, 2024.

Kung-Hsiang Huang, Philippe Laban, Alexander R Fabbri, Prafulla Kumar Choubey, Shafiq Joty, Caiming Xiong, and Chien-Sheng Wu. Embrace divergence for richer insights: A multi-document summarization benchmark and a case study on summarizing diverse information from news articles. *arXiv preprint arXiv:2309.09369*, 2023.

Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Marzena Karpinska, Katherine Thai, Kyle Lo, Tanya Goyal, and Mohit Iyyer. One thousand and one pairs: A" novel" challenge for long-context language models. *arXiv preprint arXiv:2406.16264*, 2024.

Yekyung Kim, Yapei Chang, Marzena Karpinska, Aparna Garimella, Varun Manjunatha, Kyle Lo, Tanya Goyal, and Mohit Iyyer. Fables: Evaluating faithfulness and content selection in book-length summarization. *arXiv preprint arXiv:2404.01261*, 2024a.

Yoonsu Kim, Kihoon Son, Seoyoung Kim, and Juho Kim. Beyond prompts: Learning from human communication for enhanced ai intent alignment. *ArXiv*, abs/2405.05678, 2024b. URL https://api.semanticscholar.org/CorpusID:269635257.

Nils Knoth, Antonia Tolzin, Andreas Janson, and Jan Marco Leimeister. Ai literacy and its implications for prompt engineering strategies. *Comput. Educ. Artif. Intell.*, 6:100225, 2024. URL https://api.semanticscholar.org/CorpusID:269273689.

Jakub Konrád, Jan Pichl, Petr Marek, Petr Lorenc, Van Duy Ta, Ondřej Kobza, Lenka Hỳlová, and Jan Šedivỳ. Alquist 4.0: Towards social intelligence using generative models and dialogue personalization. *arXiv preprint arXiv:2109.07968*, 2021.

Wai-Chung Kwan, Xingshan Zeng, Yuxin Jiang, Yufei Wang, Liangyou Li, Lifeng Shang, Xin Jiang, Qun Liu, and Kam-Fai Wong. Mt-eval: A multi-turn capabilities evaluation benchmark for large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 20153–20177, 2024.

Philippe Laban, John Canny, and Marti A Hearst. What's the latest? a question-driven news chatbot. *arXiv preprint arXiv:2105.05392*, 2021.

Philippe Laban, Lidiya Murakhovs' ka, Caiming Xiong, and Chien-Sheng Wu. Are you sure? challenging llms leads to performance drops in the flipflop experiment. *arXiv preprint arXiv:2311.08596*, 2023.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, 等人。Llama 3模型群。*arXiv*预印本 *arXiv:2407.21783*，2024。郭达雅, 杨德建, 张浩伟, 宋俊骁, 张若愚, 徐润欣, 朱启昊, 马世荣, 王培毅, 毕啸, 等人。Deepseek-R1：通过强化学习激励大语言模型的推理能力。*arXiv*预印本 *arXiv:2501.12948*，2025。Kunal Handa, Alex Tamkin, Miles McCain, Saffron Huang, Esin Durmus, Sarah Heck, Jared Mueller, Jerry Hong, Stuart Ritchie, Tim Belonax, 等人。哪些经济任务由人工智能执行? 来自数百万Claude对话的证据。*arXiv*预印本 *arXiv:2503.04761*，2025。Christine Herlihy, Jennifer Neville, Tobias Schnabel, 和 Adith Swaminathan。论克服基于LLM的聊天机器人中校准不当的对话先验。*arXiv*预印本 *arXiv:2406.01633*，2024。Michael C Horowitz, Lauren Kahn, Julia Macdonald, 和 Jacquelyn Schneider。采用人工智能：熟悉如何既滋生信任又滋生蔑视。*AI*与社会，39(4):1721–1735，2024。

黄孔祥、Philippe Laban、Alexander R Fabbri、Prafulla Kumar Choubey、Shafiq Joty、熊才明和吴建升。拥抱分歧以获取更丰富的洞见：一个多文档摘要基准及关于总结新闻文章中多样化信息的案例研究。*arXiv*预印本 *arXiv:2309.09369*，2023。

Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Os- trow, Akila Welihinda, Alan Hayes, Alec Radford, 等人。GPT-4o系统卡片。 *arXiv*预印本 *arXiv:2410.21276*, 2024。Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, 和 Ion Stoica。LiveCodeBench：代码大语言模型的整体且无污染评估。 *arXiv*预印本 *arXiv:2403.07974*, 2024。Marzena Karpinska, Katherine Thai, Kyle Lo, Tanya Goyal, 和 Mohit Iyyer。一千零一对：长上下文语言模型的一项“新颖”挑战。 *arXiv*预印本 *arXiv:2406.16264*, 2024。Yekyung Kim, Yapei Chang, Marzena Karpinska, Aparna Garimella, Varun Manjunatha, Kyle Lo, Tanya Goyal, 和 Mohit Iyyer。Fables：评估书籍长度摘要的忠实性与内容选择。*arXiv*预印本 *arXiv:2404.01261*, 2024a。Yoonsu Kim, Kihoon Son, Seoyoung Kim, 和 Juho Kim。超越提示词：从人类交流中学习以增强 AI 意图对齐。 *ArXiv*, abs/2405.05678, 2024b。URL https://api.semanticscholar.org/CorpusID:269635257。Nils Knoth, Antonia Tolzin, Andreas Janson, 和 Jan Marco Leimeister。AI素养及其对提示工程策略的启示。 计算机、教育与人工智能, 6:100225, 2024。URL https://api.semanticscholar.org/CorpusID:269273689。Jakub Konrád, Jan Pichl, Petr Marek, Petr Lorenc, Van Duy Ta, Ondˇrej Kobza, Lenka H`ylová, 和 Jan Šediv`y。Alquist 4.0：利用生成模型和对话个性化实现社交智能。 *arXiv*预印本 *arXiv:2109.07968*, 2021。Wai-Chung Kwan, Xingshan Zeng, Yuxin Jiang, Yufei Wang, Liangyou Li, Lifeng Shang, Xin Jiang, Qun Liu, 和 Kam-Fai Wong。Mt-eval：大语言模型的多轮能力评估基准。收录于 *2024*年自然语言处理实证方法会议论文集, 第 20153–20177 页, 2024。Philippe Laban, John Canny, 和 Marti A Hearst。最新消息是什么？一个由问题驱动的新闻聊天机器人。*arXiv*预印本 *arXiv:2105.05392*, 2021。Philippe Laban, Lidiya Murakhovs' ka, Caiming Xiong, 和 Chien-Sheng Wu。你确定吗？挑战大语言模型会导致Flipflop实验中的性能下降。 *arXiv*预印本 *arXiv:2311.08596*, 2023。

Philippe Laban, Alexander R Fabbri, Caiming Xiong, and Chien-Sheng Wu. Summary of a haystack: A challenge to long-context llms and rag systems. *arXiv preprint arXiv:2407.01370*, 2024.

Shalom Lappin. An intensional parametric semantics for vague quantifiers. *Linguistics and Philosophy*, 23:599–620, 2000. URL https://api.semanticscholar.org/CorpusID:170154611.

Yoonjoo Lee, Kihoon Son, Tae Soo Kim, Jisu Kim, John Joon Young Chung, Eytan Adar, and Juho Kim. One vs. many: Comprehending accurate information from multiple erroneous and inconsistent ai generations. *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, 2024. URL https://api.semanticscholar.org/CorpusID:269635304.

Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, et al. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. *arXiv preprint arXiv:2411.07763*, 2024.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*, 2019.

Ruosen Li, Ruochen Li, Barry Wang, and Xinya Du. Iqa-eval: Automatic evaluation of human-model interactive question answering. *Advances in Neural Information Processing Systems*, 37: 109894–109921, 2024.

Shiyang Li, Jun Yan, Hai Wang, Zheng Tang, Xiang Ren, Vijay Srinivasan, and Hongxia Jin. Instruction-following evaluation through verbalizer manipulation. *arXiv preprint arXiv:2307.10558*, 2023.

Zhenwen Liang, Dian Yu, Wenhao Yu, Wenlin Yao, Zhihan Zhang, Xiangliang Zhang, and Dong Yu. Mathchat: Benchmarking mathematical reasoning and instruction following in multi-turn interactions. *arXiv preprint arXiv:2405.19444*, 2024.

Alisa Liu, Zhaofeng Wu, Julian Michael, Alane Suhr, Peter West, Alexander Koller, Swabha Swayamdipta, Noah A Smith, and Yejin Choi. We're afraid language models aren't modeling ambiguity. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 790–807, 2023.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

Yixin Liu, Alexander R Fabbri, Pengfei Liu, Yilun Zhao, Linyong Nan, Ruilin Han, Simeng Han, Shafiq Joty, Chien-Sheng Wu, Caiming Xiong, et al. Revisiting the gold standard: Grounding summarization evaluation with robust human evaluation. *arXiv preprint arXiv:2212.07981*, 2022.

Zhiyi Ma, Sergey Edunov, and Michael Auli. A comparison of approaches to document-level machine translation. *arXiv preprint arXiv:2101.11040*, 2021.

Chaitanya Malaviya, Joseph Chee Chang, Dan Roth, Mohit Iyyer, Mark Yatskar, and Kyle Lo. Contextualized evaluations: Taking the guesswork out of language model evaluations. *arXiv preprint arXiv:2411.07237*, 2024.

Shuhaib Mehri, Xiaocheng Yang, Takyoung Kim, Gokhan Tur, Shikib Mehri, and Dilek Hakkani-Tür. Goal alignment in llm-based user simulators for conversational ai. *arXiv preprint arXiv:2507.20152*, 2025.

Lidiya Murakhovs' ka, Philippe Laban, Tian Xie, Caiming Xiong, and Chien-Sheng Wu. Salespeople vs salesbot: Exploring the role of educational value in conversational recommender systems. *arXiv preprint arXiv:2310.17749*, 2023.

Michael Mylrea and Nikki Robinson. Artificial intelligence (ai) trust framework and maturity model: Applying an entropy lens to improve security, privacy, and ethical ai. *Entropy*, 25, 2023. URL https://api.semanticscholar.org/CorpusID:263840323.

Philippe Laban, Alexander R Fabbri, Caiming Xiong, 和 Chien-Sheng Wu。大海捞针摘要：对长上下文大语言模型和检索增强生成系统的挑战。 *arXiv*预印本 *arXiv:2407.01370*, 2024。Shalom Lappin。模糊量词的内涵参数语义学。 语言学与哲学, 23:599–620, 2000。URL https://api.semanticscholar.org/CorpusID:170154611。

Yoonjoo Lee, Kihoon Son, Tae Soo Kim, Jisu Kim, John Joon Young Chung, Eytan Adar, 和 Juho Kim。一对多：从多重错误且不一致的人工智能生成中理解准确信息。 *2024*年*ACM*公平、问责与透明度会议论文集, 2024。URL https://api.semanticscholar.org/CorpusID:269635304。Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, 等人。Spider 2.0：在真实世界企业级文本到SQL工作流中评估语言模型。 *arXiv*预印本 *arXiv:2411.07763*, 2024。Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, 和 Luke Zettlemoyer。BART：用于自然语言生成、翻译和理解的去噪序列到序列预训练。 *arXiv*预印本 *arXiv:1910.13461*, 2019。Ruosen Li, Ruochen Li, Barry Wang, 和 Xinya Du。IQA-Eval：人机交互式问答的自动评估。 神经信息处理系统进展, 37: 109894–109921, 2024。Shiyang Li, Jun Yan, Hai Wang, Zheng Tang, Xiang Ren, Vijay Srinivasan, 和 Hongxia Jin。通过词表操纵进行指令遵循评估。 *arXiv*预印本 *arXiv:2307.10558*, 2023。Zhenwen Liang, Dian Yu, Wenhao Yu, Wenlin Yao, Zhihan Zhang, Xiangliang Zhang, 和 Dong Yu。MathChat：在多轮交互中评估数学推理与指令遵循能力。 *arXiv*预印本 *arXiv:2405.19444*, 2024。Alisa Liu, Zhaofeng Wu, Julian Michael, Alane Suhr, Peter West, Alexander Koller, Swabha Swayamdipta, Noah A Smith, 和 Yejin Choi。我们担心语言模型没有建模歧义性。收录于 *2023*年自然语言处理经验方法会议论文集, 第 790–807 页, 2023。Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, 和 Percy Liang。迷失在中间：语言模型如何使用长上下文。 计算语言学协会汇刊, 12:157–173, 2024。Yixin Liu, Alexander R Fabbri, Pengfei Liu, Yilun Zhao, Linyong Nan, Ruilin Han, Simeng Han, Shafiq Joty, Chien-Sheng Wu, Caiming Xiong, 等人。重新审视黄金标准：通过稳健的人工评估夯实摘要评估基础。 *arXiv*预印本 *arXiv:2212.07981*, 2022。Zhiyi Ma, Sergey Edunov, 和 Michael Auli。文档级机器翻译方法比较。 *arXiv*预印本 *arXiv:2101.11040*, 2021。Chaitanya Malaviya, Joseph Chee Chang, Dan Roth, Mohit Iyyer, Mark Yatskar, 和 Kyle Lo。情境化评估：消除语言模型评估中的猜测。 *arXiv*预印本 *arXiv:2411.07237*, 2024。Shuhaib Mehri, Xiaocheng Yang, Takyoung Kim, Gokhan Tur, Shikib Mehri, 和 Dilek Hakkani-Tür。对话式人工智能中基于大语言模型的用户模拟器的目标对齐。 *arXiv*预印本 *arXiv:2507.20152*, 2025。Lidiya Murakhovs' ka, Philippe Laban, Tian Xie, Caiming Xiong, 和 Chien-Sheng Wu。销售人员 vs 销售机器人：探索教育价值在对话式推荐系统中的作用。 *arXiv*预印本 *arXiv:2310.17749*, 2023。Michael Mylrea 和 Nikki Robinson。人工智能信任框架与成熟度模型：应用熵视角改进安全、隐私和伦理人工智能。 熵, 25, 2023。URL https://api.semanticscholar.org/CorpusID:263840323。

Tarek Naous, Philippe Laban, Wei Xu, and Jennifer Neville. Flipping the dialogue: Training and evaluating user language models. *arXiv preprint arXiv:2510.06552*, 2025.

Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, et al. 2 olmo 2 furious. *arXiv preprint arXiv:2501.00656*, 2024.

OpenAI. OpenAI o3 and o4-mini System Card — openai.com. https://openai.com/index/o3-o4-mini-system-card/, 2025. [Accessed 08-05-2025].

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pp. 311–318, 2002.

Ankur P Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqui, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. Totto: A controlled table-to-text generation dataset. *arXiv preprint arXiv:2004.14373*, 2020.

Hao Peng, Xiaozhi Wang, Jianhui Chen, Weikai Li, Yun Peng Qi, Zimu Wang, Zhili Wu, Kaisheng Zeng, Bin Xu, Lei Hou, and Juanzi Li. When does in-context learning fall short and why? a study on specification-heavy tasks. *ArXiv*, abs/2311.08993, 2023. URL https://api.semanticscholar.org/CorpusID:265212914.

Sandro Pezzelle. Dealing with semantic underspecification in multimodal nlp. *arXiv preprint arXiv:2306.05240*, 2023.

Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. Humanity's last exam. *arXiv preprint arXiv:2501.14249*, 2025.

Christian Poelitz and Nick McKenna. Synthetic clarification and correction dialogues about data-centric tasks–a teacher-student approach. *arXiv preprint arXiv:2503.14167*, 2025.

Matt Post and Marcin Junczys-Dowmunt. Escaping the sentence-level paradigm in machine translation. *arXiv preprint arXiv:2304.12959*, 2023.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

Ashwin Ram, Rohit Prasad, Chandra Khatri, Anu Venkatesh, Raefer Gabriel, Qing Liu, Jeff Nunn, Behnam Hedayatnia, Ming Cheng, Ashish Nagar, et al. Conversational ai: The science behind the alexa prize. *arXiv preprint arXiv:1801.03604*, 2018.

Siva Reddy, Danqi Chen, and Christopher D Manning. Coqa: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266, 2019.

Alexis Ross and Jacob Andreas. Learning to make mistakes: Modeling incorrect student thinking and key errors. *arXiv preprint arXiv:2510.11502*, 2025.

Rupak Sarkar, Bahareh Sarrafzadeh, Nirupama Chandrasekaran, N.Kasturi Rangan, Philip Resnik, Longqi Yang, and Sujay Kumar Jauhar. Conversational user-ai intervention: A study on prompt rewriting for improved llm response generation. *ArXiv*, abs/2503.16789, 2025. URL https://api.semanticscholar.org/CorpusID:277244656.

Yves Scherrer, Jörg Tiedemann, and Sharid Loáiciga. Analysing concatenation approaches to document-level nmt in two different domains. In *Proceedings of the Third Workshop on Discourse in Machine Translation*, Hong-Kong, November 2019. Association for Computational Linguistics.

Omar Shaikh, Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. Navigating rifts in human-llm grounding: Study and benchmark. *arXiv preprint arXiv:2503.13975*, 2025.

Tarek Naous、Philippe Laban、Wei Xu 和 Jennifer Neville。翻转对话：训练与评估用户语言模型。arXiv预印本 arXiv:2510.06552，2025。OLMo团队、Pete Walsh、Luca Soldaini、Dirk Groeneveld、Kyle Lo、Shane Arora、Akshita Bhagia、顾玉玲、黄圣一、Matt Jordan 等。2 olmo 2 furious。arXiv预印本 arXiv:2501.00656，2024。OpenAI。OpenAI o3 和 o4-mini 系统卡 — openai.com。 https://openai.com/index/o3-o4-mini-system-card/，2025。 [访问于 08-05-2025]。Kishore Papineni、Salim Roukos、Todd Ward 和 朱伟静。Bleu：一种机器翻译自动评估方法。载于《计算语言学协会第40届年会论文集》，第311–318页，2002。Ankur P Parikh、王学志、Sebastian Gehrmann、Manaal Faruqui、Bhuwan Dhingra、杨迪一 和 Dipanjan Das。Totto：一个受控表格到文本生成数据集。arXiv预印本 arXiv:2004.14373，2020。彭浩、王晓智、陈建辉、李伟凯、齐云鹏、王子木、吴志立、曾凯生、徐斌、侯磊 和 李涓子。上下文学习何时会失败？为什么？一项关于规范密集型任务的研究。ArXiv，abs/2311.08993，2023。URL https://api.semanticscholar.org/CorpusID:265212914。桑德罗·佩泽莱。处理多模态自然语言处理中的语义欠明确性。arXiv预印本 arXiv:2306.05240，2023。Long Phan、Alice Gatti、韩子文、李纳撒尼尔、胡约瑟芬娜、张休、张晨博·卡尔文、Mohamed Shaaban、John Ling、石肖恩 等。人类的最后考试。arXiv预印本 arXiv:2501.14249，2025。Christian Poelitz 和 Nick McKenna。关于以数据为中心任务的合成澄清与纠正对话——一种师生方法。arXiv预印本 arXiv:2503.14167，2025。Matt Post 和 Marcin Junczys-Dowmunt。逃离机器翻译中的句子级范式。arXiv预印本 arXiv:2304.12959，2023。Alec Radford、Jeffrey Wu、Rewon Child、David Luan、Dario Amodei、Ilya Sutskever 等。语言模型是无监督多任务学习器。OpenAI博客，1(8):9，2019。Colin Raffel、Noam Shazeer、Adam Roberts、Katherine Lee、Sharan Narang、Michael Matena、周彦祺、李威 和 刘鹏杰。探索迁移学习的极限：一个统一文本到文本转换器。机器学习研究杂志，21(140):1–67，2020。Ashwin Ram、Rohit Prasad、Chandra Khatri、Anu Venkatesh、Raefer Gabriel、刘青、Jeff Nunn、Behnam Hedayatnia、程明、Ashish Nagar 等。对话式人工智能：Alexa奖背后的科学。arXiv预印本 arXiv:1801.03604，2018。Siva Reddy、陈丹琦 和 Christopher D Manning。Coqa：一个对话式问答挑战赛。计算语言学协会汇刊，7:249–266，2019。Alexis Ross 和 Jacob Andreas。学习犯错：模拟错误的学生思维和关键错误。arXiv预印本 arXiv:2510.11502，2025。Rupak Sarkar、Bahareh Sarrafzadeh、Nirupama Chandrasekaran、N.Kasturi Rangan、Philip Resnik、杨龙奇 和 Sujay Kumar Jauhar。对话式用户-人工智能干预：一项关于提示词改写以改进大语言模型响应生成的研究。ArXiv，abs/2503.16789，2025。URL https://api.semanticscholar.org/CorpusID:277244656。Yves Scherrer、Jörg Tiedemann 和 Sharid Loáiciga。分析两个不同领域中用于文档级神经机器翻译的串联方法。载于《机器翻译中语篇处理第三次研讨会论文集》，香港，2019年11月。计算语言学协会。Omar Shaikh、Hussein Mozannar、加甘·班萨尔、Adam Fourney 和 Eric Horvitz。导航人-大语言模型对齐中的裂痕：研究与基准。arXiv预印本 arXiv:2503.13975，2025。

Jane Southworth, Kati Migliaccio, Joe Glover, Ja'Net Glover, David Reed, Christopher McCarty, Joel Brendemuhl, and Aaron Thomas. Developing a model for ai across the curriculum: Transforming the higher education landscape via innovation in ai literacy. *Computers and Education: Artificial Intelligence*, 4:100127, 2023.

Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

Michael Terry, Chinmay Kulkarni, Martin Wattenberg, Lucas Dixon, and Meredith Ringel Morris. Interactive ai alignment: specification, process, and evaluation alignment. *arXiv preprint arXiv:2311.00710*, 2023.

Pranav Narayanan Venkit, Philippe Laban, Yilun Zhou, Yixin Mao, and Chien-Sheng Wu. Search engines in an ai era: The false promise of factual and verifiable source-cited responses. *arXiv preprint arXiv:2410.22349*, 2024.

Sanidhya Vijayvargiya, Xuhui Zhou, Akhila Yerukola, Maarten Sap, and Graham Neubig. Interactive agents to overcome ambiguity in software engineering. *arXiv preprint arXiv:2502.13069*, 2025.

Justin D. Weisz, Jessica He, Michael Muller, Gabriela Hoefer, Rachel Miles, and Werner Geyer. Design principles for generative ai applications. *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024. URL https://api.semanticscholar.org/CorpusID:267301068.

Joel Wester, Tim Schrills, Henning Pohl, and Niels van Berkel. "as an ai language model, i cannot": Investigating llm denials of user requests. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pp. 1–14, 2024.

Frank Wildenburg, Michael Hanna, and Sandro Pezzelle. Do pre-trained language models detect and understand semantic underspecification? ask the dust! *ArXiv*, abs/2402.12486, 2024. URL https://api.semanticscholar.org/CorpusID:267759784.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Berkeley function calling leaderboard. https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html, 2024.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887*, 2018.

J Diego Zamfirescu-Pereira, Richmond Y Wong, Bjoern Hartmann, and Qian Yang. Why johnny can't prompt: how non-ai experts try (and fail) to design llm prompts. In *Proceedings of the 2023 CHI conference on human factors in computing systems*, pp. 1–21, 2023.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P Xing, et al. Lmsys-chat-1m: A large-scale real-world llm conversation dataset. *arXiv preprint arXiv:2309.11998*, 2023a.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023b.

Ruiqi Zhong, Tao Yu, and Dan Klein. Semantic evaluation for text-to-sql with distilled test suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 396–411, 2020.

George Kingsley Zipf. *Human behavior and the principle of least effort: An introduction to human eoclogy*. Addison-Wesley Press, 1949.

简·索斯沃斯、卡蒂·米利亚乔、乔·格洛弗、贾内特·格洛弗、大卫·里德、克里斯托弗·麦卡蒂、乔尔·布伦德穆尔 和 亚伦·托马斯。开发跨课程人工智能模型：通过AI素养创新变革高等教育格局。《计算机与教育：人工智能》，4:100127，2023。

Gemini团队、罗汉·阿尼尔、塞巴斯蒂安·博尔若、让-巴蒂斯特·阿莱拉克、余嘉辉、拉杜·索里库特、约翰·沙尔克维克、安德鲁·M·戴、安雅·豪特、凯蒂·米利肯等。Gemini：一个能力卓越的多模态模型家族。*arXiv*预印本 *arXiv:2312.11805*，2023。

迈克尔·特里、钦迈·库尔卡尼、马丁·瓦滕伯格、卢卡斯·迪克森和梅雷迪思·林格尔·莫里斯。交互式人工智能对齐：规范、过程与评估对齐。*arXiv*预印本 *arXiv:2311.00710*，2023。

普拉纳夫·纳拉亚南·文基特、Philippe Laban、周逸伦、毛一昕和吴建升。人工智能时代的搜索引擎：对事实性和可验证的引用来源回应的虚假承诺。*arXiv*预印本 *arXiv:2410.22349*，2024。

萨尼迪亚·维贾瓦尔吉亚、周旭辉、阿基拉·耶鲁科拉、马尔滕·萨普与格雷厄姆·纽比格。交互式智能体克服软件工程中的歧义性。*arXiv*预印本 *arXiv:2502.13069*，2025。

贾斯汀·D·魏斯、何洁西卡、迈克尔·穆勒、加布里埃拉·霍费尔、雷切尔·迈尔斯与维尔纳·盖尔。生成式人工智能应用的设计原则。计算机系统中的人为因素*CHI*会议论文集，2024。网址 https://api.semanticscholar.org/CorpusID:267301068。

乔尔·韦斯特、蒂姆·施里尔斯、亨宁·波尔与尼尔斯·范·伯克尔。"作为一个人工智能语言模型，我无法"：调查大语言模型对用户请求的拒绝。载于*2024*年计算机系统中的人为因素*CHI*会议论文集，第1–14页，2024。

弗兰克·维尔登堡、迈克尔·汉纳与桑德罗·佩泽莱。预训练语言模型能否检测并理解语义欠明确性？去问尘埃！*ArXiv*，abs/2402.12486，2024。网址https://api.semanticscholar.org/CorpusID:267759784。

吴青云、加甘·班萨尔、张洁玉、吴亦然、李贝宾、朱尔康、江莉、张晓云、张少坤、刘佳乐等。Autogen：通过多智能体对话实现下一代LLM应用。*arXiv*预印本 *arXiv:2308.08155*，2023。

Fanjia Yan、Huanzhi Mao、Charlie Cheng-Jie Ji、Tianjun Zhang、Shishir G. Patil、Ion Stoica 和 JosephE. Gonzalez。伯克利函数调用排行榜。 https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html，2024。

Tao Yu、Rui Zhang、Kai Yang、Michihiro Yasunaga、Dongxu Wang、Zifan Li、James Ma、Irene Li、Qingning Yao、Shanelle Roman等。Spider：一个用于复杂跨领域语义解析和文本到SQL任务的大规模人工标注数据集。*arXiv*预印本 *arXiv:1809.08887*，2018。

J Diego Zamfirescu-Pereira、Richmond Y Wong、Bjoern Hartmann 和 Qian Yang。为什么 Johnny 不会写提示词：非人工智能专家如何尝试（并失败）设计大语言模型提示词。载于《*2023*年计算机系统中的人为因素*CHI*会议论文集》，第1–21页，2023。

Lianmin Zheng、Wei-Lin Chiang、Ying Sheng、Tianle Li、Siyuan Zhuang、Zhanghao Wu、Yonghao Zhuang、Zhuohan Li、Zi Lin、Eric P Xing 等。Lmsys-chat-1m：一个大规模真实世界大语言模型对话数据集。*arXiv*预印本 *arXiv:2309.11998*，2023a。

Lianmin Zheng、Wei-Lin Chiang、Ying Sheng、Siyuan Zhuang、Zhanghao Wu、Yonghao Zhuang、Zi Lin、Zhuohan Li、Dacheng Li、Eric Xing 等。使用 mt-bench 和聊天机器人竞技场评判大语言模型作为评判者。《神经信息处理系统进展》，第36卷：46595–46623页，2023b。

Ruiqi Zhong, Tao Yu, and Dan Klein. 文本转SQL的语义评估与蒸馏测试套件。 收录于*2020*年自然语言处理实证方法会议（*EMNLP*）论文集, 第396–411页, 2020年。 George Kingsley Zipf. 人类行为与最小努力原则：人类生态学导论. Addison-Wesley出版社, 1949年。

A Related work on Underspecification

The Background (Section 2) reviews the most directly related prior work, focused on multi-turn evaluation. We now cover other related prior works that have studied underspecification.

Prior work on communication and linguistics has identified underspecification as a common feature of human language (Lappin, 2000; Ferreira, 2008; Frisson, 2009; Pezzelle, 2023).

Understanding how LLMs handle underspecified instructions is crucial for improving conversational capabilities. To this end, Herlihy et al. (2024) identified common response patterns such as hedging, refusal, clarification, and interrogation when underspecified queries are presented to conversational LLM systems, and proposed mechanisms to recover from them. Malaviya et al. (2024) highlighted the importance of supporting context for more accurate and principled evaluation of LLM responses on underspecified queries, and Sarkar et al. (2025) showed that a system that proactively rewrites user instructions to account for underspecification leads to improved LLM responses. Shaikh et al. (2025) studied the degree of grounding (*i.e.*, clarifications and follow-up questions) that LLMs perform in conversation logs and observed that they significantly lack in generating follow-up questions, where humans are 15 times more likely to do so. Chang et al. (2025) hired annotators to manually reproduce fully-specified instructions through a chat interface, and found that the users reveal the entirety of the instruction in 34% of the time, leaving some detail underspecified a majority of the time.

Several works have explored direct tasks to evaluate model ability when dealing with underspecification. Liu et al. (2023) introduced AmbiEnt, a natural language inference benchmark, which revealed that understanding ambiguous statements is still a challenge even to the state-of-the-art LLMs. Wildenburg et al. (2024) created the DUST task, which requires the language model to determine the relative levels of specifications between two sentences, finding that when interpreting underspecified sentences, LMs exhibit little uncertainty. Vijayvargiya et al. (2025) evaluated LLM agents for GitHub issue resolution in an underspecified setting, showing that follow-up interactions to supplement information help improve the resolve rate, but detecting the ambiguities in the instructions remains a challenge.

Prior work has classified different root causes for underspecification. First, task underspecification occurs when humans provide incomplete descriptions of the task at hand, which is prominent in "specification-heavy tasks" (Peng et al., 2023). Second, intent misalignment occurs when the AI fails to understand the user's intent or motivation, and is one of the common sources of user dissatisfaction (Kim et al., 2024b; Terry et al., 2023). Finally, Chaturvedi et al. (2024) discuss location and reference ambiguity, in embodied settings that involve physical spaces such as a Minecraft game.

B Precise Definition of Sharded Instructions

Section 3.1 introduces the concept of sharding at a high level. This Appendix offers a more precise definition by first defining mathematical terminology, and then defining properties that a sharded instruction must satisfy to be considered valid.

Let q refer to a single-turn complex query with intended (i.e., correct) output Y_q^*. We refer to the atomic content units (ACU) (Liu et al., 2022) of the query as

$$I(q) = [\mathcal{I}, (c_1, \cdots, c_m)]$$

where $\mathcal{I}$ is the primary intent of the query and $(c_1, \cdots, c_m)$ are the sufficient set of clarifications that specify details of how to compute Y_q^* conditioned on $\mathcal{I}$. For $I(q)$ to be considered *atomic*, any rephrasing of $I(q)$ should produce the same target output. Ie. for all q' s.t. $I(q') = I(q)$, then $Y_q'^* = Y_q^*$.

Given the above definition, the *aim* of the sharding process, for a given query q, is to identify the atomic content units $I(q)$ and construct a set of shorter instruction *shards* $\mathbf{s}$:

$$q' = [s_1, \cdots s_k] \text{ s.t. } I(q) = I(q')$$

where the shards s_j can be used to simulate multi-turn conversation, with the same intended output as q.

A sharded instruction q' is considered valid for an original query q if it fulfills the following properties:

关于欠明确性的相关研究

背景（第2节）回顾了最直接相关的先前工作，主要集中在多轮评估上。我们现在将介绍其他研究过欠明确性的相关先前工作。

关于沟通和语言学的先前研究已确定欠明确性是人类语言的一个常见特征（Lappin，2000；Ferreira，2008；Frisson，2009；Pezzelle，2023）。

理解大语言模型如何处理欠明确指令对于提升对话能力至关重要。为此，Herlihy等人（2024）识别了当对话式大语言模型系统面对欠明确查询时常见的响应模式，如模糊表达、拒绝、澄清和询问，并提出了从中恢复的机制。Malaviya等人（2024）强调了支持性上下文对于更准确、更具原则性地评估大语言模型对欠明确查询的响应的重要性，而Sarkar等人（2025）表明，一个能主动重写用户指令以应对欠明确性的系统可以改善大语言模型的响应。Shaikh等人（2025）研究了大语言模型在对话日志中进行基础信息获取（即，澄清和后续问题）的程度，并观察到它们在生成后续问题方面明显不足，人类这样做的可能性是其15倍。Chang等人（2025）聘请标注员通过聊天界面手动复现完整指定指令，发现用户仅在34%的情况下揭示了指令的全部细节，大多数时候仍会遗漏某些细节，使其处于欠明确状态。

已有若干研究探索了直接任务，以评估模型在处理欠明确性时的能力。Liu等人(2023) 引入了AmbiEnt，一个自然语言推理基准，它揭示出理解模糊陈述即使对最先进的大语言模型而言仍是一个挑战。Wildenburg等人(2024) 创建了DUST任务，该任务要求语言模型判断两个句子之间的明确性相对程度，发现当解释欠明确句子时，语言模型表现出极低的不确定性。Vijayvargiya等人(2025) 在欠明确设定下评估了大语言模型智能体解决GitHub问题的能力，结果表明通过后续交互补充信息有助于提高解决率，但检测指令中的模糊性仍然是一个挑战。

先前的研究已对欠明确性的不同根本原因进行了分类。首先，任务欠明确性发生在人类对当前任务提供不完整描述时，这在“规范密集型任务”中尤为突出（Peng等人,2023）。其次，意图错位发生在人工智能未能理解用户的意图或动机时，并且是导致用户不满的常见原因之一（Kim等人, 2024b; Terry等人, 2023）。最后，Chaturvedi等人(2024) 讨论了位置和指代模糊性，这出现在涉及物理空间（如Minecraft游戏）的具身设定中。

B 分片指令的精确定义

第3.1节在高层面上介绍了分片的概念。本附录通过首先定义数学术语，然后定义一个有效的分片指令必须满足的属性，来提供更精确的定义。

令 q 表示一个具有预期（即正确）输出 Y_q^*的单轮复杂查询。我们将该查询的原子内容单元（ACU）（Liu等人，2022）表示为

$$I(q) = [\mathcal{I}, (c_1, \cdots, c_m)]$$

其中 $\mathcal{I}$ 是查询的主要意图，而 $(c_1, \cdots, c_m)$ 是充分澄清集，用于指定如何计算以 $\mathcal{I}$ 为条件的 Y_q^* 的细节。要使 $I(q)$ 被视为原子的，对 $I(q)$ 的任何改写都应产生相同的目标输出。即，对于所有满足 $I(q') = I(q)$ 的 q'，则有 $Y_q'^* = Y_q^*$。

根据上述定义，对于给定查询 q，分片过程的目标是识别原子内容单元 $I(q)$，并构建一组更短的指令分片$\mathbf{s}$：

$$q' = [s_1, \cdots s_k] \text{ s.t. } I(q) = I(q')$$

其中分片 s_j 可用于模拟多轮对话，其预期输出与 q 相同。

一个分片指令 q' 对于原始查询 q 被认为是有效的，如果它满足以下属性：

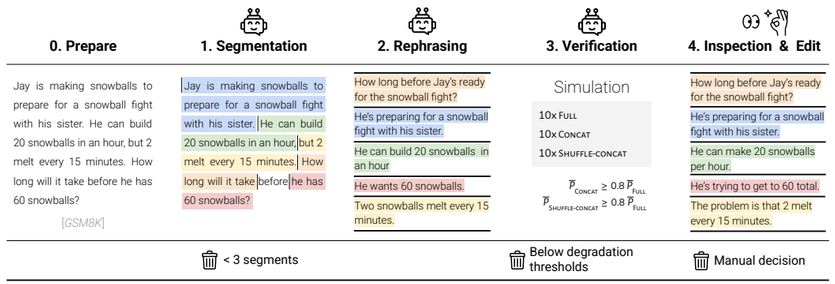


Figure 6: Process diagram of the four-step semi-automatic process to transform fully-specified instructions into a sharded instruction. The first three steps (segmentation, rephrasing, verification) are automated, while the fourth (inspect and edit) was manually completed by the authors of the work. The last row represents the rejection criteria for a sample.

P1: Information Preservation. $I(q) = I(q')$ No information from the original instruction necessary for the completion of the instruction should be lost during the sharding process.

P2: Clear Initial Intent. $\mathcal{I}_q = \mathcal{I}_{q'}$ and $s_1 = \mathcal{I}_q$. The first shard plays a distinctive role of being the *initial query* within the shard set. The initial query defines the high-level objective for the entire conversation. (e.g., "write a Python function").

P3: Order Insensitive. Apart from the first shard, the other shards should be decontextualized (Choi et al., 2021) and not refer to each other in a way that implies an order. As a result, the shard set presented in any order reveals equivalent information. Let $\rho(\mathbf{s}_{2..k})$ refer to a permutation of the shard ordering, then $I(q) = I(\tilde{q})\ \forall \tilde{q} = [s_1, \rho(\mathbf{s}_{2..k})]$

P4: Maximal Sharding. The sharding process should strive to maximize the number of shards extracted from the original instruction (maximize k). This can be achieved by producing shards that introduce a single, specific piece of information.

P5: Minimal Transformation. The sharded instruction should maintain the instruction language and avoid simplifying, altering, or interpreting elements of the original instruction as much as possible. Apart from modifications to satisfy properties P1-P4, the sharding process should attempt to limit modifications such that the shards ($[s_1, \cdots s_k]$ are semantically similar to the atomic content units $I(q)$.

C Semi-Automatic Sharding Process

We rely on a semi-automatic process to transform fully-specified instructions into their sharded equivalents. The process – illustrated in Figure 6 – consists of a sequence of three automated steps (Segmentation, Rephrasing, Verification) followed by a manual step that was conducted by an author of the paper.

We now detail each step of the process, then go over task-specific details we implemented as needed. We note that as part of our open-source release, we provide all the prompts used in the first three LLM-based steps.

Step 1: Segmentation Given an original fully-specified instruction (left-most column in Figure 6), the LLM is prompted to extract *segments* of the instructions. Segments are intended to correspond to the atomic content units (defined in Appendix B). In particular, the prompt indicates that segments must not overlap, and that not all words in the original instruction must belong to a segment. Prompts are task-specific and incorporate at least three few-shot examples of segmentation, to allow for the concept of segmentation to be illustrated through examples. At this stage, any instruction that yields fewer than three segments are filtered out and does not proceed to the next stage.

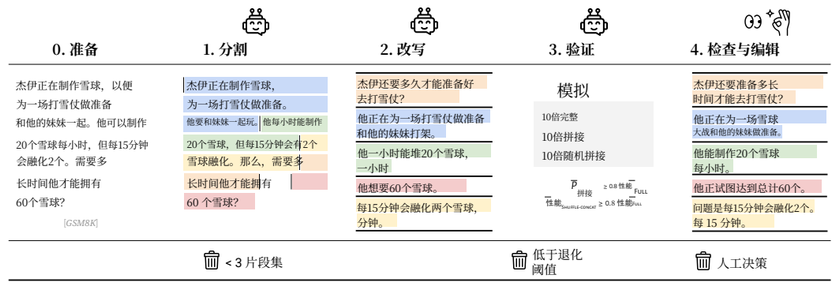


图6：将完整指定指令转化为分片指令的四步半自动流程示意图。前三步（分割、改写、验证）是自动化的，而第四步（检查与编辑）由本工作的作者手动完成。最后一行代表了一个样本的拒绝标准。

P1：信息保留。$I(q) = I(q')$ 在分片过程中，不应丢失原始指令中完成该指令所必需的任何信息。

P2：清晰的初始意图。 $\mathcal{I}_q = \mathcal{I}_{q'}$ 和 $s_1 = \mathcal{I}_q$。第一个分片扮演着独特的角色，即作为分片集中的初始查询。初始查询定义了整个对话的高层目标。（例如，“编写一个Python函数”）。

P3：顺序无关。除了第一个分片，其他分片应是去语境化的（Choi 等人，2021），并且不应以暗示顺序的方式相互引用。因此，分片集以任何顺序呈现都能揭示等效的信息。令 $\rho(\mathbf{s}_{2..k})$ 表示分片顺序的一种排列，则 $I(q) = I(\tilde{q})\ \forall \tilde{q} = [s_1, \rho(\mathbf{s}_{2..k})]$

P4：最大化分片。分片过程应努力从原始指令中提取出最大数量的分片（最大化 k）。这可以通过生成仅引入单一、特定信息片段的<code>分片</code>来实现。

P5：最小化转换。分片后的指令应保持指令的语言风格，并尽可能避免简化、改变或解释原始指令的元素。除了为满足属性 P1-P4 所做的修改外，分片过程应尝试限制修改，使得分片（$[s_1, \cdots s_k]$）在语义上类似于原子内容单元 $I(q)$。

C 半自动分片过程

我们依靠一个半自动流程，将完整指定指令转换为其分片等效形式。该过程——如图6 所示——由三个自动化步骤（分割、改写、验证）序列和一个由论文作者执行的手动步骤组成。

我们现在详细说明该过程的每个步骤，然后根据需要介绍我们实现的任务特定细节。我们指出，作为我们开源发布的一部分，我们提供了在前三个基于大语言模型的步骤中使用的所有提示词。

步骤1：分割给定一个原始的完全指定指令（图6中最左侧列），提示大语言模型提取指令的片段集。片段旨在对应原子内容单元（定义见附录B）。具体而言，提示词指明片段不得重叠，且并非原始指令中的所有单词都必须属于某个片段。提示词是任务特定的，并包含至少三个分割的少样本示例，以便通过示例说明分割的概念。在此阶段，任何产生少于三个片段的指令都会被过滤掉，不会进入下一阶段。

Step 2: Rephrasing Given the original fully-specified instruction and the extracted segments, this stage consists in rewriting each segment to be decontextualized (Choi et al., 2021) and conversational. In other words, dependencies between segments are resolved, and the ordering is changed such that obtained *shards* adhere to properties P2 and P5. In the example above, the fourth segment (highlighted in orange) becomes the first shard as it reveals the overall intent, and light rephrasing occurs in other shards. The rephrasing prompt is task-specific and includes few-shot examples of rephrasing segmented instructions.

Step 3: Verification Steps 1-2 produce a sharded instruction that can be used to simulate SHARDED and CONCAT conversations. To verify the property P1 (Information Preservation) that no information has been lost during segmentation and rephrasing, we conduct preliminary simulations to evaluate the original and sharded instruction side-by-side. Specifically for each pair of the original and the sharded instruction, we simulate ten FULL conversations with the original instruction, ten CONCAT conversations with the sharded instruction (by concatenating the shards), and ten SHUFFLE-CONCAT conversations. SHUFFLE-CONCAT is a variant of the CONCAT simulation in which all shards (except Shard 1) are randomly permuted before being concatenated. This variant can be seen as a more adversarial version of CONCAT, verifying the property P3 (Order Insensitive). For each simulation type, we calculate the averaged performance $\overline{P}$ over ten runs and filter out instructions that are below an acceptable degradation threshold. Specifically, instructions are acceptable if the following conditions are met:

$$\overline{P}_{\text{CONCAT}} \geq 0.8\, \overline{P}_{\text{FULL}}$$
$$\overline{P}_{\text{SHUFFLE-CONCAT}} \geq 0.8\, \overline{P}_{\text{FULL}},$$

where $\overline{P}_{\text{X}}$ denotes the averaged performance of the simulation type X. If more degradation is observed (*i.e.*, below 80%), it indicates a potential loss of information during sharding, or that decontextualization was not implemented accurately.

Step 4: Inspect and Edit Even though the first three steps define the sharding process and implement some level of quality assurance, they do not guarantee the level of quality required for precise and large-scale experiments due to relying on LLM outputs. To obtain high-quality shards, we reserve step 4 for manual inspection and validation. To facilitate the procedure, we developed a web-based annotation interface. In the interface, an annotator can review a pair of fully-specified and sharded instructions, edit, add, or remove individual shards, and decide to accept or reject sharded instructions. Sharded instructions included in our experiments were all manually reviewed by two authors of the work. The amount of editing and filtering required in this final stage varied by task.

Inspecting and editing an auto-generated instruction typically requires 1-3 minutes per instruction, an order of magnitude less than it would require for authors to write the sharded instructions de-novo from a given fully-specified instruction. As part of our open-source release, we provide all the prompts used during sharding, which we hope can facilitate the sharding of additional tasks.

D INSPECTION OF SIMULATED SHARDED CONVERSATION

The sharding simulation environment (described in Section 3) relies on LLM components to simulate the user, classify assistant responses, and extract answers from free-text responses. LLM-based components are likely to fail, and we performed an inspection of 200 simulated SHARDED conversations to understand the level of simulation error and the potential effect on estimating the performance of the assistant LLMs due to the error.

For each inspected conversation, we annotated user turns, assistant turns, and the overall conversation with five specific elements.

For user utterances, we annotated whether the utterance revealed exactly the information from one shard in the sharded instruction (`Shard Fully Revealed`). Specifically, we flagged turns that revealed more than one shard, and turns that revealed a shard only partially. We also annotated each user's turn for whether it is appropriately contextualized in the conversation (`Shard Contextualized`). For example, if the previous assistant's turn asked a binary clarification question (yes/no), then proper contextualization would require a Yes/No response to directly address the assistant's response.

步骤2：改写给定原始的完全指定指令和提取的片段集，此阶段包括重写每个片段，使其去语境化（Choi等人，2021年）并具有对话性。换言之，片段间的依赖关系被解决，顺序被改变，使得获得的分片遵循属性P2和P5。在上面的示例中，第四个片段（以橙色高亮显示）成为第一个分片，因为它揭示了总体意图，其他分片则进行了轻微的改写。改写提示词是任务特定的，并包含对分割指令进行改写的少样本示例。

步骤3：验证步骤1-2产生一个可用于模拟分片对话和拼接对话的分片指令。为验证属性P1（信息保留），即确认在分割和改写过程中没有信息丢失，我们进行初步模拟，以并排评估原始指令和分片指令。具体来说，针对每一对原始指令和分片指令，我们模拟十次使用原始指令的完整对话、十次使用分片指令的拼接对话（通过拼接所有分片），以及十次随机拼接对话。随机拼接对话是拼接模拟的一个变体，其中所有分片（除分片1外）在拼接前会被随机排列。此变体可被视为一种更具对抗性的拼接模拟，用于验证属性P3（顺序无关）。对于每种模拟类型，我们计算十次运行的平均性能 P，并过滤掉低于可接受退化阈值的指令。具体而言，指令需满足以下条件才被视为可接受：

$$\overline{P}_{\text{CONCAT}} \geq 0.8\, \overline{P}_{\text{FULL}}$$
$$\overline{P}_{\text{SHUFFLE-CONCAT}} \geq 0.8\, \overline{P}_{\text{FULL}},$$

其中 $\overline{P}_{\text{X}}$ 表示模拟类型 X 的平均性能。如果观察到更多性能下降（即，低于 80%），则表明在分片过程中可能存在信息损失，或者去语境化未能准确实施。

步骤 4：检查与编辑尽管前三个步骤定义了分片过程并实施了一定程度的质量保证，但由于依赖大语言模型输出，它们并不能保证达到精确且大规模实验所需的质量水平。为了获得高质量的分片，我们将步骤 4 保留用于人工检查与验证。为了便于操作，我们开发了一个基于网络的标注界面。在该界面中，标注者可以审阅一对完全指定指令和分片指令，编辑、添加或删除单个分片，并决定接受或拒绝分片指令。我们实验中包含的分片指令均由本工作的两位作者进行了人工审阅。此最终阶段所需的编辑和筛选量因任务而异。

检查与编辑一条自动生成的指令通常每条指令需要 1-3 分钟，这比作者根据给定的完全指定指令从头开始编写分片指令所需的时间少一个数量级。作为我们开源发布的一部分，我们提供了分片过程中使用的所有提示词，希望能促进更多任务的分片工作。

D 模拟SHARDED对话检查

分片模拟环境（在第3节中描述）依赖于大语言模型组件来模拟用户、对助手响应进行分类以及从自由文本响应中提取答案。基于大语言模型的组件很可能会失败，我们检查了200个模拟SHARDED对话，以了解模拟误差的水平以及该误差对评估助手大语言模型性能的潜在影响。

对于每个被检查的对话，我们用五个特定元素标注了用户轮次、助手轮次以及总体对话。

对于用户话语，我们标注了该话语是否恰好揭示了分片指令(`Shard Fully Revealed`)中的一个分片信息。具体来说，我们标记了那些揭示了超过一个分片的轮次，以及仅部分揭示了一个分片的轮次。我们还标注了每个用户轮次是否在对话(`Shard Contextualized`)中进行了恰当的语境化。例如，如果前一个助手轮次提出了一个二元澄清问题（是/否），那么恰当的语境化将要求一个直接回应助手响应的‘是’或‘否’的答案。

Inspection	All Tasks	Actions	Code	Math	Db
Shard Fully Revealed	96.0	98.3	94.9	93.4	100.0
Shard Contextualized	98.4	98.3	98.3	98.3	98.6
Strategy Accuracy	95.2	94.7	95.5	95.6	94.7
Extraction Success	97.0	100.0	93.4	98.4	100.0
Overall Success	97.8	100.0	96.0	96.0	100.0

Table 2: Results of the manual inspection of 100 simulated SHARDED conversations across four tasks: Actions, Code, Math, and Database. The first column aggregates annotation results on the four tasks.

For assistant utterances, we annotated whether the classified strategy was accurate (`Strategy Accuracy`). For example, if the response is labeled as a clarification, we confirm if it poses a clarification question to the user. When assistant utterances were labeled as answer attempts, we further labeled whether the answer extraction step was successful (`Extraction Success`).

Upon completing the inspection of each user and assistance utterance, we assigned a global label to the entire conversation on whether or not the errors that occurred during simulation (if any) affected the overall validity of the simulation. If not, the simulation was marked as successful (`Overall Success`).

We inspected conversations for four tasks: Actions, Code, Math, and Database. The other two (Summary and Data-to-text) are refining tasks that require an answer attempt at each turn, and do not rely on an LLM-based user simulator. As such, they have limited scope for simulation error.

Table 2 summarizes the results of the inspection annotation. Overall, the simulation environment is highly reliable, with roughly 98% of inspected conversations labeled as successful. Some errors occur in each component. With user simulation, a single shard is fully revealed around 96% of the time, and properly contextualized 98% of the time. The processing of assistant responses also leads to errors: the turn strategy classification is only 95% accurate, and extraction of answer attempts has an accuracy of 97%.

Utterance-level errors did not always affect the validity of the overall simulation. In some cases, we observed that the user simulator would correct an error in an early turn, subsequently in the conversation, or that an error in answer extraction on the wrong answer attempt would occur at a turn, but the extraction would be successful later on. In summary, we empirically find that the simulation environment is largely accurate: though some errors occur, large drops of performance in the SHARDED setting (beyond 2%) are not due to errors caused by the simulator. We also observed that the user simulator occasionally produced corrective-style utterances (e.g., "No, I want to sort in XYZ way"), though this behavior was not explicitly encouraged in the simulator prompt and was relatively rare.

E CONCRETE EXAMPLE OF LOSS IN APTITUDE VS. RELIABILITY

Let's imagine we are provided with ten instructions ($N = 10$), each FULL and SHARDED. We run simulations with an LLM, simulating 10 conversations per instruction and setting ($M = 10$). Let's assume the LLM achieves an averaged performance ($\overline{P}$) of 90% in the FULL, and 60% in the SHARDED setting.

Finally, let's assume that the FULL performance is achieved by having perfect performance (*i.e.*, success in 10/10 randomly sampled runs) on 9 instructions, and failing on all the sampled simulations of the last, tenth instruction. In other words:

$$S_{ij}^{\text{FULL}} = \begin{cases} 100, & \text{if } i \in \{1, \ldots, 9\} \\ 0, & \text{if } i = 10 \end{cases},$$

where S_{ij}^{FULL} represents the score for i-th instruction at j-th simulation run. The aptitude (A) and unreliability (U) of the LLM for the FULL setting is $A = 90\%$ and $U = 0\%$ (*i.e.*, for each instruction, the 10th and 90th percentile scores are equal).

检查	所有任务	行动	Code	Math	Db
分片完全揭示	96.0	98.3	94.9	93.4	100.0
分片情境化	98.4	98.3	98.3	98.3	98.6
策略准确率	95.2	94.7	95.5	95.6	94.7
提取成功率	97.0	100.0	93.4	98.4	100.0
总体成功率	97.8	100.0	96.0	96.0	100.0

表2：对100个模拟SHARDED对话进行人工检查的结果，涵盖四项任务：行动、代码、数学和数据库。第一列汇总了四项任务的标注结果。

对于助手话语，我们标注了所分类的策略是否准确(`StrategyAccuracy`)。例如，如果响应被标记为澄清，我们会确认它是否向用户提出了一个澄清问题。当助手话语被标记为答案尝试时，我们进一步标注了答案提取步骤是否成功(`Extraction Success`)。

在完成对每个用户和助手话语的检查后，我们为整个对话分配了一个全局标签，以判断模拟过程中发生的错误（如果有）是否影响了模拟的整体有效性。如果没有影响，则该模拟被标记为成功（`OverallSuccess`）。

我们检查了四项任务的对话：行动、代码、数学和数据库。另外两项（摘要和数据到文本）是精炼任务，需要在每一轮次进行答案尝试，且不依赖于基于LLM的用户模拟器。因此，它们出现模拟误差的范围有限。

表2 总结了检查标注的结果。总体而言，该模拟环境高度可靠，大约98%的被检查对话被标记为成功。每个组件都存在一些错误。在用户模拟中，单个分片大约96%的时间被完全揭示，并且98%的时间被正确语境化。助手响应的处理也会导致错误：轮次策略分类的准确率仅为95%，答案尝试提取的准确率为97%。

话语层面的错误并不总是影响整体模拟的有效性。在某些情况下，我们观察到用户模拟器会在早期轮次、或随后的对话中纠正一个错误，或者在某个轮次对错误答案尝试的答案提取会出现错误，但稍后的提取会成功。总之，我们通过实证发现模拟环境大体上是准确的：虽然存在一些错误，但分片设置中性能的大幅下降（超过2%）并非由模拟器引起的错误所致。我们还观察到，用户模拟器偶尔会产生纠正式的话语（例如，“不，我想按XYZ方式排序”），尽管这种行为在模拟器提示词中并未明确鼓励，且相对罕见。

能力与可靠性损失的具体示例

假设我们获得了十条指令（$N = 10$），每条指令都是完整且分片的。我们使用一个大语言模型进行模拟，为每条指令和设置（$M = 10$）模拟10次对话。假设该大语言模型在完整设置下的平均性能（P）为90%，在分片设置下为60%。

最后，假设完整设置下的性能是通过在9条指令上获得完美性能（即，在10/10次随机抽样运行中均成功），而在最后一条（第十条）指令的所有抽样模拟中均失败来实现的。换句话说：

$$S_{ij}^{\text{FULL}} = \begin{cases} 100, & \text{if } i \in \{1, \ldots, 9\} \\ 0, & \text{if } i = 10 \end{cases},$$

其中 S_{ij}^{FULL} 表示第 i 条指令在第 j 次模拟运行中的分数。该大语言模型在完整设置下的能力 (A) 和不可靠性 (U) 分别为 $A = 90\%$ 和 $U = 0\%$（即，对于每条指令，其第10百分位数和第90百分位数的分数相等）。

Let's now consider three conditions for the SHARDED setting that all achieve an averaged performance of $\overline{P} = 60\%$. We illustrate the conditions in Figure 7.

Situation 1: Drop in Aptitude. The LLM achieves perfect performance on six of the ten instructions:

$$S_{ij}^{\text{SHARDED}} = \begin{cases} 100, & \text{if } i \in \{1, \dots, 6\} \\ 0, & \text{if } i \in \{7, \dots, 10\} \end{cases}.$$

In situation 1, $\overline{P} = 60\%$, $A = 60\%$, and $U = 0\%$. The degradation in performance is entirely explained by a decrease in aptitude, while the reliability remains the same.

Situation 2: Drop in Reliability. The LLM achieves mixed performance (6-7 perfect scores per instruction) on nine of the 10 instructions:

$$S_{ij}^{\text{SHARDED}} = \begin{cases} 100, & \text{if } 1 \le i \le 3, 1 \le j \le 6 \\ 100, & \text{if } 4 \le i \le 9, 1 \le j \le 7 \\ 0, & \text{otherwise} \end{cases}.$$

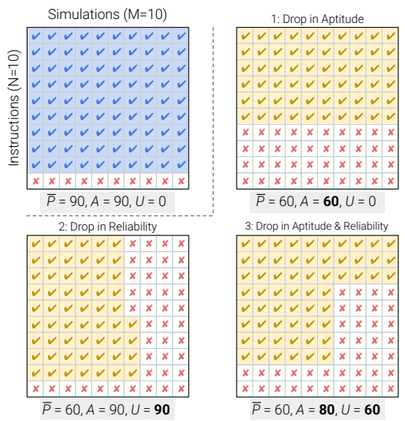


Figure 7: Illustrations for different situations. Green and red fills in each grid indicate sample-level score (*e.g.,* pass / exact match). Compared to FULL (top left), three situations in SHARDED achieve the same $\overline{P} = 60$ while varying in aptitude A and unreliability U.

In situation 2, $\overline{P} = 60\%$, with an aptitude of $A = 90\%$, and a unreliability of $U = 90\%$. The degradation in performance is entirely explained by a large drop in reliability, while sharded and fully specified aptitude are equal.

Situations 1 and 2 illustrate extreme scenarios where the average drop in performance is entirely explained by a drop in aptitude or reliability, but in practice, a combination is more likely to occur, as in situation 3.

Situation 3: Combined drop in Aptitude and Reliability. The LLM achieves perfect performance on three instructions, and mixed performance (6 perfect scores per instruction) on five of the 10 instructions:

$$S_{ij}^{\text{SHARDED}} = \begin{cases} 100, & \text{if } 1 \le i \le 3 \\ 100, & \text{if } 4 \le i \le 8, 1 \le j \le 6 \\ 0, & \text{otherwise} \end{cases}.$$

In situation 3, $\overline{P} = 60\%$, with an aptitude of $A = 80\%$, and a unreliability of $U = 60\%$. Note that situation 3 leads to a larger increase in unreliability (from 0% to 60%) than a decrease in aptitude (from 90% to 80%) when compared to fully-specific simulations. This corresponds in practice to our observation: drops in performance are explained by small drops in aptitude and large drops in reliability.

Finally, we note that though this concrete example we provide uses binary scores (0 and 100) for simulated conversation outcomes, aptitude (A) and unreliability (U) can equally be applied to continuous metrics (such as BLEU).

F QUALITATIVE ANALYSES OF SIMULATION LOGS

In the following subsections, we report qualitative analyses on the corpus of simulations from the main experiment (Section 5.1). The purpose of the analyses is to discern root causes in model

现在，让我们考虑共享设置的三种条件，它们均能达到 $P = 60\%$ 的平均性能。我们在图 7 中展示了这些条件。

情况一：能力下降。该大语言模型在十条指令中的六条上取得了完美的性能：

$$S_{ij}^{\text{SHARDED}} = \begin{cases} 100, & \text{if } i \in \{1, \dots, 6\} \\ 0, & \text{if } i \in \{7, \dots, 10\} \end{cases}.$$

在情况一中，$\overline{P} = 60\%$、$A = 60\%$ 和 $U = 0\%$。性能的下降完全由能力降低所解释，而可靠性保持不变。

情况二：可靠性下降。该大语言模型在10条指令中的9条上取得了混合性能（每条指令获得6-7个完美分数）：

$$S_{ij}^{\text{SHARDED}} = \begin{cases} 100, & \text{if } 1 \le i \le 3, 1 \le j \le 6 \\ 100, & \text{if } 4 \le i \le 9, 1 \le j \le 7 \\ 0, & \text{otherwise} \end{cases}.$$

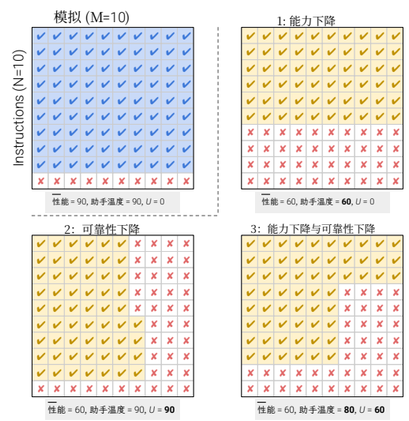


图7：不同情况的示意图。每个网格中的绿色和红色填充表示样本级分数（例如，通过/精确匹配）。与完整（左上）相比，分片中的三种情况在能力A和不可靠性U方面各不相同，但实现了相同的 $P = 60$。

在情况2中，$\overline{P} = 60\%$，能力为$A = 90\%$，不可靠性为 $U = 90\%$。性能的下降完全由可靠性的大幅下降所解释，而分片与完整指定的能力是相等的。

情况1和2说明了两种极端场景，即平均性能下降完全由能力下降或可靠性下降所解释，但在实践中，更可能发生两者结合的情况，如情况3所示。

情况3：能力与可靠性共同下降。大语言模型在三条指令上实现了完美性能，并在10条指令中的五条上实现了混合性能（每条指令6个完美分数）：

$$S_{ij}^{\text{SHARDED}} = \begin{cases} 100, & \text{if } 1 \le i \le 3 \\ 100, & \text{if } 4 \le i \le 8, 1 \le j \le 6 \\ 0, & \text{otherwise} \end{cases}.$$

在情境3中，$\overline{P} = 60\%$，能力为$A = 80\%$，不可靠性为 $U = 60\%$。请注意，与完全特定的模拟相比，情境3导致不可靠性的增加（从0%到60%）大于能力的下降（从90%到80%）。

这实际上对应了我们的观察：性能下降是由能力的小幅下降和可靠性的大幅下降所解释的。

最后，我们注意到，尽管我们提供的这个具体示例对模拟对话结果使用了二元分数（0和100），但能力（A）和不可靠性（U）同样可以应用于连续指标（例如BLEU）。

F 模拟日志的定性分析

在以下小节中，我们将对主实验（第5.1节）的模拟语料库进行定性分析。分析的目的是辨别模型

	Conversation Progress At First Answer Attempt				
Model	0-20%	20-40%	40-60%	60-80%	80-100%
First answer attempt is ...	earliest	early	midway	late	latest
3.1-8B	16.1	24.0	35.3	39.6	39.7
OLMo2	17.6	32.7	37.7	47.3	26.4
3-Haiku	27.1	35.6	47.4	58.9	70.3
4o-mini	30.2	39.2	48.4	58.2	59.9
3.3-70B	33.3	40.1	51.2	60.0	69.3
Phi-4	25.7	33.1	47.0	53.0	57.9
CMD-A	38.0	42.9	56.5	65.5	73.5
4-Scout	39.8	36.8	51.0	57.9	64.8
o3	21.0	37.9	51.9	58.4	68.0
3.7-Sonnet	29.2	35.6	55.3	68.0	71.6
R1	39.5	43.1	53.5	66.4	50.2
4o	36.0	41.4	56.2	65.6	90.4
2.5-Flash	39.0	48.6	60.2	70.8	74.6
4.1	33.9	52.7	60.6	69.0	78.6
2.5-Pro	41.1	45.7	53.5	64.6	63.8
Average	30.9	40.5	51.7	60.4	64.4

Table 3: Averaged performance ($\overline{P}$) breakdown, based on how early in the conversation the LLM makes its first answer attempt. Analysis conducted on simulations of two tasks: Code and Math.

behavior that lead to performance degradation. We identify four behaviors below and detail the analysis for each in the rest of the section:

1. LLMs attempt to answer the entire problem prematurely.
2. LLMs overly rely on previous (incorrect) answer attempts, leading to lengthier "bloated" answers.
3. LLMs overly adjust their answers based on the last conversation turn, materialized by a pronounced forgetting of middle-turns.
4. LLMs produce overly verbose answers, which likely introduce problem assumptions that detract attention from user utterances.

F.1 Premature Answer Attempts

During SHARDED simulation, responses are classified according to a seven-class conversation strategy categorization. In particular, each assistant response is tagged as being a formal *answer attempt* or not (as answer attempts require further processing: extraction and evaluation by the task-specific evaluator).

On the onset of conversation, LLMs have the least amount of information (highest level of underspecification) and are least likely to formulate correct answer attempts. Proposing a solution early might therefore plant certain incorrect elements in it, which wrongly influence the interaction later in the conversation.

To evaluate this hypothesis, we bin all simulated conversations from our experiments based on how early in the conversation the first answer attempt is generated by the LLM. Specifically, we create five bins: `0-20%` if the first answer attempt occurs within the first 20% turns of the conversation, and `20-40%`, `40-60%`, `60-80%`, and `80-100%` if it occurs in later turns of the conversation. Of the six tasks included in our experiments, only two (Math and Code) observed a significant range in LLM behavior for answer attempt timing. For the other four tasks, models attempt an answer from the first turn most of the time, rendering analysis on this parameter impossible.

Analysis results for the two remaining tasks are presented in Table 3. We observe that for every single model, conversations with a later first answer attempt lead to higher averaged performance. Across all models, conversations with the first attempt being made in the first 20% of conversations achieve

	首次回答尝试时的对话进度				
模型	0-20%	20-40%	40-60%	60-80%	80-100%
首次答案尝试于-尝试是...	最早	早期	中途	late	最新
3.1-8B	16.1	24.0	35.3	39.6	39.7
OLMo2	17.6	32.7	37.7	47.3	26.4
3-Haiku	27.1	35.6	47.4	58.9	70.3
4o-mini	30.2	39.2	48.4	58.2	59.9
3.3-70B	33.3	40.1	51.2	60.0	69.3
Phi-4	25.7	33.1	47.0	53.0	57.9
CMD-A	38.0	42.9	56.5	65.5	73.5
4-Scout	39.8	36.8	51.0	57.9	64.8
o3	21.0	37.9	51.9	58.4	68.0
3.7-Sonnet	29.2	35.6	55.3	68.0	71.6
R1	39.5	43.1	53.5	66.4	50.2
4o	36.0	41.4	56.2	65.6	90.4
2.5-Flash	39.0	48.6	60.2	70.8	74.6
4.1	33.9	52.7	60.6	69.0	78.6
2.5-Pro	41.1	45.7	53.5	64.6	63.8
平均值	30.9	40.5	51.7	60.4	64.4

表3：基于大语言模型在对话中多早进行其首次回答尝试的平均性能（$\overline{P}$）细分。分析基于两项任务的模拟进行：代码和数学。

beh行为中导致性能下降的根本原因。我们在下文识别了四种行为并详细阐述 e
anal本节其余部分对每一项的分析如下：

1. 大语言模型过早地尝试回答整个问题。2. 大语言模型过度依赖先前（错误）的答案尝试，导致产生更冗长的“臃肿”答案。3. 大语言模型过度根据最后一轮对话调整其答案，具体表现为对中间轮次的明显遗忘。4. 大语言模型产生过于冗长的回答，这可能引入问题假设，从而分散对用户话语的注意力。

F.1 过早答案尝试

在分片模拟期间，响应根据一个七类对话策略分类进行分类。具体而言，每个助手响应被标记为正式的答案尝试或非答案尝试（因为答案尝试需要进一步处理：由任务特定评估器进行提取和评估）。

在对话开始时，大语言模型拥有的信息量最少（未明确程度最高），因此最不可能形成正确的答案尝试。过早提出解决方案可能会在其中植入某些错误元素，从而在后续的对话交互中产生错误影响。

为了评估这一假设，我们根据大语言模型在对话中生成首次回答尝试的早晚，将实验中的所有模拟对话进行分箱。具体而言，我们创建了五个分箱： `0-20%` 如果首次回答尝试发生在对话的前20%轮次内，以及`20-40%`、 `40-60%`、 `60-80%`和 `80-100%` 如果它发生在对话的后续轮次中。在我们实验包含的六项任务中，只有两项（数学和代码）在大语言模型的答案尝试时机上观察到了显著的行为差异。对于其他四项任务，模型大多数时候从第一轮就开始尝试给出答案，使得针对此参数的分析无法进行。

剩余两项任务的分析结果见表3。我们观察到，对于每一个模型，首次回答尝试发生较晚的对话，其平均性能更高。在所有模型中，首次尝试发生在前20%对话中的对话，其得分为

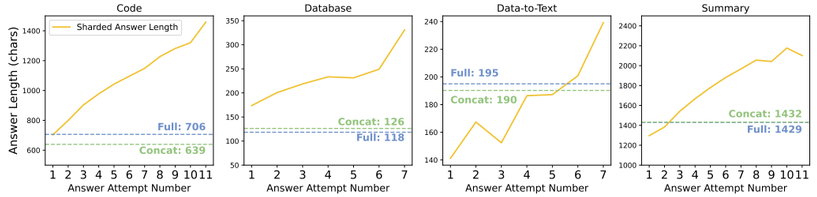


Figure 8: Average length (in number of characters) of answer attempts across four tasks (Code, Database, Data-to-text, and Summary) in SHARDED conversations. Answer attempts in the FULL and CONCAT settings tend to be shorter on average than those from SHARDED setting. SHARDED answer attempts increase in length as the LLMs make more answer attempts.

a score of 30.9, less than half of the 64.4 when the LLM waits for the last 20% of the conversation to make an answer attempt.

In other words, we find that premature answer attempts detract LLM performance. Conversations where the model clarifies user instructions or discusses the problem at a high level before moving to generating complete answer attempts lead to higher performance. We hypothesize that this is due to the model making incorrect assumptions in premature solutions, which conflict with subsequent user instructions in later turns.

F.2 ANSWER BLOAT IN MULTI-TURN CONVERSATION

In multi-turn conversation simulations, the LLM might make multiple answer attempts, with each subsequent attempt being potentially based on previous attempts. In contrast, single-turn conversations constrain conversation dynamics, with the LLM making a single, first-and-final answer attempt.

To understand multi-turn conversation dynamics, we calculate the average length of answer attempts in each simulation type. For the SHARDED setting, we calculate average length for each attempt within a simulation (*i.e.*, average length of the first attempt, second attempt, third attempt, etc.). We note for readers here that the analysis is conducted on extracted answer attempts (output of the Answer Extractor module in Figure 2) rather than the entire assistant responses. The extracted answer more accurately measures dynamics in answer attempts (i.e., generated SQL query, or Python function) rather than the entire responses, which might contain varying amounts of unrelated content.

Results of the analysis are plotted in Figure 8. Across the four tasks, we find that answer lengths in the FULL and CONCAT settings tend to be similar, typically within 2-10% of each other. On three of the analyzed tasks (Code, Database, Summary), the first answer attempt in the SHARDED setting has a similar length to FULL and CONCAT counterparts, yet for each subsequent answer attempt, we observe an increase in average answer length. The effect is such that the final answer attempts in SHARDED conversations (right portion of the four plots) tend to be 20-300% longer than the solutions generated in the FULL and CONCAT settings. We name this observation the *answer bloat effect*: as a multi-turn conversation progresses, the LLM generates incorrect answer attempts, making assumptions about portions of the instruction that remain unspecified. As the user reveals additional information in succeeding turns, the LLM does not successfully invalidate its prior assumptions and overly relies on its previous attempts. Answer bloat in multi-turn, underspecified conversation leads to longer solutions compared to single-turn equivalents.

We perform an additional analysis, focusing only on the Code and Database tasks and filtering to simulations where the LLM reaches an entirely correct solution (score of 100.0). For Code task, correct programs obtained from SHARDED setting are on average 850 characters long, which is 27% more characters than the correct solutions generated in the FULL setting (668 characters on average). For Database, correct SQL queries in the SHARDED setting are on average 129 characters, 14% more characters than those from the FULL setting (113 characters). In summary, LLMs are less likely to reach a correct solution in multi-turn settings (lower $\overline{P}$), and when they do, the final solutions they reach are longer (bloated), hinting that the solutions are qualitatively worse.

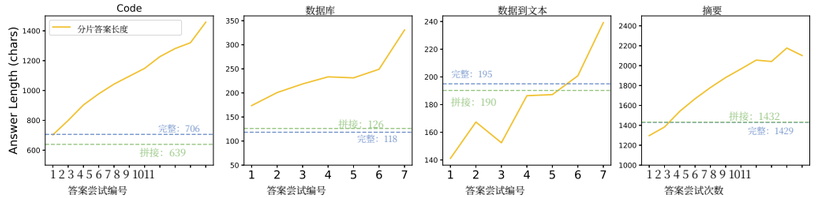


图8：在SHARDED对话中，四个任务（代码、数据库、数据到文本和摘要）的答案尝试平均长度（以字符数计）。完整和拼接设置中的答案尝试平均长度往往比分片设置中的更短。随着大语言模型进行更多答案尝试，分片设置中的答案尝试长度会增加。

30.9，不到大语言模型等到对话最后20%才进行答案尝试时得分64.4的一半。

换句话说，我们发现过早的答案尝试会损害大语言模型的性能。在模型开始生成完整的答案尝试之前，先澄清用户指令或从高层次讨论问题的对话，会带来更高的性能。我们假设这是因为模型在过早的解决方案中做出了错误的假设，这些假设与后续轮次中的用户指令相冲突。

F.2 多轮对话中的答案膨胀

在多轮对话模拟中，大语言模型可能会进行多次答案尝试，每次后续尝试都可能基于之前的尝试。相比之下，单轮对话限制了对话的动态性，大语言模型只进行一次首次即最终的答案尝试。

为了理解多轮对话的动态，我们计算了每种模拟类型中答案尝试的平均长度。对于分片设置，我们计算模拟中每次尝试的平均长度（即，第一次尝试、第二次尝试、第三次尝试等的平均长度）。我们在此提醒读者，分析是基于提取的答案尝试（图2中答案提取模块的输出）进行的，而非整个助手响应。提取的答案能更准确地衡量答案尝试中的动态（例如，生成的SQL查询或Python函数），而不是可能包含不同数量无关内容的整个响应。

分析结果绘制于图8。在所有四项任务中，我们发现完整和拼接设置下的答案长度往往相似，通常彼此相差在2-10%以内。在分析的三项任务（代码、数据库、摘要）中，分片设置下的首次回答尝试与完整和拼接设置下的对应尝试长度相似，但对于后续的每次答案尝试，我们观察到平均答案长度有所增加。其效果是，分片对话中的最终答案尝试（四个图的右侧部分）往往比完整和拼接设置下生成的解决方案长20-300%。我们将此观察命名为答案膨胀效应：随着多轮对话的进行，大语言模型会生成错误的答案尝试，对指令中未充分指定的部分做出假设。随着用户在后续轮次中揭示额外信息，大语言模型未能成功推翻其先前的假设，并过度依赖其之前的尝试。与单轮等效对话相比，在多轮、未充分指定的对话中，答案膨胀会导致解决方案更长。

我们进行了一项额外分析，仅关注代码任务和数据库任务，并筛选出大语言模型达到完全正确解决方案（100.0分）的模拟。对于代码任务，从分片设置中获得的正确程序平均长度为850个字符，这比完整设置中生成的正确解决方案（平均668个字符）多出27%的字符。对于数据库任务，分片设置中的正确SQL查询平均长度为129个字符，比完整设置中的查询（113个字符）多出14%的字符。总之，大语言模型在多轮对话场景中达到正确解决方案的可能性较低（较低的 P），而当它们确实达到时，其最终解决方案更长（臃肿），这暗示解决方案在质量上更差。

F.3 Over-adjust based on Last Turn of Conversation

Because the summary task requires the assistant to attribute its summary back to documents through citation, the task offers a unique opportunity to analyze what turns of information LLMs pay attention to as the multi-turn conversation progresses. As a reminder, the summary task involves a user introducing new documents at each turn. Therefore, our analysis aims to understand whether document introduction order (across turns) affects the likelihood of the LLM citing a document.

In Figure 9, we plot the results of our analysis. Each row corresponds to the analysis of summaries generated at a given turn in the sharded simulation. At turn 1 (top row), 96% of the cited documents were introduced in the first turn. The missing 4% corresponds to hallucinated citations to documents that were not introduced, and explains why none of the rows' distributions sum to 100%. At turn two (second row from the top), summaries include citation in roughly equal proportion for turn-1 and turn-2 documents (i.e., 48% and 49%).

We interpret this to mean that in 2-turn conversations, LLMs pay roughly equal attention to documents in either turn. Analysis of summaries generated in turns 3-8 of sharded simulations reveals an imbalance in the documents the LLM cites to. In eighth-turn summaries, 20% of citations are to documents introduced in turn 8, compared to 8% from turn 2 and 3 (150% difference). At a high level, as the conversation progresses, LLMs are most likely to cite either documents in the first or last turns, and less likely to cite documents introduced in intermediary (middle) turns. This finding mirrors findings of a *loss-in-the-middle* phenomenon of LLMs paying more attention to documents at the start or end of their provided context, at the cost of middle-context content (Huang et al., 2023; Liu et al., 2024; Laban et al., 2024). In short, we observe that the lost-in-the-middle phenomenon occurs not only in single-turn long-context settings but also in multi-turn conversations. We name this phenomenon *loss-in-middle-turns*.

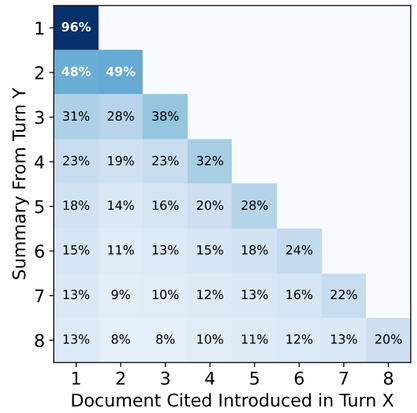


Figure 9: Analysis of citation patterns in summaries generated by LLMs with the Sharded simulation. At each turn, the LLM generates an updated summary (y-axis), which includes citations from the documents that have been revealed up to this turn. Percentages in a row do not add up to 100% due to citation hallucinations that occur for some models.

We note that the analysis presented in Figure 9 averages numbers across the 15 LLMs included in our main experiment. Even though we observe some loss-in-middle-turns in all models, the magnitude of the effect varies across models, typically with more performant models having a more muted effect, showing they have better capabilities of handling attribution across multiple turns of context. We do not include model-specific analyses in this work and leave it for future work.

F.4 Overly-verbose Assistant Responses

When simulating multiple conversations based on a common instruction, we observe variation in responses, particularly in the length of the response generated by the LLM. To understand how verbosity (length of a response) affects model performance, we perform a verbosity analysis.

One difficulty with assessing verbosity is that different tasks and instructions might require different levels of verbosity. For example, generating a Python function likely requires a longer than generating an SQL query. To regularize for task-specific variations, we assign a *verbosity tag* calculated for each (LLM, instruction) tuple. For each simulated sharded conversation involving an LLM on an instruction, we calculate the average length of the per-turn response (number of total characters in assistant responses divided by number of turns). We then bin conversations into quintiles according to this metric. More specifically, since we simulated $N = 10$ conversations for each (model, instruction)

F.3 基于对话最后一轮的过度调整

由于摘要任务要求助手通过引用将其摘要归因于文档，该任务提供了一个独特的机会，可以分析随着多轮对话的进展，大语言模型关注哪些轮次的信息。需要提醒的是，摘要任务涉及用户在每一轮引入新文档。因此，我们的分析旨在了解文档引入顺序（跨轮次）是否会影响大语言模型引用文档的可能性。

在图9中，我们绘制了分析结果。每一行对应于在分片模拟中特定轮次生成的摘要的分析。在第1轮（顶行），96%的被引用文档是在第一轮引入的。缺失的4%对应于对未引入文档的幻觉引用，这也解释了为何所有行的分布总和均未达到100%。在第二轮（从上数第二行），摘要对第一轮和第二轮文档的引用比例大致相等（即48%和49%）。

我们将其解释为：在双轮对话中，大语言模型对任一**轮次**的文档给予大致同等的关注。对分片模拟中第3至8轮生成的摘要的分析揭示了大语言模型所引用文档存在不平衡。

在第八轮摘要中，20%的引用指向第8轮引入的文档，而来自第2轮和第3轮的引用仅占8%（相差150%）。总体而言，随着对话进度推进，大语言模型最倾向于引用首轮或最后一轮的文档，而较少引用中间轮次引入的文档。这一发现与中间信息丢失现象相呼应，即大语言模型对其所提供上下文的首尾部分文档投入更多关注，而牺牲了中间上下文内容（Huanget al., 2023; Liu etal., 2024; Labanet al., 2024）。简而言之，我们观察到中间信息丢失现象不仅出现在单轮长上下文设置中，也存在于多轮对话中。我们将此现象命名为中间轮次信息丢失。

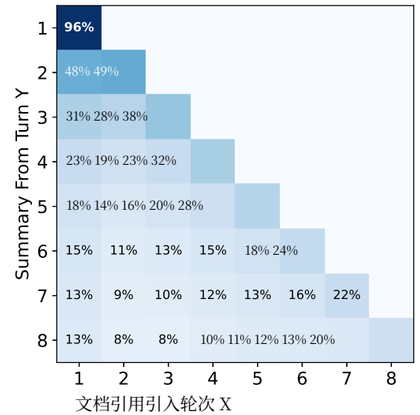


图9：使用分片模拟分析大语言模型生成摘要中的引用模式。在每一轮次，大语言模型生成一个更新的摘要（y轴），其中包含截至该轮次已揭示文档的引用。由于某些模型存在引用幻觉，同一行中的百分比相加不等于100%。

我们注意到，图9中呈现的分析数据是我们主实验中包含的15个大语言模型的平均值。尽管我们在所有模型中都观察到一定程度的中间轮次信息丢失，但其影响程度因模型而异，通常性能更强的模型影响更弱，这表明它们具备更好的能力来处理跨越多个轮次上下文的归因。本文未包含针对特定模型的分析，留待未来工作完成。

F.4 过度冗长的助手响应

当基于同一指令模拟多个对话时，我们观察到响应存在差异，尤其是在大语言模型生成的响应长度方面。为了解冗长度（响应长度）如何影响模型性能，我们进行了一项冗长度分析。

评估冗长度的一个难点在于，不同的任务和指令可能需要不同级别的冗长度。例如，生成一个Python函数很可能比生成一个SQL查询需要更长的响应。为了规范化任务特定的差异，我们为每个（大语言模型，指令）元组分配一个冗长度标签。对于每个涉及大语言模型在一条指令上的模拟分片对话，我们计算每轮响应的平均长度（助手响应中的总字符数除以轮次数）。然后，我们根据此指标将对话分为五个五分位数。更具体地说，由于我们为每个（模型，指令）模拟了 $N = 10$ 个对话，

Task	Relative Assistant Verbosity				
	0-20%	20-40%	40-60%	60-80%	80-100%
Assistants responses are ...	shortest	short	median	long	longest
Code	55.3	52.3	48.9	46.9	42.5
Math	62.9	64.0	62.1	60.9	56.1
Database	43.8	40.0	37.3	34.3	31.3
Actions	41.5	49.6	54.2	53.6	50.8
Data-to-Text	25.0	24.3	24.0	23.1	21.8
Summary	15.4	14.7	13.5	12.0	10.3
Average	40.7	40.8	40.1	38.6	35.6

Table 4: Averaged performance ($\overline{P}$) of LLMs on the six experimental tasks, arranged based on model relative verbosity (length of response). Performance degrades when models generate longer responses on five of the six tasks.

pair, we assign 2 simulations per quintile, which we name: shortest, short, median, long, and longest. We then calculate the averaged performance ($\overline{P}$) on the six experimental tasks, arranged based on this verbosity tag. Results are summarized in Table 4.

On five of the six tasks, performance is 10-50% higher in simulated conversations with the shortest response length, compared to conversations with the longest response length. As assistant responses get longer (left to right in the Table), performances gradually drop. The Actions task is the only task where such an effect is not observed, and where the shortest response length from the assistant is detrimental.

However, models primarily achieve higher performance when they generate shorter responses. We hypothesize that deterioration due to over-verbosity is due to longer responses typically containing more assumptions or hypotheses from the assistant, which can lead to confusion in following turns. On the other hand, short turns tend to be focused (e.g., a single clarification question) and keep the conversation on track.

Deterioration due to over-verbosity is noteworthy, as besides deteriorating underlying model performance, longer responses also take longer for users to read, which is undesirable. Therefore, the finding indicates that longer LLM responses are bad both for models and end-users.

G Implications

G.1 Implications for System and Agent Builders

Building LLM-based applications typically involves complex processes: decomposition of problems, retrieval of relevant information, use of tools, and calling of actions. Such processes are typically orchestrated by an agentic framework (such as Autogen (Wu et al., 2023) or LangChain (Chase, 2022)) that allows system builders to compose workflows with LLM calls as individual blocks. As such, an argument could be made that multi-turn capabilities are not a necessary feature of LLMs, as it can be offloaded to the agent framework. In other words, do we need native multi-turn support in LLMs when an agent framework can orchestrate interactions with users and leverage LLMs only as single-turn operators?

To answer this question, we implemented two agent-style conversation simulation types: Recap and Snowball. Both preprocess user utterances before sending them to the LLM. In Recap, a conversation proceeds in the same way as Sharded, but a user turn is added at the end, which recapitulates all the previous user turns. Snowball is a more gradual recapitulation: at each turn, the user simulator reveals a new shard, and repeats all previously revealed shards at that point. Both simulation types repeat the past user's turn information to make it more prominent and give the LLM a chance to leverage the redundancy to improve its responses. We include the experimental detail in Appendix N.

Task	相对助手冗长度				
	0-20%	20-40%	40-60%	60-80%	80-100%
助手回-应是.....	最短	短	中位数	long	最长
Code	55.3	52.3	48.9	46.9	42.5
Math	62.9	64.0	62.1	60.9	56.1
数据库	43.8	40.0	37.3	34.3	31.3
行动	41.5	49.6	54.2	53.6	50.8
数据到文本	25.0	24.3	24.0	23.1	21.8
摘要	15.4	14.7	13.5	12.0	10.3
平均值	40.7	40.8	40.1	38.6	35.6

表4：大语言模型在六项实验任务上的平均性能（$\overline{P}$），按模型相对冗长度（响应长度）排列。在六项任务中的五项上，当模型生成更长的响应时，其性能会下降。

我们为每个五分位数分配2个模拟，并将其命名为：最短、短、中位数、长和最长。然后，我们计算在这六个实验任务上的平均性能（$\overline{P}$），并根据此冗长度标签进行排列。结果总结在表4中。

在六项任务中的五项上，与响应长度最长的对话相比，在响应长度最短的模拟对话中，性能要高出10-50%。随着助手响应变长（表中从左到右），性能逐渐下降。行动任务是唯一未观察到这种效应、且助手最短响应长度反而有害的任务。

然而，模型主要在生成较短响应时获得更高的性能。我们假设，过度冗长导致的性能下降是因为较长的响应通常包含更多来自助手的假设或推测，这可能导致后续轮次的对话产生混淆。另一方面，简短的轮次往往更聚焦（例如，一个单一的澄清问题），并能使对话保持在正轨上。

因过度冗长导致的性能退化值得注意，因为除了会降低底层模型的性能外，更长的响应也需要最终用户花费更长时间阅读，这是不可取的。因此，这一发现表明，更长的大语言模型响应对模型和最终用户都是不利的。

G 启示

G.1 对系统和智能体构建者的启示

构建基于大语言模型的应用通常涉及复杂的过程：问题分解、相关信息检索、工具使用以及行动调用。此类过程通常由智能体框架（如 Autogen (Wu 等人, 2023) 或 LangChain (Chase,2022)) 来编排，它允许系统构建者将大语言模型调用作为独立模块来组合工作流。因此，可能会有人认为多轮对话能力并非大语言模型的必要特性，因为它可以卸载给智能体框架。换句话说，当智能体框架可以编排与用户的交互，并仅将大语言模型用作单轮操作器时，我们是否还需要大语言模型具备原生多轮对话支持？

为了回答这个问题，我们实现了两种智能体风格对话模拟类型：Recap和Snowball。两者在将用户话语发送给大语言模型之前都会进行预处理。在Recap中，对话的进行方式与Sharded相同，但在末尾添加了一个用户轮次，该轮次回顾了之前所有的用户轮次。Snowball 是一种更渐进的回顾：在每一轮次，用户模拟器揭示一个新的分片，并在该点重复所有先前已揭示的分片。两种模拟类型都重复过去用户的轮次信息，以使其更加突出，并给予大语言模型利用冗余来改进其响应的机会。我们在附录N中包含了实验细节。

Model	Simulation Type				
4o-mini	86.8	84.4	50.4	66.5	61.8
4o	93.0	90.9	59.1	76.6	65.3

Table 5: Experimental Results with additional simulation types: Recap and Snowball. Both strategies involve repeating user-turn information to mitigate models getting lost in conversations.

Model	Tasks				Avg.
4o	67.9	**65.0**	42.3	61.3	59.1
4o + system prompt	**68.7**	59.0	**45.2**	**68.0**	**60.2**

Table 6: Comparing performance in SHARDED conversations of GPT-4o given no system prompt (default) vs. providing a specialized system prompt hinting that the conversation will likely be underspecified. Results reported on four tasks: Math, Actions, Database, and Code.

Table 5 summarizes the results on all instructions for four tasks (Code, Database, Math, Actions) for two tested models (GPT-4o, GPT-4o-mini). Both RECAP and SNOWBALL demonstrate some level of success, with improvements over SHARDED simulations, but the performance still lags behind FULL or CONCAT. While RECAP outperforms SNOWBALL, we note that RECAP is an unrealistic setting because the intervention is conducted on the *last turn* of the conversation, which is not known a priori when conversation unfolds with a real user. SNOWBALL gives a sense of realistic performance gains achievable through user-turn repetition: it can mitigate the FULL-to-SHARDED performance deterioration by 15-20%. In short, relying on an agent-like framework to process information might be limiting, and we argue LLMs should natively support multi-turn interaction.

Besides agent-like procedures such as snowball and recap, we conducted an additional experiment to study the effect of system prompts on assistant model performance. Specifically, we re-ran sharded simulations with the GPT-4o adding an additional sentence at the end of the system prompt provided to the assistant during simulation: "*Be aware that the user might not provide all the information needed to solve the task at once, and that you can ask for clarifications if needed.*" The intent of this additional system specification is to observe the model can shift its behavior based on the provided hint, and effectively avoid getting lost in conversation.

Results of the experiments on the four non-refinement tasks are summarized in Table 6. In summary, adding the system prompt yields small improvements (+1 to +3) on two tasks, large improvements on one task (+7) and a drop on one task (-6), totaling a small improvement (+1%) on average. In conclusion, we do observe qualitatively that adding a hint to the system prompt affects model behavior, but it does not successfully help the model avoid getting lost in the conversation.

G.2 IMPLICATIONS FOR LLM BUILDERS

A lot of effort has been put in improving LLM *aptitude*: demonstrating that LLMs can accomplish tasks of increasing intellectual complexity, with recent results showing LLMs can compete in mathematics Olympiads, or solve Ph.D.-level technical questions in a benchmark aptly named Humanity's Last Exam (Phan et al., 2025).

In this work, we call on LLM builders to prioritize *reliability* of the models they build, as our experiments demonstrate that the randomness involved in generating text with LLMs leads to catastrophic unreliability in all the models we tested, degrading the quality of responses the average LLM users see.

LLMs are probabilistic systems, with parameters such as *temperature* that can adjust the degree of randomness that occurs while generating text. A possible argument is therefore: does setting the temperature to its lowest setting ($T = 0$) effectively resolve the reliability concern, as it makes the generation process more (but not entirely) deterministic?

模型	模拟类型				
4o-mini	86.8	84.4	50.4	66.5	61.8
4o	93.0	90.9	59.1	76.6	65.3

表5：包含额外模拟类型的实验结果：回顾与滚雪球。两种策略都涉及重复用户轮次信息，以缓解模型在对话中迷失的问题。

模型	任务				Avg.
4o	67.9	**65.0**	42.3	61.3	59.1
4o + 系统提示	**68.7**	59.0	**45.2**	**68.0**	**60.2**

表6：比较GPT-4o在SHARDED对话中的性能，分别使用无系统提示（默认）与提供专用系统提示（暗示对话可能未充分指定）的情况。结果在四个任务上报告：数学、行动、数据库和代码。

表5总结了针对四个任务（代码、数据库、数学、行动）和两个测试模型（GPT-4o、GPT-4o-mini）在所有指令上的结果。RECAP和SNOWBALL都展示了一定程度的成功，相较于SHARDED模拟有所改进，但其性能仍然落后于FULL或CONCAT。虽然RECAP的表现优于SNOWBALL，但我们注意到RECAP是一种不现实的设置，因为干预是在对话的最后一轮进行的，而这在与真实用户展开对话时是无法先验知晓的。SNOWBALL给出了通过用户轮次重复可实现的现实性能增益感：它可以将FULL到SHARDED的性能下降减轻15-20%。简而言之，依赖类智能体框架来处理信息可能具有局限性，我们认为大语言模型应原生支持多轮交互。

除了滚雪球和回顾这类类代理程序外，我们还进行了一项额外实验，以研究系统提示对助手模型性能的影响。具体来说，我们使用GPT-4o重新运行了分片模拟，并在模拟期间提供给助手的系统提示末尾添加了一句话："请注意，用户可能不会一次性提供解决任务所需的全部信息，如果需要，你可以请求澄清。" 这一额外系统规范的目的是观察模型能否根据提供的提示调整其行为，从而有效避免在对话中迷失。

在四个非精炼任务上的实验结果总结于表6。总之，添加系统提示在两个任务上带来了小幅改进（+1 至 +3），在一个任务上带来大幅改进（+7），在一个任务上出现下降（-6），平均总计为小幅改进（+1%）。综上所述，我们确实在定性上观察到，在系统提示中添加提示会影响模型行为，但它并未成功帮助模型避免在对话中迷失。

G.2 对大语言模型构建者的启示

大量的努力被投入到提升大语言模型的能力上：证明大语言模型能够完成智力复杂度日益增长的任务，最近的结果显示大语言模型可以在数学奥林匹克竞赛中竞争，或者在一个恰如其分地命名为"人类终极考试"的基准测试中解决博士级技术问题（Phan 等人，2025）。

在这项工作中，我们呼吁大语言模型构建者优先考虑他们所构建模型的可靠性，因为我们的实验表明，使用大语言模型生成文本所涉及的随机性会导致我们测试的所有模型出现灾难性不可靠性，从而降低普通大语言模型用户所见响应的质量。

大语言模型是概率系统，具有诸如温度等参数，可以调整生成文本时出现的随机性程度。因此，一个可能的论点是：将温度设置为最低值（$T = 0$）是否有效地解决了可靠性问题，因为它使生成过程更加（但并非完全）确定?

Simulation	4o-mini AT=1.0	4o-mini AT=0.5	4o-mini AT=0.0	4o AT=1.0	4o AT=0.5	4o AT=0.0
FULL	16.0	15.0	6.8	17.8	8.0	2.8
CONCAT	20.2	17.8	9.5	20.2	17.8	5.8
UT=1.0	49.8	46.8	51.0	41.0	43.8	31.8
UT=0.5	31.7	34.0	40.5	39.5	40.8	31.8
UT=0.0	38.5	28.0	30.5	35.8	38.0	29.7

Table 7: Unreliability of models when changing assistant temperature (AT) and user temperature (UT) in FULL, CONCAT and SHARDED settings. The lower the number the more reliable the assistant is.

To evaluate this argument, we conducted a supplementary experiment in which the assistant's temperature for generating responses (AT) was varied to three values: 1.0, 0.5, and 0.0. Additionally, since SHARDED simulation uses an LLM-based user simulator, we also varied the user's temperature (UT) with the same three values. Further details on the experiment, including sample selection and simulation scale, are in Appendix M.

Table 7 summarizes the experimental findings. Looking at the FULL and CONCAT settings (first two rows), both GPT-4o-mini and GPT-4o observe a large improvement in reliability when temperature is decreased, with a drop in unreliability (U_{10}^{90}) of 50-80% when the assistant temperature decreases. Results from SHARDED simulations are more alarming: GPT-4o-mini does not see improvements in reliability as AT is decreased (in all user-temperature settings), and GPT-4o only sees minor improvements, on the order of 15-20%. Even when both the user and assistant temperatures are set to 0.0, there remains a large unreliability of around 30%. Even though language models are supposed to be deterministic at $T = 0.0$, this is known to practically not be the case for modern LLMs (see Appendix O for discussion). At a high level, single-turn conversations have limited scope for deviation, whereas one token difference in an early turn of a multi-turn conversation can lead to cascading deviations, which we observe as stagnated unreliability. For settings that involve multi-turn interaction, we find that **lowering the temperature of the LLM when generating responses is ineffective in improving system reliability.**

We invite and challenge LLM builders to jointly optimize model aptitude and reliability. A reliable LLM should: (1) achieve similar aptitude in single- and multi-turn settings, (2) have small unreliability ($U_{10}^{90} < 15$) in multi-turn settings, (3) achieve these at unmodified temperature ($T = 1.0$), demonstrating that the underlying language model can handle variations that naturally occur in language generation.

G.3 IMPLICATIONS FOR NLP PRACTITIONERS

Our experiments demonstrate that model behavior in single- and multi-turn settings on the same underlying set of instructions can diverge in important ways, for example, with large observed degradations in performance and reliability.

We selected the initial six tasks to span a wide range of generation tasks, from programming to multi-document summarization. Yet this set of tasks is limited across multiple dimensions, such as focusing on English-language instructions and analytical (i.e., non-creative) tasks. We put effort into making the sharding process scalable by automating portions that could be handled by an LLM, while manually validating and finalizing samples for quality control. The sharding process – detailed in Appendix C – required an average of three hours of manual work (prompt engineering or inspection) from an author to prepare and finalize 100 sharded instructions.

We encourage NLP practitioners to experiment with sharding and release sharded versions of their tasks and instructions alongside fully specified ones.

To illustrate the feasibility of sharding new tasks, and understand compatibility requirements for sharding, we prepared sharded instructions for a seventh task: Translation. The task consists of translating an entire document (10 sentences) from German to English, leveraging paired documents from WMT 2019 on document-level translation (Scherrer et al., 2019). In the SHARDED setting, each turn reveals two additional sentences from the source document and requires the assistant to translate

模拟	4o-mini 助手温度=1.0	4o-mini 助手温度=0.5	4o-mini 助手温度=0.0	4o 助手温度=1.0	4o 助手温度=0.5	4o 助手温度=0.0
FULL	16.0	15.0	6.8	17.8	8.0	2.8
拼接	20.2	17.8	9.5	20.2	17.8	5.8
UT=1.0	49.8	46.8	51.0	41.0	43.8	31.8
UT=0.5	31.7	34.0	40.5	39.5	40.8	31.8
UT=0.0	38.5	28.0	30.5	35.8	38.0	29.7

表7：在完整、拼接和分片设置中，改变助手温度和用户温度时模型的不可靠性。数值越低，助手越可靠。

为了评估这一论点，我们进行了一项补充实验，其中助手生成响应的温度（AT）被调整为三个值：1.0、0.5和0.0。此外，由于SHARDED模拟使用基于大语言模型的用户模拟器，我们也以相同的三个值调整了用户的温度（UT）。关于实验的更多细节，包括样本选择和模拟规模，请参见附录M。

表7总结了实验结果。观察完整和拼接设置（前两行），GPT-4o-mini和GPT-4o在温度降低时都观察到可靠性的大幅提升，当助手温度降低时，不可靠性（U_{10}^{90}）下降了50-80%。分片模拟的结果则更令人担忧：GPT-4o-mini在助手温度降低时（在所有用户温度设置下）并未看到可靠性提升，而GPT-4o仅观察到约15-20%的微小提升。即使将用户和助手温度都设置为0.0，仍存在约30%的较大不可靠性。尽管语言模型在 $T = 0.0$时理应是确定性的，但已知对于现代大语言模型而言，实际情况并非如此（讨论见附录O）。从高层次来看，单轮对话产生偏差的范围有限，而多轮对话早期轮次中的一个词元差异可能导致级联偏差，我们将其观察为停滞的不可靠性。对于涉及多轮交互的设置，我们发现**在生成响应时降低大语言模型的温度，对于提升系统可靠性是无效的。**

我们邀请并挑战大语言模型构建者，共同优化模型能力与可靠性。一个可靠的大语言模型应做到：(1) 在单轮与多轮设置中实现相近的能力，(2) 在多轮设置中具有较低的不可能性（$U_{10}^{90} < 15$），(3) 在未修改温度 ($T = 1.0$) 的情况下实现上述目标，以证明底层语言模型能够处理语言生成中自然出现的变体。

G.3 对自然语言处理从业者的启示

我们的实验表明，模型在相同底层指令集上，于单轮与多轮设置中的行为可能在重要方面出现分歧，例如，观察到性能与可靠性的大幅下降。

我们选择初始六个任务，旨在覆盖从编程到多文档摘要的广泛生成任务范围。然而，这组任务在多个维度上存在局限，例如专注于英语指令和分析性（即非创造性）任务。我们努力使分片过程具备可扩展性，通过自动化可由大语言模型处理的部分，同时手动验证并最终确定样本以进行质量控制。该分片过程——详见附录C ——平均需要作者投入三小时的人工工作（提示工程或检查）来准备并最终确定100条分片指令。

我们鼓励自然语言处理从业者尝试进行分片，并在发布完全指定的版本的同时，也发布其任务和指令的分片版本。

为了说明对新任务进行分片的可行性，并理解分片的兼容性要求，我们为第七个任务——翻译——准备了分片指令。该任务包括将整个文档（10个句子）从德语翻译成英语，利用来自WMT 2019的配对文档进行文档级翻译（Scherrer等人，2019）。在分片设置中，每一轮次揭示源文档中的两个额外句子，并要求助手翻译

Model	Translation		
4o-mini	41.7	43.4	42.1
4o	35.9	38.5	40.9

Table 8: Performance on the translation task for FULL, CONCAT, and SHARDED simulations.

all sentences provided so far, whereas the FULL and CONCAT settings reveal the entire document in the first turn. Evaluation is conducted with the standard BLEU metric (Papineni et al., 2002). We describe practical implementation details in Appendix J.

Results from FULL, CONCAT, and SHARDED simulations are summarized in Table 8. Both models we tested – GPT-4o-mini and GPT-4o – do *not* exhibit degradation in performance in the SHARDED setting, with BLEU scores being within 10% difference of each other in all settings. We believe this result reflects that the task can largely be accomplished at the sentence-level despite some prior work that has framed translation at the document-level (Post & Junczys-Dowmunt, 2023), and that the BLEU score does not adequately capture document-level nuances (Ma et al., 2021). In other words, if a task is episodic (i.e., it can be decomposed into turn-level subtasks), the models can avoid getting lost in conversation by completing each subtask without having to handle multi-turn context. In short, the SHARDED Translation task simulates multi-turn conversations that are not underspecified.

We now list task properties we believe are important in leading models to get lost in conversation in multi-turn settings. First, generative tasks (*i.e.*, unlike extractive QA or classification) are more prone to model confusion, as they typically involve editing and refinement of new content. Second, the generative tasks should be sufficiently complex, involving multiple explicit specifications that will yield a multitude of shards. For example, an instruction: "Write a Python program that calculates $1 + 1$" is too simple to shard. Third, the solution or answer should be non-decomposable, such that revealing a shard modifies the entire solution (unlike the translation task, where each additional shard only asks to translate and append to the ongoing solution). We hypothesize that LLMs tested on tasks with the aforementioned three properties will likely get lost in conversation, evidenced by a large drop in averaged performance and reliability in SHARDED simulations.

G.4 IMPLICATIONS FOR USERS OF CONVERSATIONAL SYSTEMS

Users of LLM-based products should be aware of the lack of reliability of LLMs, particularly when used in multi-turn settings. Generally available generative technology is new, and prior work has identified the randomness in LLM-generated text as a point of confusion for users (Mylrea & Robinson, 2023; Weisz et al., 2024; Venkit et al., 2024; Lee et al., 2024). We make two practical recommendations that can help users of LLM-based systems get the most out of their exchanges.

If time allows, try again. If a conversation with an LLM did not lead to expected outcomes, starting a new conversation that repeats the same information might yield significantly better outcomes than continuing an ongoing conversation. This is because current LLMs can get lost in the conversation, and our experiments show that persisting in a conversation with the model is ineffective. In addition, since LLMs generate text with randomness, a new conversation may lead to improved outcomes.

Consolidate before retrying. Since LLMs are ineffective at dealing with information dispersed across multiple turns, consolidating instruction requirements into a single instruction is an effective strategy to improve the model's aptitude and reliability (as shown by the CONCAT experiments). When a user notices that a model is lost in conversation, they can ask the LLM: "Please consolidate everything I've told you so far," then bring the response to a new conversation, alleviating the need for manual consolidation. In practice, there is anecdotal evidence that early adopters of LLM-based applications are aware that LLMs get lost in conversation. For example, users of the Cursor LLM-based coding environment report that frequently creating new conversations "whenever they can"

模型	翻译		
4o-mini	41.7	43.4	42.1
4o	35.9	38.5	40.9

表8：完整、拼接和分片模拟在翻译任务上的性能。

迄今为止提供的所有句子，而完整和拼接设置则在第一轮次就揭示整个文档。评估使用标准的BLEU指标（Papineni等人，2002）进行。我们在附录J中描述了实际实现细节。

完整、拼接和分片模拟的结果总结在表8中。我们测试的两个模型——GPT-4o-mini和GPT-4o——在分片设置中并未表现出性能下降，在所有设置中，BLEU分数彼此相差均在10%以内。我们认为这一结果反映出，尽管先前有些工作将翻译定位在文档层面（Post& Junczys-Dowmunt，2023），但该任务在很大程度上可以在句子层面完成，并且BLEU分数无法充分捕捉文档层面的细微差别（Ma等人，2021）。换言之，如果一个任务是片段式的（即可以分解为轮次级子任务），模型可以通过完成每个子任务而无需处理多轮次上下文，从而避免在对话中迷失。简而言之，分片翻译任务模拟的是非未充分指定的多轮对话。
我们现在列出我们认为在多轮对话场景中导致大语言模型对话迷失的重要任务属性。首先，生成式任务（即，与抽取式问答或分类不同）更容易引发模型混淆，因为它们通常涉及对新内容的编辑和精炼。其次，生成式任务应足够复杂，包含多重明确规范，从而产生大量分片。例如，指令"编写一个计算1 + 1的Python程序"就过于简单，无法进行分片。第三，解决方案或答案应具有不可分解性，使得揭示一个分片会修改整个解决方案（这与翻译任务不同，在翻译任务中，每个新增分片仅要求翻译并附加到正在进行的解决方案上）。我们假设，在具有上述三个属性的任务上进行测试的大语言模型很可能在对话中迷失，这体现在分片模拟中平均性能和可靠性的大幅下降。

G.4 对对话系统用户的启示

基于大语言模型产品的用户应意识到大语言模型缺乏可靠性，尤其是在多轮对话场景中使用时。目前普遍可用的生成式技术尚属新兴领域，先前的研究已指出大语言模型生成文本的随机性会给用户带来困惑（Mylrea &Robinson, 2023；Weisz 等人, 2024；Venkit 等人, 2024；Lee 等人, 2024）。我们提出两条实用建议，可帮助基于大语言模型的系统用户从其交流中获得最大收益。

如果时间允许，请重试。如果与大语言模型的对话未能达到预期结果，开启一个重复相同信息的新对话，可能比继续当前对话产生显著更好的结果。这是因为当前的大语言模型可能会在对话中迷失方向，且我们的实验表明，坚持与模型进行持续对话是无效的。此外，由于大语言模型生成文本具有随机性，新对话可能会带来更好的结果。

在重试前进行整合。由于大语言模型在处理分散在多个轮次中的信息方面效果不佳，将指令要求整合为单一指令是提升模型能力和可靠性的有效策略（如CONCAT实验所示）。当用户注意到模型在对话中迷失时，他们可以要求大语言模型："请将我迄今为止告诉你的一切内容整合起来"，然后将响应带入一个新对话，从而免去手动整合的需要。实际上，有轶事证据表明，基于大语言模型应用的早期使用者已经意识到大语言模型会在对话中迷失。例如，使用Cursor大语言模型编码环境的用户报告称，频繁创建新对话"在可能的情况下"

Name	Description	Example
Answer attempt	The response contains a complete answer attempt to the question that can be extracted verbatim.	The dog is 50 meters away from the house.
Clarification	The response is a brief single question that directly inquires about one aspect of the query.	To calculate the distance, I need to know how long the dog ran. Could you provide more information about that?
Interrogation	The response contains multiple questions addressed to the user.	I cannot answer the question without knowing (1) speed, (2) duration, and (3) starting position. Please tell me about these points and I can calculate the distance!
Discussion	The response discusses the question in detail without answering, asking, or refusing to answer.	The question is trying to measure the distance between the dog and the house. We can calculate based on this equation: [Equation]. [. . .]
Hedging	The response provides multiple answer candidates based on hypotheticals (ifs, cases).	1. If the dog was originally in the house, it would be 50 meters away now. 2. If the dog was at the park, it would be 100 meters away from the house now.
Refusal	The response refuses to answer the question without a follow-up question or a request.	I can't answer your question because I don't have sufficient information.
Missing	The response is empty.	[blank]

Table 9: Definition of turn categories. We include the description in the prompt to categorize assistant responses.

is a recommended strategy to ensure high quality responses even though the tool allows to keep conversations going indefinitely.[6]

These two recommendations remain cumbersome for users and can only offer patched solutions rather than a principled approach. Once future LLMs can more reliably handle multi-turn conversations, the need for such recommendations should be alleviated, allowing users to communicate underspecified instructions over multiple turns naturally with less risk of the model getting lost in conversation.

H Assistant Response Categorization

We categorize each assistant response into one of the seven categories to capture the answer attempt and evaluate if that is the case, as well as to understand the model's behavior tendency. Herlihy et al. (2024) defined seven turn categories for LLM responses and classified them using LLM, uncovering that GPT-4 prefers answering directly even when the query is underspecified. Motivated by this study, we similarly define seven response categories, which we list in Table 9, together with example responses. Key differences are discussion and answer attempt; we observed many responses containing a large body of text formulating the question in our preliminary experiments, which led to redefining "Miscellaneous" from (Herlihy et al., 2024) into "Discussion" in our experiment. "Direct Response" in (Herlihy et al., 2024) corresponds to our "Answer Attempt."

I Model Access

We accessed models that were used in the experiments from various vendors. The short form names we used throughout the paper, the corresponding versions, and the providers are summarized in Table 10. Except for the exploration with various temperatures (Section G.2), we set the temperature to $T = 1.0$ and used the default values for the rest of the configurable hyperparameters. We set the maximum response length to 1,000 tokens for all models, and did not observe models exceeding this

[6] https://www.reddit.com/r/cursor/comments/1j72r8d/when_to_start_a_new_chat/

Name	描述	示例
答案尝试	响应中包含一个完整的答案尝试，该答案可以从问题中提取逐字记录。	狗距离房子50米。
澄清	该响应是一个简短的单一问题，旨在直接询问查询的某个方面。	为了计算距离，我需要知道长 狗 跑。您能提供更多关于那个的信息吗?
询问	该响应包含多个向用户提出的问题。	我无法在不知道包括 (1) 速度、(2) 持续时间，以及 (3) 起始位置。请告诉我这些信息，我就可以计算距离了！
讨论	该响应详细讨论了问题，但并未回答、询问或拒绝回答。	这个问题试图测量狗与房子之间的距离。我们可以根据这个方程计算：{v1}方程{v2}。根据此方程计算： [方程]。[. . .]
模糊表达	该响应提供了多重答案可基于假设（如果、情况）的候选方案。	1. 如果狗最初在房子里，它现在应该在50米外。 2. 如果狗在公园，它现在距离房子应该有100米远。
拒绝	该响应拒绝回答问题且没有后续问题或请求。	我无法回答您的问题，因为我没有足够的信息。
缺失	响应为空。	[blank]

表9：轮次类别定义。我们在提示词中包含此描述，用于对助手响应进行分类。

是确保高质量响应的推荐策略，尽管该工具允许对话无限期持续下去。[6]

这两项建议对用户而言仍然繁琐，并且只能提供修补性的解决方案，而非原则性的方法。一旦未来的大语言模型能够更可靠地处理多轮对话，此类建议的需求就应得到缓解，从而允许用户更自然地通过多轮次传达欠明确指令，同时降低模型在对话中迷失的风险。

H 助手响应分类

我们将每个助手响应归类到七个类别之一，以捕捉答案尝试并评估情况是否如此，同时理解模型的行为倾向。Herlihy等人(2024)为大语言模型响应定义了七个轮次类别，并使用大语言模型对其进行了分类，揭示了GPT-4即使在查询未充分指定时也倾向于直接回答。受此研究启发，我们类似地定义了七个响应类别，并列于表9中，同时附有示例响应。关键区别在于讨论和答案尝试；我们在初步实验中观察到许多响应包含大量阐述问题的文本，这促使我们将(Herlihy等人, 2024)中的“杂项”重新定义为本次实验中的“讨论”。(Herlihy等人, 2024)中的“直接响应”对应于我们的“答案尝试”。

I 模型访问

我们从不同供应商处获取了实验中使用的模型。本文通篇使用的简称、对应版本及供应商信息总结于表10。除了在不同温度参数下的探索（章节G.2），我们将温度设置为 $T = 1.0$，其余可配置超参数均使用默认值。我们为所有模型设置了1000词元的最大响应长度，且未观察到模型超出此限制。

[6] https://www.reddit.com/r/cursor/comments/1j72r8d/when_to_start_a_new_chat/

limit frequently when generating responses. For thinking models (o3, Deepseek-R1), we increased the limit to 10,000 tokens to account for the additional test-time compute (thinking tokens).

Short Form	Name	Version	Access Provider
4o	GPT-4o	`gpt-4o-2024-11-20`	OpenAI / Microsoft API
4o-mini	GPT-4o-mini	`gpt-4o-mini-2024-07-18`	OpenAI API
4.1	GPT-4.1	`gpt-4.1-2025-04-14`	OpenAI / Microsoft API
o3	o3	`o3-2025-04-16`	OpenAI / Microsoft API
3-Haiku	Claude 3 Haiku	`claude-3-haiku-20240307`	Amazon Bedrock
3.7-Sonnet	Claude 3.7 Sonnet	`claude-3-7-sonnet-20250219`	Amazon Bedrock
2.5-Flash	Gemini 2.5 Flash	`gemini-2.5-flash-preview-04-17`	Gemini API
2.5-Pro	Gemini 2.5 Pro	`gemini-2.5-pro-preview-03-25`	Gemini API
3.1-8B	Llama-3.1-8B-Instruct	N/A	Local Ollama
3.3-70B	Llama-3.3-70B-Instruct	N/A	Amazon Bedrock
4-Scout	Llama-4-Scout-17B-16E	N/A	Together AI
CMD-A	Command-A	`command-a-03-2025`	Cohere API
R1	Deepseek-R1	N/A	Amazon Bedrock
OLMo2	OLMo2-13B	N/A	Local Ollama
Phi-4	Phi-4	N/A	Local Ollama

Table 10: Specific model versions used as part of our experiments. For each model, we define the exact `Version` of the model accessed (for models that have versioning) and the `Access Provider` to facilitate result reproducibility.

J TASK-SPECIFIC IMPLEMENTATION DETAILS

We provide task implementation details. For each task, we specify: (1) the selection of original single-turn fully-specified instruction, (2) the evaluation metric that was repurposed from the original dataset, and (3) what the initial system messages consist of (if any).

J.1 CODE

The Code instructions are sourced from a combination of HumanEval (Chen et al., 2021), a dataset of 164 basic Python programming problems given the function header and the docstring that specifies the problem, and LiveCodeBench (Jain et al., 2024), an evolving dataset of Python algorithmic challenges. In particular, we source from the "call-based" problem subset in LiveCodeBench v5, with the difficulty of either "Easy" and "Medium", to align the solution formats between the two sources.

We first sharded all HumanEval problems following the protocol mentioned in Appendix C, obtaining 45 high-quality sets of shards that meet the criteria. The rest of the dataset was discarded because it was simplistic and did not produce shared instructions with enough shards. Subsequently, we shuffled and sharded the aforementioned subset from LiveCodeBench until obtaining 100 valid sharded instructions.

We follow the original prompts used by the benchmark authors as much as possible for the single-turn (FULL and CONCAT) evaluation. Specifically, FULL prompt from HumanEval includes the function header and the docstring provided as `prompt` in HumanEval dataset, and FULL & CONCAT from LiveCodeBench includes `starter_code` consisting of the function signature.

Both HumanEval- and LiveCodeBench-derived problems come with test cases, which we use to compute the functional accuracy of the answer attempt by the LLMs. We re-use the evaluation codebase maintained by Jain et al. (2024), which (1) wraps the candidate function in a test module, (2) execute given the inputs, and (3) checks the equivalence of the output from the expected output, with a default timeout set to prevent the evaluator from getting trapped during evaluation (*e.g.*, brute-force implementation may not pass under the set time budget). When multiple code blocks are present in a response, the answer extraction module selects the last function definition in the last markdown code block.

在生成响应时经常限制。对于思维模型（o3、Deepseek-R1），我们将限制提高到10,000个词元，以考虑额外的测试时计算（思维词元）。

短格式	Name	版本	访问提供商
4o	GPT-4o	`gpt-4o-2024-11-20`	OpenAI / 微软API
4o-mini	GPT-4o-mini	`gpt-4o-mini-2024-07-18`	OpenAI API
4.1	GPT-4.1	`gpt-4.1-2025-04-14`	OpenAI / 微软API
o3	o3	`o3-2025-04-16`	OpenAI / 微软API
3-Haiku	Claude 3 Haiku	`claude-3-haiku-20240307`	Amazon Bedrock
3.7-Sonnet	Claude 3.7 Sonnet	`claude-3-7-sonnet-20250219`	Amazon Bedrock
2.5-Flash	Gemini 2.5 Flash	`gemini-2.5-flash-preview-04-17`	Gemini API
2.5-Pro	Gemini 2.5 Pro	`gemini-2.5-pro-preview-03-25`	Gemini API
3.1-8B	Llama-3.1-8B-Instruct	N/A	本地Ollama
3.3-70B	Llama-3.3-70B-Instruct	N/A	Amazon Bedrock
4-Scout	Llama-4-Scout-17B-16E	N/A	Together AI
CMD-A	Command-A	`command-a-03-2025`	Cohere API
R1	Deepseek-R1	N/A	Amazon Bedrock
OLMo2	OLMo2-13B	N/A	本地Ollama
Phi-4	Phi-4	N/A	本地Ollama

表10：我们实验中使用的具体模型版本。对于每个模型，我们定义了所访问模型的确切 `Version`（针对有版本控制的模型）以及 `Access Provider`，以便于结果复现。

J 任务特定实现细节

我们提供任务实现细节。对于每个任务，我们说明：(1) 原始单轮完全指定指令的选择，(2) 从原始数据集中重新调整用途的评估指标，以及 (3) 初始系统消息包含哪些内容（如果有的话）。

J.1 代码

代码指令来源于HumanEval（Chen等人, 2021）和LiveCodeBench（Jain等人, 2024）的组合。HumanEval是一个包含164个基础Python编程问题的数据集，给定函数头和指定问题的文档字符串。LiveCodeBench是一个不断演进的Python算法挑战数据集。具体而言，我们从LiveCodeBench v5的“基于调用”的问题子集中选取难度为“简单”或“中等”的问题，以使两个来源的解决方案格式保持一致。

我们首先按照附录C中提到的协议对所有HumanEval问题进行分片，获得了45组符合标准的高质量分片。数据集的其余部分因过于简单且无法产生具有足够分片的分片指令而被丢弃。随后，我们对上述来自LiveCodeBench的子集进行随机打乱和分片，直到获得100个有效的分片指令。

对于单轮（完整和拼接）评估，我们尽可能遵循基准作者使用的原始提示词。具体来说，HumanEval的完整提示词包含HumanEval数据集中提供的函数头和文档字符串 `prompt`，而LiveCodeBench的完整和拼接提示词包含由函数签名组成的 `starter_code`。

无论是源自HumanEval还是LiveCodeBench的问题都附带有测试用例，我们利用这些测试用例来计算大语言模型答案尝试的功能准确率。我们复用了由Jain等人(2024)维护的评估代码库，该代码库会：(1) 将候选函数包装在一个测试模块中，(2) 根据给定的输入执行，以及(3) 检查输出与预期输出是否等价，并设置了默认超时以防止评估器在评估过程中陷入停滞（例如，暴力实现可能无法在设定的时间预算内通过）。当响应中存在多个代码块时，答案提取模块会选择最后一个Markdown代码块中的最后一个函数定义。

J.2 DATABASE

The Database instructions are sourced from the validation portion of the Spider dataset (Yu et al., 2018). We note that though a more recent version of Spider has been released (Spider 2.0 (Lei et al., 2024)), the instructions in the second iteration are more advanced and represent less typical database use, and we select instructions from the more realistic Spider 1.0.

The authors of Spider categorized queries into four levels of difficulty (EASY, MEDIUM, HARD, XHARD), based on the syntax complexity of a reference SQL query. We filtered out queries of EASY complexity, as they tended to yield fewer than three shards when processed. The rest of the 433 natural language queries in Spider were gradually sharded until reaching a total of 107 valid sharded instructions.

Each original instruction in Spider supplies a database schema, represented in SQL as a series of table schema (i.e., each defines a series of columns including name, type, and optional index). We include the database schema as part of the system message (i.e., before the first turn of conversation), and inform the LLM that users will provide natural-language queries that must be answered using a database with the provided schema.

Each original instruction in Spider is paired with a reference SQL solution. We follow Zhong et al. (2020) for the evaluation methodology. For a given original instruction, the candidate and reference SQL queries are executed on a fixed set of databases, and an exact match of the results on all databases is required to mark the candidate as successful (Score = 100). If a discrepancy is observed on any test database, the candidate is incorrect (Score = 0). One limitation of SQL execution is that false positives can occur: two queries can return the same output on a given database, even when they are not semantically equivalent. Zhong et al. (2020) found that by evaluating on an increased number of databases, false positives become negligible. Finally, any invalid candidate that does not successfully execute (e.g., syntax error) is considered incorrect (Score = 0).

J.3 ACTIONS

The Actions instructions are sourced from the released test portion of the Berkeley Function Calling Leaderboard V3 (BFCL) (Yan et al., 2024). BFCL V3 consists of three sub-genres of instructions: (1) Parallel, (2) Multiple, and (3) Multiple-Parallel. Initial experimentation with the sub-genres identified Parallel as the most suited for sharding, as Parallel instructions specify multiple subtasks that should be used and combined into a single action that accomplishes the entirety of the instruction. We shuffled all the BFCL V3 Parallel instructions, and sharded gradually until we obtained 105 valid sharded instructions.

We note that though a more recent iteration of BFCL includes multi-turn instructions, it differs from sharding experiments as it does not involve underspecification, with each turn having an independent intermediate solution (which we call episodic multi-turn conversations). Our implementation in comparison shards original instructions, allowing us to simulate multi-turn underspecified conversations for this task setting. The Background section (Section 2) discusses the relationship between episodic and underspecified multi-turn conversation more in-depth.

Each instruction in BFCL comes with tool set documentation, a JSON object that specifies the set of available actions (APIs) for the assistant to complete user instructions. We include the tool set documentation as part of the system message, along with a message indicating that user queries will require the use of the provided tools to be completed.

Each instruction in BFCL comes with a reference answer, consisting of the API calls that should be called to accomplish the user instruction. The maintainers of BFCL have released an evaluation toolkit that assesses semantic equivalence between a candidate answer and the reference answer. We leverage the official evaluation toolkit, assigning a score of S=100 for candidate answers that are considered semantically equivalent to the reference answer, and a score of S=0 otherwise. When the evaluation toolkit is not able to parse a candidate answer (e.g., a syntax error), the candidate is considered incorrect (S=0).

J.2 数据库

数据库指令来源于Spider数据集的验证部分（Yu等人,2018）。我们注意到，尽管Spider已发布了更新的版本（Spider 2.0（Lei等人,2024）），但第二次迭代中的指令更为高级，代表的是不那么典型的数据库使用场景，因此我们选择了更贴近现实的Spider 1.0中的指令。

Spider的作者根据参考SQL查询的语法复杂性，将查询分为四个难度级别（简单、中等、困难、极难）。我们过滤掉了简单复杂度的查询，因为它们在处理时往往产生少于三个分片。Spider中剩余的433个自然语言查询被逐步分片，最终得到总共107个有效的分片指令。

Spider中的每个原始指令都提供了一个数据库模式，在SQL中表示为一系列表模式（即每个模式定义了一系列列，包括名称、类型和可选的索引）。我们将数据库模式作为系统消息的一部分（即在对话的第一轮之前），并告知大语言模型，用户将提供自然语言查询，必须使用具有所提供模式的数据库来回答。

Spider中的每个原始指令都配有一个参考SQL解决方案。我们遵循Zhong等人(2020)的评估方法。对于给定的原始指令，候选和参考SQL查询会在固定的数据库集合上执行，并且需要在所有数据库上结果完全匹配，才能将候选标记为成功（分数 = 100）。如果在任何测试数据库上观察到差异，则候选为错误（分数 = 0）。SQL执行的一个局限性是可能出现误报：两个查询在给定数据库上可能返回相同的输出，即使它们在语义上并不等价。Zhong等人(2020)发现，通过在更多数量的数据库上进行评估，误报可以忽略不计。最后，任何未能成功执行（例如，语法错误）的无效候选都被视为错误（分数 = 0）。

J.3 行动

行动指令来源于伯克利函数调用排行榜V3（BFCL）已发布的测试部分（Yan等人,2024）。BFCL V3包含三种指令子类型：(1) 并行，(2) 多重，以及(3) 多重并行。对这些子类型的初步实验确定并行类型最适合进行分片，因为并行指令指定了多个应被使用并组合成单一行动以完成整个指令的子任务。我们打乱了所有BFCL V3并行指令，并逐步进行分片，最终获得了105个有效的分片指令。

我们注意到，尽管BFCL更新的迭代版本包含了多轮指令，但它与分片实验不同，因为它不涉及欠明确性，每一轮都有一个独立的中间解决方案（我们称之为情景式多轮对话）。相比之下，我们的实现是对原始指令进行分片，这使我们能够为此任务设置模拟多轮次欠明确对话。背景章节（第2节）更深入地讨论了情景式与欠明确性多轮对话之间的关系。

BFCL中的每条指令都附带工具集文档，这是一个JSON对象，它规定了助手为完成用户指令所能使用的可用操作（API）集合。我们将工具集文档作为系统消息的一部分，同时附上一条消息，表明用户查询需要借助提供的工具来完成。

BFCL中的每条指令都附带一个参考答案，其中包含为完成用户指令而应调用的API调用。BFCL的维护者发布了一个评估工具包，用于评估候选答案与参考答案之间的语义等价性。我们利用官方评估工具包，为被认为与参考答案语义等价的候选答案分配分数S=100，否则分配分数S=0 。当评估工具包无法解析候选答案（例如，出现语法错误）时，该候选答案被视为错误（S=0）。

J.4 MATH

The Math instructions are sourced from the "main" portion of the GSM8K dataset (Cobbe et al., 2021). We did not perform a filter on the original 8,700 instructions. We shuffled the instructions and sharded incrementally until we obtained 103 valid sharded instructions. Each GSM8K is paired with a numerical reference answer. We used the official toolkit released alongside GSM8K to standardize numerical answers (i.e., strip formatting, etc.). Standardized candidate numerical answers can then be compared through exact match to the reference answer. If the toolkit detects a match, the candidate answer is considered correct (Score=100), and incorrect otherwise (Score = 0). A short, single-sentence system prompt is used to indicate to the assistant that it will be solving mathematical problems.

J.5 DATA-TO-TEXT

The Data-to-Text instructions are based on instructions in the released test set ToTTo dataset (Parikh et al., 2020). In ToTTo, fully-specified instructions have the following information elements: (1) a HTML-formatted table extracted from a Wikipedia page, (2) a subset of cells in the table that have been *highlighted*, (3) the name of the Wikipedia page that included the Table, (4) the name of the Section in the Wikipedia page that included the Table. Given these elements, the task objective is to generate a caption for the Table, specifically focusing on the highlighted cells and considering the available metadata. Instructions were shuffled and sharded incrementally until we obtained 120 valid sharded instructions.

For each instruction, we generate sharded instructions by assigning different information elements to individual shards. The first shard consists of the initial HTML-formatted table without highlighting. The second shard provides an updated table with the highlighting present, the third shard provides the Wikipedia page name, the fourth shard provides the Wikipedia Section name. Finally, a fifth shard provides a fixed set of 10 randomly-selected example captions from the training set of the ToTTo dataset.

Each instruction in ToTTo is assigned one to three reference captions that were collected by the authors of the original dataset. Evaluation on a candidate caption calculates the BLEU score (Papineni et al., 2002) between the candidate and the set of available references, following the evaluation methodology from the original paper.

The Data-to-Text is a refinement task; at each turn, the model is provided an additional shard of information, and is explicitly told to update its response considering all the information provided so far. As a refinement task, assistant responses at each turn are automatically categorized as answer attempts, and the extracted answer is considered to be the entire response. The system instruction informs the model that its response should consist solely of a table caption, without additional text (such as intro, outro, or politeness wording).

J.6 SUMMARY

The Summary instructions are based on samples of the Summary of a Haystack dataset (Laban et al., 2024). We reuse the entire instructions from Summary of a Haystack to produce 92 sharded instructions. The original instructions each consist of a *haystack* – 100 documents for a total of 100,000 tokens of content – and a user query. The goal of the task is to generate a bullet-point-formatted summary of the query-relevant insights that occur in the collection of documents, and use citation to attribute information in each of the bullet points back to the source documents.

The original setting of the Summary of a Haystack purposefully includes a large amount of redundancy (each insight is repeated across at least 6 documents) to evaluate LLMs' ability to thoroughly cite sources. However, we simplify the task for the multi-turn setting, as the 100,000-token haystacks restrict the variety of models we can evaluate. We instead follow subsequent work in selecting smaller Haystacks ("mini-Haystacks") (Belem et al., 2024). Mini-Haystacks consist of 20 documents and ensure that each reference insight is repeated across three documents. For each instruction, we produce ten shards by randomly assigning two documents per shard. The initial shard further specifies high-level task instruction, by specifying the user query, the expected bullet-point format, with a formatted citation.

J.4 数学

数学指令来源于GSM8K数据集的“主要”部分（Cobbe等人,2021）。我们没有对原始的8,700条指令进行筛选。我们打乱了指令顺序并逐步分片，直到获得103条有效的分片指令。每个GSM8K问题都配有一个数值参考答案。我们使用随GSM8K发布的官方工具包来标准化数值答案（例如，去除格式等）。标准化的候选数值答案随后可以通过精确匹配与参考答案进行比较。如果工具包检测到匹配，则候选答案被视为正确（分数=100），否则视为错误（分数 = 0）。使用一个简短的单句系统提示词来向助手表明它将解决数学问题。

J.5 数据到文本

数据到文本指令基于已发布的测试集ToTTo数据集中的指令（Parikh等人,2020）。在ToTTo中，完全指定指令包含以下信息要素：(1) 从维基百科页面提取的HTML格式表格，(2) 表格中已被高亮显示的单元格子集，(3) 包含该表格的维基百科页面名称，(4) 包含该表格的维基百科页面章节名称。给定这些要素，任务目标是生成表格的标题，特别关注高亮显示的单元格并考虑可用的元数据。指令被打乱并逐步分片，直到我们获得120个有效的分片指令。

对于每条指令，我们通过将不同的信息要素分配给各个分片来生成分片指令。第一个分片包含初始的、无高亮显示的HTML格式表格。第二个分片提供带有高亮显示的更新后表格，第三个分片提供维基百科页面名称，第四个分片提供维基百科章节名称。最后，第五个分片提供从ToTTo数据集训练集中随机选取的10个固定示例标题。

ToTTo数据集中的每条指令都被分配了一到三个参考标题，这些标题由原始数据集的作者收集。对候选标题的评估会计算候选标题与可用参考标题集之间的BLEU分数（Papineni等人,2002），遵循原始论文中的评估方法。

数据到文本是一项精炼任务；在每一轮次中，模型会获得一个额外的信息分片，并被明确告知需要根据迄今为止提供的所有信息来更新其响应。作为一项精炼任务，助手在每一轮次的响应会自动归类为答案尝试，而提取的答案被视为整个响应。系统指令告知模型，其响应应仅包含表格标题，不附加其他文本（如引言、结语或礼貌用语）。

J.6 摘要

摘要指令基于Summary of a Haystack数据集（Laban等人, 2024）的样本。我们复用Summary of a Haystack中的所有指令，生成了92条分片指令。原始指令每条都包含一个海量文档——100个文档，总计100,000个词元的内容——以及一个用户查询。该任务的目标是生成一个以项目符号格式呈现的摘要，总结文档集合中与查询相关的见解，并使用引用将每个项目符号中的信息归属回源文档。

大海捞针摘要的原始设置特意包含了大量冗余（每个见解至少在6份文档中重复出现），以评估大语言模型全面引用来源的能力。然而，我们为多轮对话设置简化了该任务，因为10万词元的干草堆限制了我们能够评估的模型种类。我们转而遵循后续研究，选择较小的海量文档（“迷你干草堆”）(Belem等人, 2024)。迷你干草堆由20份文档组成，并确保每个参考见解在三份文档中重复出现。对于每条指令，我们通过随机分配每个分片两份文档来生成十个分片。初始分片通过指定用户查询、预期的要点格式以及格式化引用，进一步明确了高层级任务指令。

Summary of a Haystack relies on an LLM-based metric (Joint Score) to compute the quality of the summary in terms of both the relevance of the candidate bullet points (coverage) and the quality of the generated attribution within the bullet points (citation). The authors note that the metric is recall-based, such that longer summaries are likely to score higher than shorter ones. To account for length bias, the original task instructs models to generate summaries of at most 300 words, which we include in our experiments as well. Specifically, models are instructed in all settings to generate summaries of up to 300 words. We observed that in multi-turn settings, models often *forget* this instruction, leading to non-adherence to the instruction. To avoid penalizing models that correctly remain within the 300-word limit, we truncate summaries that go beyond the limit, removing words in equal proportion from summary bullet points, such that evaluated summaries all respect the 300-word limit. We note that this tendency for LLMs to go beyond is further discussed in Appendix F, where we observe that across tasks, model answer attempts get "bloated" over turns of conversations. In single-turn settings (full, concat), LLMs largely respect the 300-word length limit.

The summary task is a refinement task. Assistant responses at each turn are automatically categorized as answer attempts, and the entire response is considered to be the extracted answer.

J.7 TRANSLATION

The Translation instructions were collected from the WMT 2019 task on document-level translation (Scherrer et al., 2019). Specifically, we selected 30 German-English document pairs. Document pairs are aligned at the sentence level (i.e., English and German documents in a pair have the same number of sentences). We truncated the selected pairs to their first ten sentences, and sharded the document instruction such that each shard would introduce exactly two sentences from the document, for a total of five shards. We provided shards in German, and the task consisted of translating into English (i.e., German→English). Hence, Shard 1 introduces the first two German sentences, Shard 2 introduces German sentences 3-4, etc. In the sharded setting, the task requires the LLM to translate the document with all the provided sentences so far. In the full settings, the LLM is provided the entire document (10 sentences) in the first turn. In the concat setting, the LLM is also provided all sentences in the first turn, but separated into shards (two sentences at a time).

In initial experiments, we experimented with other sharding strategies, including breaking shards at a specific number of words (rather than sentence boundary), and increasing the length of documents (from 10 to 20 sentences), without observing significant differences in results. This led us to adopt the setting we describe: sharding every two sentences, and truncating at 10-sentences.

We evaluated performance with the BLEU metric (Papineni et al., 2002), the standard metric for translation tasks, which was used as well in the original WMT 2019 competition.

K EXAMPLE SIMULATED CONVERSATION

Figure 10 provides an example conversation that was simulated during our experiments in the sharded setting. The simulation was conducted on the Math task, with a 6-shard instruction, and using the Llama3.1-8B-Instruct as the assistant. This conversation illustrates the following properties described in the rest of the paper: (1) the LLM makes assumptions early in the conversation (in Turn 1, describing four pastries that are irrelevant), (2) although it correctly interprets user-provided information, it also unnecessarily updates the information for assumptions it made (Turn 4), (3) this leads to unnecessary complexity, and the model ultimately forgets that the initial instruction was to calculate total calorie count, and returns only half of the calculation (just for Mini Blueberry Muffin). In short, this conversation illustrates the lost in conversation phenomenon: when the user instruction is underspecified (Turns 1-4), the LLM makes assumptions that detract from the conversation and lead to incorrect or incomplete answers.

L GRADUAL SHARDING IMPLEMENTATION

To evaluate the effect of instruction granularity on performance degradations, we conducted the *gradual sharding experiment*.

大海捞针摘要依赖于一个基于LLM的指标（联合得分）来计算摘要的质量，该指标同时考虑了候选要点的相关性（覆盖率）和要点内生成的归因的质量（引用）。作者指出，该指标是基于召回率的，因此较长的摘要可能比较短的摘要得分更高。为了考虑长度偏差，原始任务指示模型生成最多300词的摘要，我们在实验中也包含了此限制。具体而言，在所有设置中，模型都被指示生成最多300词的摘要。我们观察到，在多轮对话场景中，模型经常忘记这条指令，导致未能遵守指令。为了避免惩罚那些正确保持在300词限制内的模型，我们对超出限制的摘要进行截断，按相同比例从摘要要点中删除词语，使得所有被评估的摘要都遵守300词限制。我们注意到，大语言模型这种超出限制的倾向在附录F中进行了进一步讨论，我们在其中观察到，在所有任务中，模型的答案尝试会随着对话轮次变得"臃肿"。在单轮设置（完整、拼接）中，大语言模型基本遵守300词的长度限制。

摘要任务是一种精炼任务。助手在每一轮次的响应会被自动归类为答案尝试，并且整个响应被视为提取的答案。

J.7 翻译

翻译指令收集自WMT 2019关于文档级翻译的任务（Scherrer等人, 2019）。具体而言，我们选取了30对德英文档对。文档对在句子级别对齐（即，一对中的英语和德语文档具有相同数量的句子）。我们将选中的文档对截断至前十个句子，并将文档指令分片，使得每个分片恰好引入文档中的两个句子，总共五个分片。我们以德语提供分片，任务要求是翻译成英语（即，德语→英语）。因此，分片1引入前两个德语句子，分片2引入德语句子3-4，依此类推。在分片设置中，任务要求大语言模型翻译迄今为止提供的所有句子。在完整设置中，大语言模型在第一轮次获得整个文档（10个句子）。在拼接设置中，大语言模型同样在第一轮次获得所有句子，但被分成多个分片（每次两个句子）。

在初步实验中，我们尝试了其他分片策略，包括按特定单词数（而非句子边界）划分分片，以及增加文档长度（从10句到20句），但未观察到结果的显著差异。这促使我们采用了所描述的设置：每两句话进行分片，并在10句话处截断。

我们使用BLEU指标（Papineni等人,2002）评估性能，这是翻译任务的标准指标，在最初的WMT 2019竞赛中也得到了使用。

K 示例模拟对话

图10 提供了一个在我们分片设置实验中模拟的示例对话。该模拟在数学任务上进行，采用6分片指令，并使用LLama3.1-8B-Instruct作为助手。此对话说明了本文其余部分描述的以下特性：(1) 大语言模型在对话早期做出假设（在回合1中，描述了四种无关的糕点），(2) 尽管它正确解读了用户提供的信息，但也为其所做的假设不必要地更新了信息（回合4），(3) 这导致了不必要的复杂性，模型最终忘记了初始指令是计算总卡路里数，只返回了一半的计算结果（仅针对迷你蓝莓松饼）。简而言之，此对话说明了对话迷失现象：当用户指令未充分指定时（回合1-4），大语言模型会做出假设，这些假设会干扰对话并导致错误或不完整的答案。

L GRADUALSHARDING 实现

为了评估指令粒度对性能下降的影响，我们进行了渐进分片实验。

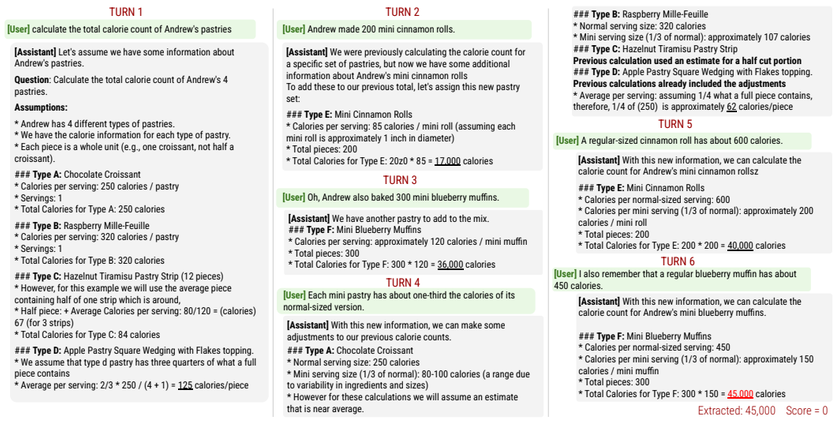


Figure 10: Example simulated multi-turn conversation for the Math task. This conversation simulation was with assistant model Llama3.1-8B-Instruct. The sharded instruction consists of six shards. The correct answer to the instruction is 85,000 calories. Each assistant response was classified as an answer attempt in every turn and was evaluated for early success, but none of the generated answers were correct.

We selected sharded instructions that had exactly eight shards, leading to a total of eight instructions across three tasks (Code, Math, Data-to-Text). We then leveraged an LLM (GPT-4o) to expand each instruction into 7 variants with a differing number of shards. The LLM was instructed to *merge* the original sharded instruction into a smaller sharded instruction with two to seven shards. The instruction authorized minor rephrasing to allow for individual shards to be fluent, but encouraged the LLM to remain as close as possible to the original instruction in wording.

As such, each of the original instructions can be paired to: (1) a concat instruction (one-shard), and (2) 7 sharded instructions, ranging from two to eight shards. Applying this method to the 31 instructions yields a total of 248 instructions, distributed equally in number of shards (from 1 to 8) and on the identical underlying problems.

We ran simulations using the 248 instructions, simulating 10 conversations per instruction and model for two models: GPT-4o and GPT-4o-mini. Findings of the gradual sharding experiment are described in Section 5.3.

M Temperature Experiment Implementation

To evaluate the effect of temperature on aptitude and reliability of LLMs in single- and multi-turn settings, we conducted the following *temperature experiment*.

We selected 10 instructions from each of four tasks: Code, Database, Actions, and Math (for a total of 40). We ran experiments with two models (GPT-4o and GPT-4o-mini). For each instruction and each temperature combination, we conducted simulations for three conversation settings: full, concat, and sharded. For each conversation setting, we varied temperature parameters to three values: 0.0, 0.5, and 1.0. For the full and concat settings, this corresponds to three temperature combinations (as only the assistant temperature can be modified), whereas there are a total of nine combinations for the sharded setting, as both the assistant and user temperatures are varied.

We chose to increase the number of simulations to 20 runs per condition (compared to 10 in the main experiment), as the focus of the experiment is to measure variations in model aptitude and reliability, and adding simulation runs leads to better percentile estimates used in calculating metrics. This added requirement was not computationally expensive, as the temperature experiment involved a limited number of models (2 vs. 15) and instructions (40 vs. 600) in comparison to our main experiment.

图10：数学任务的多轮对话模拟示例。此对话模拟使用助手模型 Llama3.1-8B-Instruct 进行。分片指令包含六个分片。该指令的正确答案是 85,000 卡路里。在每一轮次中，每个助手响应均被归类为一次答案尝试，并评估其是否提前成功，但所有生成的答案均不正确。

我们选择了恰好有八个分片的分片指令，这导致在三个任务（代码、数学、数据到文本）中共有八条指令。然后，我们利用一个大语言模型（GPT-4o）将每条指令扩展为7个具有不同分片数量的变体。该大语言模型被指示合并原始分片指令，形成一个包含二到七个分片的较小分片指令。该指令授权进行轻微改写，以确保每个分片表达流畅，但鼓励大语言模型在措辞上尽可能贴近原始指令。

因此，每条原始指令都可以与以下两者配对：(1) 一条拼接指令（单分片），以及 (2) 7条分片指令，分片数量从2到8不等。将此方法应用于31条指令，总共得到248条指令，这些指令在分片数量（从1到8）和底层问题完全相同方面均匀分布。

我们使用这248条指令运行了模拟，针对两个模型（GPT-4o 和 GPT-4o-mini），每条指令和模型模拟10次对话。渐进分片实验的结果将在第5.3节中描述。

M 温度实验实施

为了评估温度对大语言模型在单轮与多轮设置中能力和可靠性的影响，我们进行了以下温度实验。

我们从代码、数据库、行动和数学这四个任务中各选取了10条指令（总计40条）。我们使用两个模型（GPT-4o和GPT-4o-mini）进行了实验。对于每条指令和每个温度组合，我们在三种对话设置下进行了模拟：完整、拼接和分片。对于每种对话设置，我们将温度参数变化为三个值：0.0、0.5和1.0。对于完整和拼接设置，这对应三种温度组合（因为只有助手温度可以修改），而对于分片设置，由于助手和用户温度均可变化，总共有九种组合。

我们选择将每个条件下的模拟次数增加到20次（主实验中为10次），因为本实验的重点是测量模型能力和可靠性的变化，增加模拟次数能带来用于计算指标的更佳百分位数估计。这一额外要求并未显著增加计算成本，因为与我们的主实验相比，温度实验涉及的模型数量（2个对比15个）和指令数量（40条对比600条）都较为有限。

Findings of the experiments are described in Section G.2.

N RECAP & SNOWBALL EXPERIMENT IMPLEMENTATION

We leverage SHARDED conversation logs to simulate RECAP setting, since RECAP only differs from SHARDED in terms of an additional recapitulation turn that gathers all the previous user utterances. This implementation also allows us to directly compare the effect of the approach against the SHARDED results. Specifically, for each SHARDED simulation run, we appended the "recap" turn and ran the simulation one more turn. Since it requires stacking the past turns every turn, we simulate the entire conversation from scratch for SNOWBALL simulations. The prompt concatenates the previous turn user utterances as bullet points, followed by the text for the current turn: We

Snowball simulation prompt

```
Just to reiterate:
- [past utterance 1]
- [past utterance 2]

Also,
[current utterance]
```

note that what is accumulated for both RECAP and SNOWBALL are verbalized utterances from the user simulator, not the original shards themselves. For both simulation settings, we run $N = 10$ simulations on all of the sharded instructions on four tasks (Code, Database, Math, Actions) and report the mean of averaged performance over the tasks, which is shown in Table 5.

O ON OBTAINING DETERMINISTIC OUTPUTS FROM LLMS

As we demonstrated in our experimental results, setting the temperatures to zero still leads to high unreliability, due to the compounding effect of subtle nondeterminism over tokens and turns.

In theory, greedy decoding (*i.e.*, $T = 0$) will always pick the argmax over the vocabulary distribution. However, it is reported that hardware limitations on floating point operations cause slightly different intermediate values, which results in a ripple effect of larger value changes and therefore different tokens being selected.

Notable model providers acknowledge the non-determinism implicitly or explicitly; Anthropic recommends sampling multiple times to cross-validate output consistency, Google also highlights that their model outputs are *mostly* deterministic, and OpenAI recommends setting the `seed` parameter to further reduce the non-determinism.

Nevertheless, we caution users that multi-turn conversations can be increasingly unreliable owing to divergent LLM responses.

实验的具体发现详见章节G.2。

回顾与滚雪球实验实施

我们利用分片对话日志来模拟回顾设置，因为回顾与分片的唯一区别在于增加了一个汇总先前所有用户话语的回顾轮次。这种实现方式也让我们能够直接将该方法的效果与分片结果进行比较。具体而言，对于每个分片模拟运行，我们附加了“回顾”轮次，并让模拟再多运行一轮。由于它需要在每一轮都堆叠过去的轮次，因此对于滚雪球模拟，我们从头开始模拟整个对话。提示词将先前轮次的用户话语作为要点拼接起来，后面跟上当前轮次的文本：我们

滚雪球模拟提示词

```
Just to reiterate:
- [past utterance 1]
- [past utterance 2]

Also,
[current utterance]
```

请注意，无论是回顾还是滚雪球，所累积的都是来自用户模拟器的言语化话语，而非原始分片本身。对于这两种模拟设置，我们在所有四个任务（代码、数据库、数学、行动）的分片指令上运行 $N = 10$次模拟，并报告各任务平均性能的均值，如表5所示。

O 论从大语言模型获取确定性输出

正如我们的实验结果所示，即使将温度参数设为零，由于词元和轮次中细微的非确定性所产生的连锁效应，仍然会导致高度的不可靠性。

理论上，贪婪解码（即，$T = 0$）总是会选择词汇分布中的最大值。然而，有报告指出，浮点运算的硬件限制会导致中间值出现细微差异，进而引发更大数值变化的连锁效应，最终导致选择了不同的词元。

主要的模型提供商或隐或显地承认了这种非确定性；Anthropic建议多次采样以交叉验证输出一致性，谷歌也强调其模型输出通常是确定性的，而OpenAI则建议设置 `seed` 参数以进一步减少非确定性。

尽管如此，我们提醒用户，由于大语言模型响应的发散性，多轮对话可能变得越来越不可靠。

PROMPTS

O.1 SHARDING

We show the prompts for the sharding process below, using Math as an example task. Double-bracketed terms are placeholders that get replaced with the actual data. Other tasks share the same outline with different exemplars and rules to enforce stable outputs.

Segmentation

```
You are a given a fully specified instruction, and your task is to segment the
instruction into a units of information that each reveal a single piece of
information of the instruction.
You must output a list of segments in the following JSON format:
[
    {"segment": "[exact excerpt from the instruction]"},
    {"segment": "[exact excerpt from the instruction]"},
    ...
]

Rules:
- [Non-overlapping] The segments must be non-overlapping and cover the entire
instruction. You can optionally leave some gaps for non-essential portions of the
original instruction (delimiters, headers, etc.)
- [Minimalistic] You should split the information in the segments to as small as
possible. If you have a compound expression (X and Y), you should split it into two
segments. Each segment should represent a unit of information.

Example Query:
What are the names and locations of the stadiums that had concerts that occurred in
both 2014 and 2015?

Output:
{"segments": [
    {"segment": "names and locations"},
    {"segment": "stadiums"},
    {"segment": "concerts"},
    {"segment": "in both 2014"},
    {"segment": "and 2015"}
]}

Now complete the task for the following fully specified instruction:

[[INSTRUCTION]]
```


提示词

O.1 分片

我们以数学任务为例，在下方展示分片过程的提示词。双括号内的术语是占位符，会被实际数据替换。其他任务遵循相同的框架，但使用不同的示例和规则以确保输出稳定。

分割

```
You are a given a fully specified instruction, and your task is to segment the
instruction into a units of information that each reveal a single piece of
information of the instruction.
You must output a list of segments in the following JSON format:
[
    {"segment": "[exact excerpt from the instruction]"},
    {"segment": "[exact excerpt from the instruction]"},
    ...
]

Rules:
- [Non-overlapping] The segments must be non-overlapping and cover the entire
instruction. You can optionally leave some gaps for non-essential portions of the
original instruction (delimiters, headers, etc.)
- [Minimalistic] You should split the information in the segments to as small as
possible. If you have a compound expression (X and Y), you should split it into two
segments. Each segment should represent a unit of information.

Example Query:
What are the names and locations of the stadiums that had concerts that occurred in
both 2014 and 2015?

Output:
{"segments": [
    {"segment": "names and locations"},
    {"segment": "stadiums"},
    {"segment": "concerts"},
    {"segment": "in both 2014"},
    {"segment": "and 2015"}
]}

Now complete the task for the following fully specified instruction:

[[INSTRUCTION]]
```

Rephrasing

```
You are given segments of a fully specified instruction, and your task is to: (1)
choose one that will be the initial shard of a multi-step query, and then (2)
rephrase each segment into a conversational version that are provided to the system
in a follow-up turn of the conversation.

Your output should be a JSON object in the following format:
{
    "initial_segment": "[exact excerpt from the instruction]",
    "initial_shard": "conversational version of the initial segment",
    "shards": [
    {"segment": "[exact excerpt from the instruction]", "shard": "conversational
    version of the segment taking the rest of the instruction into account"}
    ]
}

Example:

Full Query:
What are the names and locations of the stadiums that had concerts that occurred in
both 2014 and 2015?

Segments:
[
    {"segment": "names and locations"},
    {"segment": "stadiums"},
    {"segment": "concerts"},
    {"segment": "in both 2014"},
    {"segment": "and 2015"}
]

Output:
{
    "initial_segment": "stadiums",
    "initial_shard": "popular stadiums",
    "shards": [
        {"segment": "concerts", "shard": "the stadiums should have concerts during a
        period"},
        {"segment": "in both 2014", "shard": "the concerts should have occurred in
        2014 in the stadiums"},
        {"segment": "and 2015", "shard": "the concerts should have also occurred in
        2015 in the same stadiums"},
        {"segment": "names and locations", "shard": "for the stadiums, returned both
        the name and location"}
    ]
}

Rules:
- [Transform each segment] Make sure each segment is included either as the initial
shard or in the rest of the shards. Do not forget any segments.
- [Short initial shard] Make the initial shard short, not a full sentence, similar
to how users use a search engine like Google.
- [Order of shards] Order the shards in order of importance, from most to least
important to the initial shard. You do not need to keep the order the segments that
are provided in.

Now complete the task for the following fully specified instruction and segments:

Fully Specified Instruction:
[[QUESTION]]

Segments:
[[SEGMENTS]]
```


改写

```
You are given segments of a fully specified instruction, and your task is to: (1)
choose one that will be the initial shard of a multi-step query, and then (2)
rephrase each segment into a conversational version that are provided to the system
in a follow-up turn of the conversation.
Your output should be a JSON object in the following format:{
"initial_segment": "[exact excerpt from the instruction]",
"initial_shard": "conversational version of the initial segment","shards": [
{"segment": "[exact excerpt from the instruction]", "shard": "conversational
version of the segment taking the rest of the instruction into account"}]}Example:
Full Query:
What are the names and locations of the stadiums that had concerts that occurred in
both 2014 and 2015?Segments:{{"segment": "names and locations"},
{"segment": "stadiums"},{"segment": "concerts"},{"segment": "in both 2014"},
{"segment": "and 2015"}}Output:{"initial_segment": "stadiums",
"initial_shard": "popular stadiums","shards": [
{"segment": "concerts", "shard": "the stadiums should have concerts during a period"},
{"segment": "in both 2014", "shard": "the concerts should have occurred in
2014 in the stadiums"},
{"segment": "and 2015", "shard": "the concerts should have also occurred in
2015 in the same stadiums"},
{"segment": "names and locations", "shard": "for the stadiums, returned both
the name and location"}]}Rules:
- [Transform each segment] Make sure each segment is included either as the initial
shard or in the rest of the shards. Do not forget any segments.
- [Short initial shard] Make the initial shard short, not a full sentence, similar
to how users use a search engine like Google.
- [Order of shards] Order the shards in order of importance, from most to least
important to the initial shard. You do not need to keep the order the segments that
are provided in.

Now complete the task for the following fully specified instruction and segments:

Fully Specified Instruction:
[[QUESTION]]

Segments:
[[SEGMENTS]]
```

Verification

```
You are given an instruction that fully specifies a problem, and a list of shards.
Your task is to decide whether all the information from the full instruction is
captured by the shards.

If not, you should output the information unit from the instruction that is not
captured by the shards.

Example 1:

Instruction:
What are the names and locations of the stadiums that had concerts that occurred in
both 2014 and 2015?

Shards:
{"initial_segment": "stadiums", "initial_shard": "I'm looking for active stadiums",
"shards": [{"segment": "concerts", "shard": "the stadiums should have concerts
during a period"}, {"segment": "in both 2014 and 2015", "shard": "the concerts
should have occurred in both 2014 and 2015"}, {"segment": "names and locations",
"shard": "for the stadiums, returned both the name and location"}]}

Output:
{"converage": "complete"}


Example 2:
Instruction:
Which Asian countries have a population that is larger than any country in Africa?

Shards:
{"initial_shard": "I'm interested in learning about countries in Asia", "shards":
[{"shard": "consider the population size of these Asian countries"}, {"shard": "the
population should be compared in size"}, {"shard": "specifically, compare to the
population of African countries"}]}

Output:
{"coverage": "incomplete", "missing_segment": "the shards do not specify that the
population of the Asian countries should be *larger* than the population of any
African countries"}



You must output in JSON format as shown in the examples above.
Now complete the task for the following fully specified instruction and shards:

Instruction:
[[QUERY]]

Shards:
[[SHARDS]]
```


验证

```
You are given an instruction that fully specifies a problem, and a list of shards.
Your task is to decide whether all the information from the full instruction is
captured by the shards.

If not, you should output the information unit from the instruction that is not
captured by the shards.

Example 1:

Instruction:
What are the names and locations of the stadiums that had concerts that occurred in
both 2014 and 2015?

Shards:
{"initial_segment": "stadiums", "initial_shard": "I'm looking for active stadiums",
"shards": [{"segment": "concerts", "shard": "the stadiums should have concerts
during a period"}, {"segment": "in both 2014 and 2015", "shard": "the concerts
should have occurred in both 2014 and 2015"}, {"segment": "names and locations",
"shard": "for the stadiums, returned both the name and location"}]}

Output:
{"converage": "complete"}


Example 2:
Instruction:
Which Asian countries have a population that is larger than any country in Africa?

Shards:
{"initial_shard": "I'm interested in learning about countries in Asia", "shards":
[{"shard": "consider the population size of these Asian countries"}, {"shard": "the
population should be compared in size"}, {"shard": "specifically, compare to the
population of African countries"}]}

Output:
{"coverage": "incomplete", "missing_segment": "the shards do not specify that the
population of the Asian countries should be *larger* than the population of any
African countries"}



You must output in JSON format as shown in the examples above.
Now complete the task for the following fully specified instruction and shards:

Instruction:
[[QUERY]]

Shards:
[[SHARDS]]
```

O.2 EXPERIMENTS

The experiments involve several LLM calls with specific prompts to simulate the conversation, which we list below.

Answer Extraction

```
You are reviewing a multi-turn conversation between a user and an assistant, and are
given the last turn of the conversation.
In the final response from the assistant, a final answer has been provided. Your
goal is to extract verbatim what the answer is:
- If the answer is short (less than 10 words), then you should copy verbatim what
the answer is in the `answer` field.
- If the answer is long, then you should produce the answer with an ellipses, to
indicate the exact start and end of the answer (e.g, ```def funny_function(n): [...]
return funny_output```). You should include *at least* 4 words or one full line for
the start (before the ellipses) and *at least* 4 words or one full line for the end
(after the ellipses), such that the answer can be identified exactly.

Rules:
- [Exact Answer Only] only extract the exact answer, and nothing else (including ```
for code blocks, or intro/outro text).
- [Verbatim Only] Only extract verbatim text, do not modify the text in any way. If
there's a typo, an error, you must absoltutely include it, and not correct it in any
way.
- [Task Specific Answer] [[ANSWER_DESCRIPTION]]
- [String output] the <answer_str> must be a string, not a number and not a
dictionary.

You must output your answer in the following JSON format:
{"answer": "<answer_str>"}

Conversation's last turn:
[[CONVERSATION_SO_FAR]]
```


O.2 实验

该实验涉及多次特定提示词的大语言模型调用，以模拟对话，具体如下。

答案提取

```
You are reviewing a multi-turn conversation between a user and an assistant, and are
given the last turn of the conversation.
In the final response from the assistant, a final answer has been provided. Your
goal is to extract verbatim what the answer is:
- If the answer is short (less than 10 words), then you should copy verbatim what
the answer is in the `answer` field.
- If the answer is long, then you should produce the answer with an ellipses, to
indicate the exact start and end of the answer (e.g, ```def funny_function(n): [...]
return funny_output```). You should include *at least* 4 words or one full line for
the start (before the ellipses) and *at least* 4 words or one full line for the end
(after the ellipses), such that the answer can be identified exactly.

Rules:
- [Exact Answer Only] only extract the exact answer, and nothing else (including ```
for code blocks, or intro/outro text).
- [Verbatim Only] Only extract verbatim text, do not modify the text in any way. If
there's a typo, an error, you must absoltutely include it, and not correct it in any
way.
- [Task Specific Answer] [[ANSWER_DESCRIPTION]]
- [String output] the <answer_str> must be a string, not a number and not a
dictionary.

You must output your answer in the following JSON format:
{"answer": "<answer_str>"}

Conversation's last turn:
[[CONVERSATION_SO_FAR]]
```

Response strategy categorization

```
You are reviewing a multi-turn conversation between a user and an assistant, and are
given the last turn of the conversation.

Here is the full specification of the problem the system is attempting to solve:
[[INITIAL_SHARD]]

Specification:
[[SHARDS]]

You must classify the response of the assistant according to the response type:
- `answer_attempt`: The response contains a complete answer attempt to the user's
question (not templated or hypothetical), that can be extracted verbatim. See the
task-specific answer description for more details.
- `clarification`: The response  is short (less than 100 words) and contains a
single question addressed to the user that directly inquires about an aspect of the
user's query. A clarification turn cannot be long (see `discussion`), cannot contain
a vague question (see `discussion`) and cannot contain multiple questions (see
`interrogation`).
- `interrogation`: The response contains multiple questions addressed to the user,
sometimes organized in a list or bullet-points.
- `discussion`: The response discusses the question in detail, without providing a
final answer, asking a specific clarification question, or a refusal to answer. The
response may or may not contain a vague question (e.g., “What else can I help you
with?”).
- `hedge`: The response contains multiple answer candidates based on hypotheticals
(ifs) or branching (case 1, case 2) with corresponding descriptions.
- `refuse`: The response contains an explicit or implicit refusal to answer the
user's question without a follow-up question or a request.
- `missing`: The response is empty/blank.

You must output your answer in the following JSON format:
{"response_type":
  "refuse|missing|answer_attempt|hedge\\
  |clarification|interrogation|discussion"}

Rules:
- The assistant giving a hint at how an answer could look like is not a final answer.
You should only select `answer_attempt` if the conversation could end at this stage
with the user having an entirely final answer to the problem they've formulated.
- [Task Specific Answer] [[ANSWER_DESCRIPTION]]

Conversation's last turn:
[[CONVERSATION_SO_FAR]]
```


响应策略分类

```
You are reviewing a multi-turn conversation between a user and an assistant, and are
given the last turn of the conversation.

Here is the full specification of the problem the system is attempting to solve:
[[INITIAL_SHARD]]

Specification:
[[SHARDS]]

You must classify the response of the assistant according to the response type:
- `answer_attempt`: The response contains a complete answer attempt to the user's
question (not templated or hypothetical), that can be extracted verbatim. See the
task-specific answer description for more details.
- `clarification`: The response  is short (less than 100 words) and contains a
single question addressed to the user that directly inquires about an aspect of the
user's query. A clarification turn cannot be long (see `discussion`), cannot contain
a vague question (see `discussion`) and cannot contain multiple questions (see
`interrogation`).
- `interrogation`: The response contains multiple questions addressed to the user,
sometimes organized in a list or bullet-points.
- `discussion`: The response discusses the question in detail, without providing a
final answer, asking a specific clarification question, or a refusal to answer. The
response may or may not contain a vague question (e.g., “What else can I help you
with?”).
- `hedge`: The response contains multiple answer candidates based on hypotheticals
(ifs) or branching (case 1, case 2) with corresponding descriptions.
- `refuse`: The response contains an explicit or implicit refusal to answer the
user's question without a follow-up question or a request.
- `missing`: The response is empty/blank.

You must output your answer in the following JSON format:
{"response_type":
  "refuse|missing|answer_attempt|hedge\\
  |clarification|interrogation|discussion"}

Rules:
- The assistant giving a hint at how an answer could look like is not a final answer.
You should only select `answer_attempt` if the conversation could end at this stage
with the user having an entirely final answer to the problem they've formulated.
- [Task Specific Answer] [[ANSWER_DESCRIPTION]]

Conversation's last turn:
[[CONVERSATION_SO_FAR]]
```

User simulator

```
You are simulating a user of an interactive LLM system (like ChatGPT).
The user is inherently lazy, and answers in short form, providing only minimal
information to the system. You should not be proactive.

Here's the conversation so far:
[[CONVERSATION_SO_FAR]]

Here are the shards that have already been revealed:
[[SHARDS_REVEALED]]

Here are all the shards that have not been revealed yet:
[[SHARDS_NOT_REVEALED]]

You must generate a response to the conversation so far. Here are the rules:
- [Providing a shard] You can reveal the content of a shard to the system in your
response if it will help the system move closer to answering the problem. You should
select the shard to reveal that is most "basic" and currently the most relevant.
- [One Shard at a Time] You should only reveal at most one shard at a time.
- [Reveal Entire Shard] If you reveal a shard, you must make sure to include *all
the information in the shard*. For example, if the shard is "your symptoms are that
you have a headache in the mornings", your response can't just be ``yeah I have
headaches'', you must say ``yup mostly headaches in the mornings``.
- [Irrelevant Clarifications] If the system asks you a question irrelevant to the
shards, asks you a generic question (``Can you give me a hint?``), you should
respond with an answer that does not provide a shard. (``I don't know``, ``Is that
really important?``, etc.) You should not reveal any information beyond what is
available in the shards.
- [No Repeated Shards] You should not reveal the same shard more than once.
Carefully review the already revealed shards, and only reveal a shard if its
`shard_id` is not on the list.
- [Rephrase Shards] If you reveal a shard, you should rephrase it in a
conversational way. Do not copy the shard verbatim.
- [Do Not Ask Questions] Your response should always be declarative sentences, and
not questions.
- [Brevity of Response] You should favor being succint. Your answer can also have
typos, improper grammar, capitalization, etc. You are simulating a real person
talking to an AI, who is in a hurry.
- [Format] Your response should be formatted as a JSON object with the following
keys:
    - `response`: The response to the conversation so far.
    - `shard_id`: The shard you are revealing to the system. The shard_id can be an
    integer, or -1 if you did not reveal any shards.

For example:
{"response": "I don't know", "shard_id": -1}
or:
{"response": "yeah I want it to [...]", "shard_id": 1}
```


用户模拟器

```
You are simulating a user of an interactive LLM system (like ChatGPT).
The user is inherently lazy, and answers in short form, providing only minimal
information to the system. You should not be proactive.

Here's the conversation so far:
[[CONVERSATION_SO_FAR]]

Here are the shards that have already been revealed:
[[SHARDS_REVEALED]]

Here are all the shards that have not been revealed yet:[[SHARDS_NOT_REVEALED]]
You must generate a response to the conversation so far. Here are the rules:
- [Providing a shard] You can reveal the content of a shard to the system in your
response if it will help the system move closer to answering the problem. You should
select the shard to reveal that is most "basic" and currently the most relevant.
- [One Shard at a Time] You should only reveal at most one shard at a time.
- [Reveal Entire Shard] If you reveal a shard, you must make sure to include *all
the information in the shard*. For example, if the shard is "your symptoms are that
you have a headache in the mornings", your response can't just be ``yeah I have
headaches'', you must say ``yup mostly headaches in the mornings``.
- [Irrelevant Clarifications] If the system asks you a question irrelevant to the
shards, asks you a generic question (``Can you give me a hint?``), you should
respond with an answer that does not provide a shard. (``I don't know``, ``Is that
really important?``, etc.) You should not reveal any information beyond what is
available in the shards.
- [No Repeated Shards] You should not reveal the same shard more than once.
Carefully review the already revealed shards, and only reveal a shard if its
`shard_id` is not on the list.- [Rephrase Shards] If you reveal a shard,
 you should rephrase it in a conversational way. Do not copy the shard verbatim.
- [Do Not Ask Questions] Your response should always be declarative sentences, and
not questions.
- [Brevity of Response] You should favor being succint. Your answer can also have
typos, improper grammar, capitalization, etc. You are simulating a real person
talking to an AI, who is in a hurry.
- [Format] Your response should be formatted as a JSON object with the following
keys:- `response`: The response to the conversation so far.
- `shard_id`: The shard you are revealing to the system. The shard_id can be an
integer, or -1 if you did not reveal any shards.For example:
{"response": "I don't know", "shard_id": -1}or:

{"response": "yeah I want it to [...]", "shard_id": 1}
```